

***CyberMESA*: Evaluation Procedures and Metrics for Charging Infrastructure Cybersecurity**

December 2025

Disclaimer

This work was prepared as an account of work sponsored by an agency of the United States Government. Neither the United States Government nor any agency thereof, nor any of their employees, nor any of their contractors, subcontractors or their employees, makes any warranty, express or implied, or assumes any legal liability or responsibility for the accuracy, completeness, or any third party's use or the results of such use of any information, apparatus, product, or process disclosed, or represents that its use would not infringe privately owned rights. Reference herein to any specific commercial product, process, or service by trade name, trademark, manufacturer, or otherwise, does not necessarily constitute or imply its endorsement, recommendation, or favoring by the United States Government or any agency thereof or its contractors or subcontractors. The views and opinions of authors expressed herein do not necessarily state or reflect those of the United States Government or any agency thereof, its contractors or subcontractors.

Authors

The authors of this report are:

Idaho National Lab (INL): Kenneth Rohde, Richard “Barney” Carlson

Pacific Northwest National Lab (PNNL): Thomas E. Carroll, Lindsey Hampton, Clayton Story, Jared Yearsley, Tommy Bryan

National Laboratory of the Rockies (NLR): Myungsoo Jun, Dana Hatic, Ryan Cryar

Acknowledgments

This report was prepared by Idaho National Laboratory for the U.S. Department of Energy, Office of Energy Efficiency and Renewable Energy, Vehicle Technologies Office.

INL/RPT-25-88240

List of Acronyms

AcCCS	Access Capabilities for CCS (AcCCS - pronounced "access" /'ak ses/) provides a flexible and inexpensive solution to enable communications testing of various EV technologies that use the CCS charging standard(s). This codebase is an example of tools and scripts capable of communicating with and emulating an EVCC and/or a SECC. https://github.com/IdahoLabResearch/AcCCS
CAN	Controller Area Network is a shared broadcast serial bus connecting several ECU together for fast and reliable communications.
CC	Common Criteria (CC) (ISO/IEC 15408). An international standard for evaluating the security properties of IT products and systems. It provides a comprehensive framework for specifying, implementing, and evaluating security features and capabilities.
CCS	The Combined Charging System is a connector and communications standard for DC charging of EV. https://en.wikipedia.org/wiki/Combined_Charging_System
CHAdeMO	Fast charging standard for electric vehicles (EVs) developed by the CHAdeMO Association, which includes major Japanese automotive manufacturers like Nissan and Mitsubishi. https://en.wikipedia.org/wiki/CHAdeMO
DIN SPEC 70121	An early release communications protocol supporting basic DC charging functionality. Almost all EV and EVSE continue to support this specification. https://www.vector.com/se/en/know-how/smart-charging/charging-standards/#c237005
ECU	Electronic Control Unit. A general term for an embedded device found in an EV responsible for a specific function within the vehicle.
E-Fuse	Electronic Fuse. A programmable device used to provide overcurrent protection and circuit protection in electronic systems. Unlike traditional fuses that must be physically replaced once blown, e-fuses can be electronically programmed and reset, offering a more flexible and reusable solution.
EV	Electric Vehicle, include plug-in hybrid electric and full battery electric vehicles.

EVCC	Electric Vehicle Communications Controller. The primary ECU in an EV responsible for communications with an EVSE.
EVSE	Electric Vehicle Supply Equipment. These devices are the charging stations providing AC or DC power to the EV for battery storage.
IEC 62443	A series of standards that address security for operational technology in automation and control systems. The series is divided into different sections and describes both technical and process-related aspects of automation and control systems security.
ISO 15118	Road Vehicles – Vehicle to Grid communications protocol. There are two current versions (-2 and -20) supported by many EVs and EVSE. https://en.wikipedia.org/wiki/ISO_15118
I2C	Inter-Integrated Circuit is a synchronous multi-master multi-slave lower speed serial bus for attaching peripheral devices to microprocessors and microcontrollers.
JTAG	Joint Test Action Group is an industry standard for verifying board designs and testing printed circuit boards after manufacturing. JTAG implements standards for on-chip instrumentation and debugging of microprocessors and microcontrollers.
LO/TO	Lockout/Tagout. A safety procedure used to ensure that dangerous equipment is properly shut off and not able to be started up again prior to the completion of maintenance or repair work. It requires that hazardous energy sources be "isolated and rendered inoperative" before work is started on the equipment in question.
MCS	Megawatt Charging System. A charging connector currently in development to support charge rates of up to a maximum of 3.75 megawatts. https://en.wikipedia.org/wiki/Megawatt_Charging_System
NACS / SAE J3400	North American Charging System. The connector and communications standard published by Tesla and being adopted by several EV and EVSE manufactures. https://en.wikipedia.org/wiki/North_American_Charging_System
PCI / Mini-PCI	Peripheral Component Interconnect. A high-speed serial interconnect for hardware components inside computer or embedded systems.

OCD	On-chip Debugging. Low level debugging of a microprocessor by connecting a debug interface like JTAG or SWD and using software to monitor and control the processor.
OEM	Original Equipment Manufacturer. Companies that produce vehicles and the original parts and components used in those vehicles.
PCB	A Printed Circuit Board is used to hold and route all of the components necessary to create an electronic device.
SAE J1772	EV charging plug and communications standard for AC charging. https://en.wikipedia.org/wiki/SAE_J1772
SECC	Supply Equipment Communications Controller. The primary communications circuit board found in an EVSE responsible for communicating with an EV.
SoS	System of Systems. A disparate collection of independent, operational systems that are integrated to achieve a common mission, often resulting in emergent capabilities not present in the individual systems.
SPI	Serial Peripheral Interface is a synchronous serial communications protocol used for short distance wired communications between integrated circuits. SPI uses a master-slave architecture for coordinated communications.
SWD	Serial Wire Debug. A two-pin interface protocol developed by ARM for debugging and programming ARM-based microcontrollers. It is a streamlined alternative to the traditional JTAG interface, offering reduced pin count and improved performance for debugging tasks
TPM	Trusted Platform Module. A secure storage chip often integrated with a microcontroller and often used for storing security information like encryption keys.
TTL	Transistor-Transistor Logic. This low voltage serial communication channel is generally provided by a UART interface and operates at the voltage of the microprocessor.
UART	Universal Asynchronous Receiver/Transmitter. This hardware device is used for serial communications between devices or components. A basic UART interface typically operates at the TTL voltage level.

Executive Summary

The grid integration of Electric Vehicle Charging Infrastructure (EVCI) is a large and complex system of systems (SoS) made up of components from many manufacturers and integrated by various companies responsible for their operation and maintenance. This complexity makes it challenging to implement security standards like IEC 62443 across the entire infrastructure. This document aims to aid in the vulnerability assessments of individual devices, such as Electric Vehicle Supply Equipment (EVSE), to support vendors in adopting and implementing security standards such as UL 2900, NIST SP 800-53 and Common Criteria (CC) (ISO/IEC 15408).

This document serves as a guide to help evaluate the cybersecurity posture of EVCI assets. It aims to provide a consistent method for evaluating EVCI, enabling cybersecurity research teams to systematically test a wide range of electric vehicle (EV) assets. The target audiences include cybersecurity research teams, vendor development teams, automotive OEMs, and third-party evaluators.

Cybersecurity teams can use this guide to identify major gaps or weaknesses in the security posture of a device. The goal is to help developers and vendors create secure products before they are deployed in the marketplace. This guide assists in creating a threat assessment and vulnerability analysis of EV equipment, including identifying how security metrics for the device might be better implemented. Scoring cybersecurity metrics poses several challenges and difficulties, including:

Dynamic Threat Landscape: Constantly evolving cyber threats require continuous updates to scoring criteria.

Contextual Variability: Security effectiveness varies based on specific hardware, use cases, and environments.

Subjectivity: Some assessments rely on expert judgment, leading to potential inconsistencies in scoring.

Complexity of Implementation: Technical details of certain security measures can be hard to assess accurately.

Interdependencies: Security measures often influence each other, complicating the scoring process.

Resource Constraints: Comprehensive evaluations require significant time, expertise, and tools.

Evolving Standards: Keeping the scoring system up-to-date with the latest standards and best practices is essential.

Quantifying Security: Translating qualitative assessments into quantitative scores is challenging.

False Sense of Security: High scores may not account for all risks, creating a false sense of security.

Balancing Detail and Usability: The system must be detailed enough for insights but simple enough for practical use.

These challenges necessitate an adaptive and thoughtful approach to scoring cybersecurity metrics, ensuring relevance, accuracy, and practical utility. This guide aims to simplify the scoring process, providing an objective method for evaluating the security posture of the device under test.

This guide assumes physical access to the assets being evaluated, as well as the necessary access to the connected electrical infrastructure (e.g., circuit breakers, cutoff switches, etc.).

Table of Contents

Executive Summary	vii
Table of Contents	ix
1 Physical Access Cybersecurity Evaluation.....	12
1.1 Metrics and Measurements.....	12
1.2 Assessment Procedures	13
Assessment Steps	14
1.2.1 Step#0 – Installation and Setup	14
1.2.2 Step#1 – Identify configuration and main components	15
1.2.3 Step#2 – De-energize EVSE and isolate all hazards.....	15
1.2.4 Step#3 – Gain physical access to all enclosures to be evaluated.....	15
1.2.5 Step#4 – Relinquish physical access of all enclosures.....	17
1.2.6 Step#5 – Evaluation of identified hardware and communications.....	17
1.2.7 Step#6 – Assessment and measurement of communications interfaces..	18
1.3 Step#7 – Assessment analysis	25
1.3.1 Metric: Physical and Anti-Tampering Protection	26
1.3.2 Metric: Hardware Based Security Features	26
1.3.3 Metric: Firmware and Boot Integrity	27
1.3.4 Metric: Side-Channel Attack Resistance.....	28
1.3.5 Metric: Authentication and Access Control	28
1.4 Analysis, Scoring, and Interpretation	29
1.4.1 Interpretation of Scores.....	30
2 Communications and Networking.....	30
2.1 Background.....	31
2.2 Structure	31
2.3 Metrics and Measurements.....	32
2.4 Assessment Procedures	33
2.4.1 Infrastructure Interface (INF).....	33
2.4.2 OCPP.....	38
2.4.3 Operator Management (OM).....	47
2.4.4 EV interface (EV)	59

2.4.5	Integration (INT).....	75
2.4.6	TLS	78
2.5	Cybersecurity Analysis and Scoring.....	84
2.5.1	Interpretation of Scores.....	88
3	EVSE Energy Management Cybersecurity Evaluation.....	89
3.1	Overview	89
3.2	Metrics, Measurements, and Scoring.....	89
3.3	Procedures.....	90
3.3.1	Open ADR	90
3.3.2	Open Charge Point Protocol (OCPP)	93
3.3.3	Messaging Queuing Telemetry Transport (MQTT)	97
3.4	Cybersecurity Analysis and Scoring.....	99
3.4.1	Interpretation of Scores.....	99
4	Summary and Next Steps	100
	References.....	101
Appendix A	- List of Hardware and Software Tools	103
Appendix B	- EVSE Hardware Overview	104
Appendix C	- Common Embedded Systems Hardware Components.....	106
Appendix D	- Firmware Analysis Resources.....	107
D.1	Software Bill of Material (SBOM)	107
D.2	Filesystem Examination	108
D.3	Static Analysis Tools.....	109
Appendix E	- Software Scanning Tools and Information	110
E.1	nmap.....	110
E.2	sslscan	110
E.3	SSH	112
Appendix F	- Standard Configuration Settings	114
F.1	TLS Parameter Configuration	114
F.2	SSH Parameters	117
Appendix G	- x.509 Certificate Issuance.....	119

List of Figures

Figure 1. Flow chart of physical access cybersecurity assessment steps	14
Figure 2: Circuit board with unpopulated 20-pin header; two means for connection enabling access	19
Figure 3. A screenshot of a VTN virtual machine.	91
Figure 4. A screenshot of a VTN virtual machine.	91
Figure 5. Packet capturing using WireShark.	92
Figure 6. OpenADR packet capturing with WireShark.....	93
Figure 7. An OCPP server using CitrineOS.....	94
Figure 8. User interface of EVERest OCPP client.....	94
Figure 9. A screenshot of OCPP packet captures with WireShark.....	95
Figure 10. A screenshot of OCPP packet captures showing OCPP version.	97
Figure 11. A screenshot of MQTT packet captures with WireShark.....	98
Figure B1: AC Level 1 or Level 2 EVSE Block Diagram.....	105
Figure B2: DC EVSE Block Diagram.....	106
Figure E1. sslscan Example Output.....	112

List of Tables

Table 1. Physical Access Evaluation Metrics and Measurements.....	12
Table 2: Number of pins for common communication interfaces.....	18
Table 3. Physical Assessment Scoring Table	29
Table 4 Communications and Networking Metrics and Measurements.....	32
Table 4. Communications and Networking Scoring Table.....	84
Table 5. EVSE Energy Management Metrics and Measurements	89
Table 6. EVSE Energy Management Assessment Scoring Table	99
Table A1: Cybersecurity Assessment Tools.....	103
Table B1: EVSE Common Components.....	104
Table C1: Common Circuit Board Communications Ports.....	106

1 Physical Access Cybersecurity Evaluation

1.1 Metrics and Measurements

To make the security analysis of EV infrastructure easier, this guide defines metrics and measurements in simple terms. A metric is an ideal or concept you aim for, like securely storing Personally Identifiable Information (PII). A measurement is an observation that relates to a metric, such as checking if the mass storage where PII is kept is encrypted.

The purpose of this assessment procedure is to identify specific measurements that relate to established cybersecurity metrics. The following metrics will help interpret these measurements and give a general idea of the security status of the targeted EV charging infrastructure. For instance, to determine if the Secure Firmware metric is being implemented, you might look for measurements such as the presence of cryptographic signatures, encrypted storage, and secure boot processes.

Table 1. Physical Access Evaluation Metrics and Measurements

Metric	Description	Measurements
Physical and Anti-Tampering Protection	Assesses the physical protections in place to prevent unauthorized physical access or tampering, such as tamper-evident seals and locking enclosures.	1. Tamper resistant or tamper evident enclosure. 2. Chip decapsulation countermeasures. 3. Self-destruct mechanisms. 4. Hardware Bill of Material analysis. 5. Rouge device detection.
Hardware Based Security Features	Evaluates the presence and effectiveness of built-in security features such as Trusted Platform Modules (TPMs), Hardware Security Modules (HSMs), and secure enclaves.	1. Trusted Platform Module in use. 2. Hardware Security Module in use. 3. Memory (storage) encryption is utilized. 4. Network transmitted data is properly encrypted. 5. Process isolation, such as containers.
Firmware and Boot Integrity	Checks the mechanisms that verify the integrity and authenticity of firmware updates, such as cryptographic signatures and secure boot processes.	1. Secure boot is implemented. 2. Secure firmware updates are implemented. 3. Signed firmware images are used. 4. JTAG ports are protected or disabled.
Side-Channel Attack Resistance	Measures the hardware's ability to withstand side-channel attacks, which exploit physical characteristics like power consumption or electromagnetic emissions	1. Debug ports (serial, JTAG) are disabled. 2. Trusted execution or containerized processes. 3. Fault injection or timing attack resistance features are present.

	to extract sensitive information.	
Authentication and Access Control	Measures the effectiveness of mechanisms that ensure only authorized users and devices can access the hardware.	1. Authentication mechanism of allowed users (RFID, Bluetooth, NFC, etc.). 2. Logging of all access attempts. 3. Authentication protocols, such as OCPP, are securely implemented (TLS, etc).

1.2 Assessment Procedures

The following steps outline a procedure to safely assess the physical components of a grid connected electrical infrastructure device. The most prevalent EV infrastructure is the Electric Vehicle Supply Equipment (EVSE), or charging stations, that are used to recharge an EV battery. EVSEs operate at extremely high voltages and power levels and may expose assessment teams to dangerous conditions. As such, this style of assessment typically requires an engineer or subject matter expert familiar with the equipment so that it can be accessed in a safe manner. The cybersecurity professionals shall work closely with the engineering staff during all steps of the process. As an example, this procedure illustrates the assessment of a modern EVSE capable of DC smart charging.

Preparing for an assessment... See **Error! Reference source not found.** for a list of hardware and software to gather for the assessment.

WARNING: HIGH VOLTAGE: HAZARD OF ELECTRICAL SHOCK. DISCONNECT INCOMING POWER BEFORE WORKING ON THIS EV Charging Infrastructure.
WARNING: Coordinate with the equipment owner after turning off the input power supply to ensure sufficient time has passed for dangerous electronics to discharge before opening the enclosure, performing maintenance, or performing an inspection. Otherwise, there is the danger of electric shock.

The procedures detailed in this document follow a specific order. **Error! Reference source not found.** is a diagram representation of the flow path of the evaluation procedures and the necessary steps to ensure a safe and successful security evaluation.

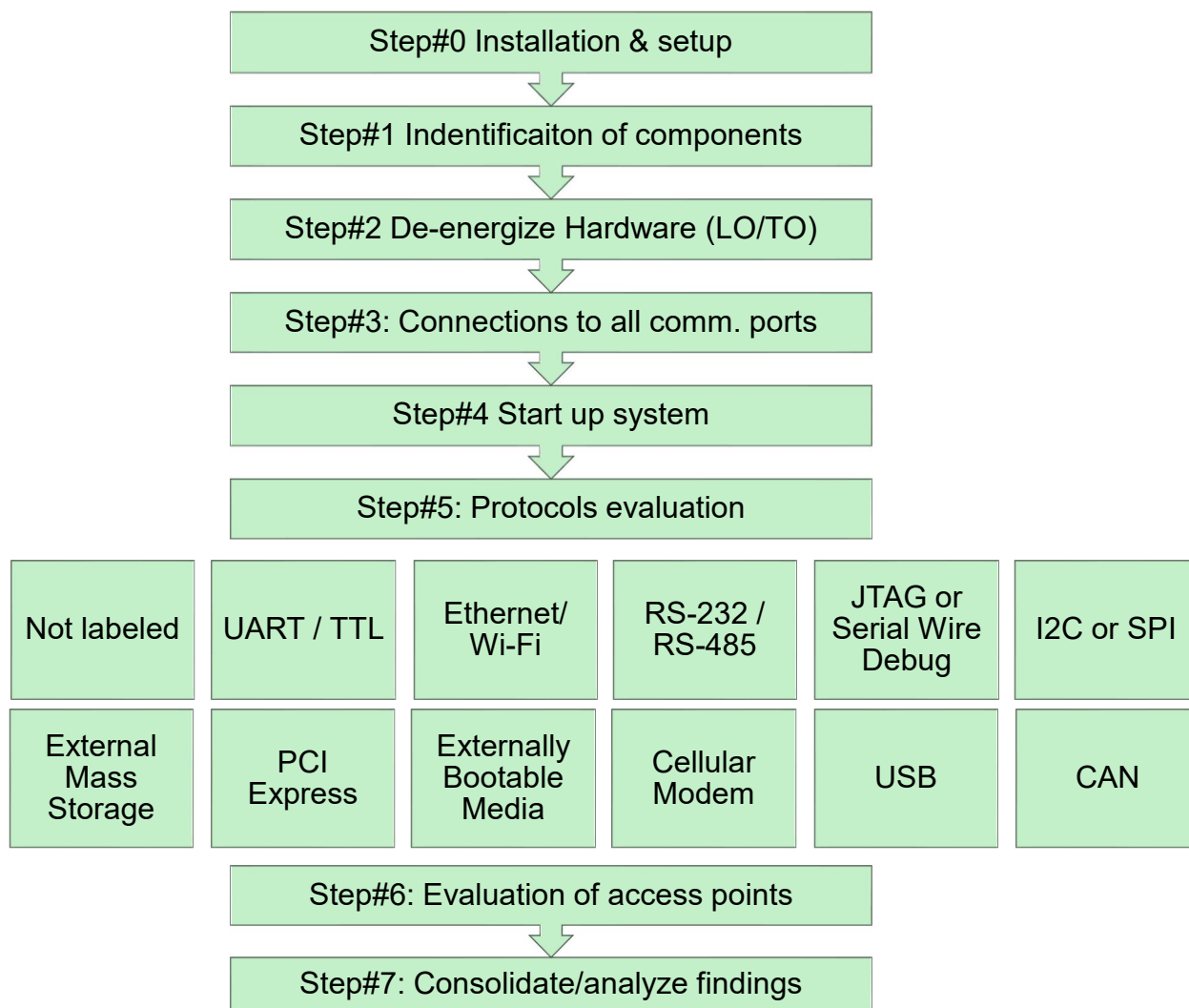


Figure 1. Flow chart of physical access cybersecurity assessment steps

Assessment Steps

1.2.1 Step#0 – Installation and Setup

1. Receive, install, and commission the EVSE(s) and all required enclosure(s) per the manufacturer's specifications and requirements
2. Verify proper working order according to the manufacturer requirements using a compatible electric vehicle.

3. Ensure all settings and configurations are in a deployed intended mode (rather than debug mode, or end-of-line test mode, or display mode, etc.)

1.2.2 Step#1 – Identify configuration and main components

1. Identify the EVSE type and configuration
 - a. Note the form of power transfer: AC or DC or Wireless
 - b. Note the configuration:
 - i. Standalone (one EVSE with one or multiple cabinets)
 - ii. Multi-EVSE installation configuration where many EVSEs utilize one communication or control hub (which may be embedded within one of the EVSE) for various communication or control functions
2. Identify EV cable types utilized (CCS-1, CCS-2, CHAdeMO, NACS, MCS, etc.)
 - a. Note the number of each cable type
 - b. Note if the cable be accessed without authorization (i.e. locked into pedestal/dispenser without authorization or payment)

See Error! Reference source not found. for a detailed list of EVSE components and block diagrams.

1.2.3 Step#2 – De-energize EVSE and isolate all hazards

1. De-energize the EVSE and all associated communication enclosures to be evaluated per the manufacturer's specifications and requirements
2. Use LO/TO from NFPA-70E to enable safe access of the EVSE(s) and enclosure(s)
 - <https://www.nfpa.org/codes-and-standards/nfpa-70e-standard-development/70e>
 - "Exclusive control" is an acceptable method included in NFPA-70E. Exclusive control means the authorized employee working on a machine or equipment has the sole authority and ability to prevent others from re-energizing or starting it during servicing or maintenance.

1.2.4 Step#3 – Gain physical access to all enclosures to be evaluated

1.2.4.1 Step#3A – Prepare EVSE for system, sub-system, component, and control board testing in-situ

1. Without using the manufacturer's supplied key, attempt to open the EVSE(s) and enclosure(s)
 - Note the tools or equipment required to gain physical access (screwdriver, lock pick, no tool required, etc.)
 - Note the duration required to gain physical access (seconds)
 - Note any tamper evident devices to assist in identifying if or when the EVSE enclosure has been accessed.
2. If after 900 seconds (15 min.), physical access is not successful (ex. secured with a robust uniquely keyed lock), use the manufacturer's supplied key to gain access to the EVSE(s) and enclosure(s)

- This 15-minute time period also allows for any residual stored electrical energy to be dissipated. (i.e. capacitance within the EVSE)
- 3. Open the EVSE(s) and enclosure(s)
 - Conduct a zero-energy verification, as required by NFPA-70E
 - Locate the main components of the EVSE(s) and enclosure(s)
 - Access control components and controls
 - Main control boards
 - EVSE to EV communication board (i.e. for CHAdeMO CAN, CCS PLC, or other)
 - Thermal management system, communications, and controls
 - Power electronics hardware, communications, and controls
 - Verify power transfer routing between main components
 - AC input hardware, isolation / transformer, power converters, relays / contactors, DC output cable connection, etc.
- 4. Identify communication connection points on all control boards
 - USB, UART, Serial port, Ethernet port, CAN bus terminal, etc.
 - Identify communication systems that do not have an available access port

See Appendix C for a list of common interfaces found inside an EVSE.
- 5. Connect signal cabling to all communication connection points within the EVSE
 - Route this signal wiring to exit the EVSE(s) and enclosure(s), while avoiding close proximity to high power components and wiring. These signal cables will be used to monitor and evaluate communications of the EVSE(s) and enclosure(s) after physical access has been secured (i.e. door closed, lid in place, etc.), and the equipment has been powered up and fully operational.
 - Use twisted shielded signal cables when possible
 - If possible, route the signal cables through strain-relief openings in the EVSE(s) and enclosure(s). This is in contrast to routing the signal cable through the open door currently accessed. If not possible, carefully plan the signal cable routing to minimize the risk of damaging or compromising the signal cable integrity.
 - Ensure the signal cables have >2m of length external to the EVSE(s) and enclosure(s). This will enable ergonomic utilization of these signal cables adjacent to the system(s) being evaluated.

1.2.4.2 Optional Step 3B – Remove Control Board (Test Article) from Enclosure for Benchtop Assessment

For evaluation focused solely on a single control board, without the evaluation of its communication with other systems, it is possible to remove the control board from the enclosure and operate its controls on a workbench using a benchtop power supply.

1. After completing step#0, #2, #3.2, and #3.3 to gain safe entry into the enclosure, identify and note all of the connections on the control board intended to be evaluated on the workbench.

2. Identify or determine the electrical power required by the control board for proper operation.
 - a. The voltage requirement may be labeled on the control board.
 - b. Alternatively, you may need to connect measurement wiring to the power terminals of the control board, and route this wiring outside of the enclosure. Then proceed to step 4 of these procedures to relinquish physical access of the enclosure, power up the system, and determine the voltage required by the control board. (i.e. 5V, +/-12V, or 24V, etc.)
 - c. Once the voltage requirement is determined, repeat steps# 2, 3.2, and 3.3 to again safely enter the enclosure.
3. Remove the control board from the enclosure
4. Setup the control board on a non-conductive workbench surface with a power supply providing the correct voltage as determined in the previous step of this procedure. Continue to Step#5.

1.2.5 Step#4 – Relinquish physical access of all enclosures

1. Close and secure all EVSE(s) and enclosure(s) to the manufacturer's configuration for operations
 - a. Secure the areas of access used in the previous steps (close the door(s), latch the lids, etc.)
 - b. Ensure the EVSE(s) and enclosure(s) are returned to the manufacturer's recommended configuration (i.e. locked door, lids in place, etc.)
 - c. Power up the EVSE(s) and enclosure(s) per the manufacturer's recommended procedures.
 - d. Ensure the system is functioning the same as previously observed prior to physical access and the addition of signal cables. Verify proper working order according to the manufacturer requirements using a compatible electric vehicle.

1.2.6 Step#5 – Evaluation of identified hardware and communications

1. Determine the functionality and accessibility for each remote communication signal cable connected in Step# 3.5.
2. Determine communication protocols on unidentified communication ports (i.e. serial port RS485 or RS232, CAN bus baud rate, etc.)
3. Document the findings of each hardware component or communications interface.

Note: Use the guidance found in Appendix A section of this document to safely and accurately assess the various communications access points of the system.

1.2.7 Step#6 – Assessment and measurement of communications interfaces

For the various communication interfaces within the charging infrastructure, use the following information for each type of interface.

1.2.7.1 Unknown or Unpopulated Headers

The printed circuit boards (PCB) inside EVSEs can often have unlabeled or unpopulated headers. These headers are often on the board for production testing prior to the device leaving the factory. Some guesswork is involved here, and care must be taken to not potentially damage the circuit board components.

The first logical step is to check the header for a ground pin(s). This is done with a voltmeter looking for continuity between the header pins and a known ground source. Once the ground pin(s) are identified, an oscilloscope should then be used to observe any signaling that may be present. An oscilloscope and voltmeter are the primary tools for attempting to identify the unpopulated header.

If signaling is identified on any of the pins, tools like a JTAGulator¹ or Bus Pirate² can help identify the type of interface connected to the header. Take precautions here to ensure that proper voltages are always used so that damage to the PCB or assessment equipment does not occur.

The number of pads or pins associated with a header may provide insight into its role as detailed in **Error! Reference source not found.** Communication port headers might be unpopulated. To enable access to the port, attaching a multi-pin header, or individual wire leads via soldering enables access to this port as shown in **Error! Reference source not found.** Often the headers are of a common format, such as the TI 20-pin header.

Table 1: Number of pins for common communication interfaces

Number of Pins	Common Usage
3-5	TTL/UART, RS-232, or Serial Wire Debug (SWD)
4-6	RS-232, RS-485, or JTAG
12-20	JTAG

¹ JTAGulator - <https://grandideastudio.com/portfolio/security/jtagulator/>

² Bus Pirate - <https://buspirate.com/>

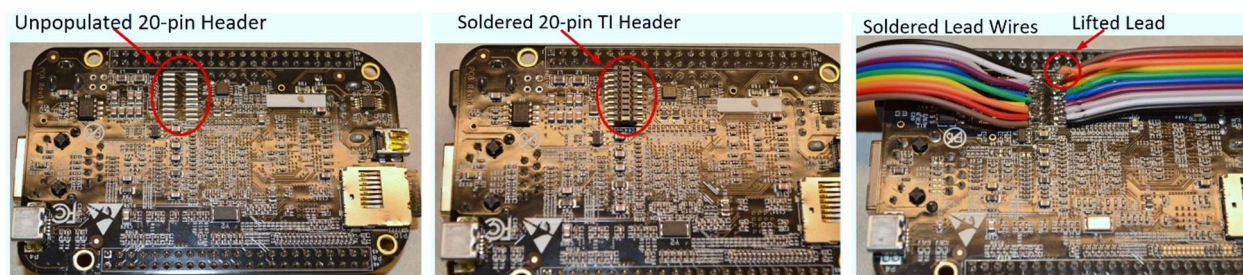


Figure 1: Circuit board with unpopulated 20-pin header; two means for connection enabling access

1.2.7.2 UART/TTL

This is a basic serial interface connected directly to a microcontroller. It will operate at the chip voltage level (1.8, 3.3 or 5 VDC) and needs to be verified by an oscilloscope. To connect to and interface with this connection, use a FTDI basic³ interface that is labeled as TTL compatible with the observed voltage.

Oftentimes TTL serial ports provide debug console output from the connected microcontroller. This may provide a login shell or boot-time output when the system is powered on. Determine the baud rate of this serial port and connect the FTDI while ensuring the host RX and TX wires are crossed with the interface RX and TX ports.

Use a serial terminal program to connect to the FTDI using the device name (`/dev/ttyUSBx` or `/dev/ACMx`) created by Linux when it was attached to the assessment laptop. In Linux, two common terminal programs are `minicom` and `screen`. For example, to connect to the TTL interface using `screen`, use the following command:

```
screen /dev/ttyUSB0 115200
```

Determine if this serial connection provides interactive Linux shell access, or at a minimum, if you can see the Linux boot log when the device is powered on. When you boot the device, save all of the log information to gain further details about the host (kernel version, flash memory details, etc.).

1.2.7.3 Serial RS-232/RS-485

RS-232 and RS-485 ports are most often used for long distance digital connections. There are often unused serial ports in a system that may still provide some information or system level access. A serial port might provide console terminal access to the host (Linux, etc.), or it may be used for sending data between other hosts.

³ FTDI Basic - <https://www.sparkfun.com/sparkfun-ftdi-basic-breakout-3-3v.html>

Use an oscilloscope to monitor the signaling and voltages used on the serial port to determine if it is RS-232 or RS-485. RS-232 can use as few as three pins or wires for asynchronous communications. RS-485 can use 3-6 pins depending on the type of serial (e.g. half-duplex or full-duplex). Don't forget to cross the RX and TX pins between the header and assessment laptop interface for RS-232 connections. The RS-485 pins are not crossed but connected as labeled (TX+, TX-, RX+, RX-, Data (A-), Data (B+), etc.).

A logic analyzer, such as a Saleae Logic⁴, can also be used to observe and decode the serial communications. Ensure the logic analyzer is capable of the higher voltages found in these serial connections (often +/- 12 VDC or higher). Many logic analyzers can only be used at 5 VDC or less. This is often the easiest method to monitor and decode serial connections due to their typical use as point-to-point connections.

1.2.7.4 I2C or SPI

These serial interfaces are generally used to communicate between a microcontroller and peripheral devices. They are not typically found on PCB headers, but a logic analyzer can be used to determine if the header pins are indeed providing I2C or SPI connectivity.

Monitoring I2C and SPI might provide insight into data moving from peripherals to the microcontroller. This can include very interesting data, such as Flash storage memory access or Trusted Platform Modules (TPM). Several Logic Analyzers can capture and decode these communications, but other tools like a Bus Pirate and software on the assessment laptop can also be used effectively.

1.2.7.5 JTAG or Serial Wire Debug

On-chip debug ports are a very effective method for interrogating a microcontroller. The most common debug ports available in today's electronics are either JTAG or Serial Wire Debug (SWD). Either of these interfaces may provide direct access to the microcontroller and other peripherals on the PCB. Some PCB manufactures will leave JTAG or SWD headers available on the PCB, but the JTAG interface is disabled after production. This can lead to frustrating troubleshooting of the JTAG connection.

⁴ Saleae Logic - <https://www.saleae.com/products/saleae-logic-pro-16>

There are several JTAG and SWD interfaces available, including Segger JLink⁵, Bus Pirate, JTAGulator, and the FTDI-FT232H Breakout⁶ board. Any of these interfaces are compatible with debug software such as GDB and OpenOCD⁷.

To successfully connect a SWD/JTAG interface to a microcontroller, the interface pins must be correctly identified. This can be challenging if you are dealing with a non-standard header or a header that is not labeled. Tools like the JTAGulator, or the replacement BlueTag⁸ project, can aid in enumerating JTAG pins and identifying each of the required connections (TDI, TDO, TMS, etc.).

OpenOCD provides all the low-level functions to interrogate and manipulate a microcontroller. This software can interrogate the JTAG chain for other connected devices and allows access to flash memory and the runtime parameters of the system (e.g. pause and step through execution).

1.2.7.6 External Mass Storage

Embedded systems must store their firmware somewhere in non-volatile memory. This can vary greatly from one system to the next, so care must be taken to inspect the system and identify all persistent storage devices.

Some systems will store the firmware in persistent memory located in the microcontroller, but it is also common to have external memory devices such as Flash, SD, SSD, or even SATA devices. In extreme situations, analyzing a system may only be possible by directly accessing the external storage and reading the memory contents directly from the device. This is a very advanced reverse engineering activity and requires special tools, and often the external storage is encrypted. External Flash may also be located on the JTAG chain of the microcontroller. If JTAG is enabled, and the Flash is accessible, the device may be accessible for reads and writes using tools like OpenOCD.

NOTE: Careful planning is essential before performing read/write access, as many microcontrollers are often part of a larger System on Chip (SoC) designed board. SoC architectures frequently include memory management interfaces that must be properly initialized before attempting to read or write to the Flash memory. Failure to do so can leave the SoC or microcontroller in an uninitialized and unbootable state.

⁵ Segger JLink Base - <https://www.segger.com/products/debug-probes/j-link/models/j-link-base/>

⁶ FTDI FT232H Breakout - <https://www.adafruit.com/product/2264>

⁷ OpenOCD - <https://openocd.org/>

⁸ BlueTag - <https://github.com/Aodruez/blueTag>

1.2.7.7 PCI Express

Mini-PCI ports are occasionally found on embedded systems, and the PCI bus might provide direct access to system memory. This is an advanced hardware hacking technique, but it is worth investigating if the system is equipped with a PCI interface. Tools such as PIC-Leech⁹ may allow direct memory access to the system being tested.

1.2.7.8 Externally Bootable Media

SD, Micro-SD, and USB ports might provide a means with which to boot a system from removable media. A boot button might exist that changes the default boot behavior from booting from flash to booting from an external storage device. This can be a lengthy process if the test system is using a less common microcontroller, but several of the major microcontroller manufacturers provide basic bootable media. If booting from external media is accomplished, direct access to the on-board flash memory is likely possible.

1.2.7.9 Ethernet/Wi-Fi

Local networking options are often available to provide connectivity between devices. A common deployment strategy includes equipping one device with a cellular modem and then allowing the neighboring devices to use the “gateway” device for shared internet access.

Dedicated Ethernet cables and Ad-Hoc Wi-Fi networks are used often to allow all site devices to connect back to vendor or network operator servers. These network connections need to be monitored for good security practices. Scanning the Ethernet or Wi-Fi interface is also needed to determine if remote ports or services are available from these connections.

If you can connect to a local Ethernet or Wi-Fi network, start by performing passive monitoring of the network using tools like Wireshark or TCP dump. Monitor the network as the system is restarted and look for valuable information. Analyze the connection for any clear text protocols, etc.

Following the passive network monitoring, active interrogation of the network should then be done using tools like Nmap. This can reveal open ports or services that may not be fully secured (e.g. Telnet, HTTP, etc.).

1.2.7.10 Cellular Modem

A cellular modem might be a device directly attached to a PCB, or it might be an external device connected to an internal Ethernet network. If the modem is a separate

⁹ PIC-Leech software: <https://github.com/ufrisk/pcileech>

network-attached device, monitor the network traffic to and from the modem by using a network hub or a network tap. If the cellular modem is directly attached to the PCB, monitoring the network traffic will likely first require root access to the assessment system.

If access to the network traffic leaving the test system through the cellular modem is available, the traffic needs to be monitored for security best practices. There are often several connections used by these devices including remote access for the vendor and the network operator. Ensure these connections are properly wrapped in secure protocols like SSL/TLS, SSH or VPN. Any clear text protocols used should be noted.

1.2.7.11 USB

USB ports are often provided internally to the charging equipment cabinet. These ports can be used for several purposes including USB Host features and debug consoles. Depending on the USB port type, use your creativity and see what types of USB devices might function with the charging equipment. A few common tools for testing USB capabilities include the Bash Bunny¹⁰ or the Rubber Ducky¹¹. These pen testing tools will connect to a USB host port and impersonate different USB device types with the intent of gathering system information or even local system access.

Another common test for a USB host port is to add a USB network interface device to the charging equipment. Connect the charging equipment to an assessment laptop with a dedicated Ethernet cable and use the assessment laptop to provide traditional network service like DHCP and DNS. If this is successful, the charging equipment will believe this new network connection is a routable connection to the Internet. The assessment team can now monitor all the outbound traffic as well as perform network scans.

If the USB port is a USB-C connector, it can operate in Host or Peripheral mode, but the capabilities are determined by the implementation of the charging equipment. Attempt to connect various devices and determine what the charging equipment allows to function.

USB mini and micro connectors may allow the charging equipment to connect to the assessment laptop as a peripheral device. It is not uncommon for this connector to provide a serial port that is connected to the debug console of the charging equipment. If this is the case, use tools like `minicom` or `screen` to connect and monitor the output.

¹⁰ Bash Bunny - <https://shop.hak5.org/products/bash-bunny>

¹¹ Rubber Ducky - <https://shop.hak5.org/products/usb-rubber-ducky>

1.2.7.12 *Controller Area Network (CAN)*

A Controller Area Network (CAN) is a shared serial Bus that is used to interconnect several compute modules for dedicated high-speed communications. CAN is often recognized in the automotive industry, but it is also used quite extensively in other areas requiring automation and control. A typical CAN Bus connects devices together with a dedicated pair of twisted wires. Several systems provide this connectivity using a pair of wire inside a typical CAT-5/CAT-6 Ethernet cable rather than running an unshielded set of twisted pair wire.

CAN Bus communications are quickly identified with an oscilloscope. Connect an oscilloscope to each of the wires in a CAN Bus and monitor them simultaneously. One wire will be pulled high with the signal pulling the voltage low, and the other wire is held low with the signal pulling the voltage high. The two signals will look like mirror images of one another and represent what is known as CAN high and CAN low signals. This is known as a differential signal.

CAN is often used in high-power charging equipment for the critical control of the power electronics. After determining the proper baud rate, connect a CAN interface to an assessment laptop and use common tools to monitor the CAN traffic. This includes the Linux `socketcan` utilities and even Wireshark. Monitor the CAN bus and see if the traffic can be interpreted in some manner. Some manufacturers may even use a higher-level protocol on top of CAN, like the CANOpen¹² protocol. This can make message decoding a bit easier.

1.2.7.13 *MQTT*

MQTT¹³ is a lightweight, publish–subscribe, machine-to-machine network protocol for message queue/message queuing service. It is designed for connections with remote locations that have devices with resource constraints or limited network bandwidth, such as Internet of Things (IoT) devices. It must run over a transport protocol that provides ordered, lossless, bi-directional connections—typically, TCP/IP.¹⁴

Some charging equipment uses MQTT for managing internal electronic components, such as power electronics. This is typically done on an internal Ethernet network found only inside of the charging equipment cabinet. Installation of a network tap or a network hub will allow you to monitor the internal network traffic and determine if MQTT is being used and if it is used with secure protocols (e.g. SSL/TLS).

¹² CANOpen - <https://en.wikipedia.org/wiki/CANOpen>

¹³ MQTT - <https://mqtt.org/>

¹⁴ MQTT Wikipedia - <https://en.wikipedia.org/wiki/MQTT>

If the identified MQTT traffic is not secured properly, and assessment laptop can be added to the network as an additional subscriber. This will allow an assessment team to send arbitrary MQTT messages on the network that may affect the charging system.

1.2.7.14 *RFID/NFC*

Several EVSE manufacturers and owners use RFID or NFC communications to authorize charging sessions. There are many documented weaknesses using these technologies, specifically cloning a user's RFID or NFC and gaining unauthorized access.

If RFID or NFC is present, use tools such as the Flipper Zero¹⁵ or the Proxmark Platform¹⁶ to evaluate the security of the provided authentication methods. Examine the provided RFID/NFC tokens for security features such as encryption.

1.2.7.15 *CCS/NACS Digital Communications*

If the charging station is equipped with a CCS or NACS cordset, digital communications are used between the vehicle (Electric Vehicle Communications Controller – EVCC) and the charging station (Supply Equipment Communications Controller – SECC). Tools like the open-source AcCCS¹⁷ system and the Micro Megawatt Charger¹⁸ provide direct access to these digital communications for assessment purposes.

Although the CCS/NACS communications are supposed to be a direct point-to-point connection to the SECC and EVCC, weaknesses in the implementation of this internal network are often identified and lead to unintended access to other compute modules located inside a charging station. Tools like AcCCS and the Micro Megawatt Charger allow assessment teams to test this network access and determine if other systems are reachable through this dedicated network.

1.3 Step#7 – Assessment analysis

Use the following scoring system to document and rank the cybersecurity posture of each interface or hardware component.

The provided scoring system is designed to evaluate the implementation of critical hardware security measures through a structured and quantifiable approach. Each security metric is assessed using specific measurements, which are rated on a scale from 0 to 2 points: Fully Implemented (2 points), Partially Implemented (1 point), and

¹⁵ Flipper Zero - <https://flipperzero.one/>

¹⁶ Proxmark Platform - <https://www.proxmark.com/>

¹⁷ Access Capabilities for CCS (AcCCS) - <https://github.com/IdahoLabResearch/AcCCS>

¹⁸ Micro Megawatt Charger - <https://shop.pionix.com/products/umwc-micro-mega-watt-charger>

Not Implemented (0 points). By summing the weighted points for each measurement, a total score is calculated. This total score reflects the overall security posture of the hardware. The scoring system provides a clear interpretation of the results, enabling organizations to identify strengths and weaknesses in their hardware security implementations and prioritize areas for improvement.

Scoring Criteria for each assessment metric is:

- **Fully Implemented:** 2 points
- **Partially Implemented:** 1 point
- **Not Implemented:** 0 points

1.3.1 Metric: Physical and Anti-Tampering Protection

1.3.1.1 Tamper resistant or tamper evident enclosure. Measurements include locking devices, tamper evident seals, and intrusion notifications (door switches).

○ Score: _____

1.3.1.2 Chip decapsulation countermeasures. Measurements include epoxy coatings, optical coatings to prevent IR or X-ray imaging, etc.

○ Score: _____

1.3.1.3 Self-destruct mechanisms. Measurements include volatile memory, e-fuses and electronic pulse generation.

○ Score: _____

1.3.1.4 Hardware component labeling: Use of the Bill of Material (BOM) by the attacker may enable reverse engineering or identification of known vulnerabilities. Measurements include the removal of component identification to reduce the ease of reverse engineering of the device hardware.

○ Score: _____

1.3.1.5 Rogue device detection. Measurements include secure boot, firmware verification, and restriction of allowed USB devices.

○ Score: _____

1.3.2 Metric: Hardware Based Security Features

1.3.2.1 Trusted Platform Module (TPM) in use. Measurements include use with secure boot, encrypted storage or key management.

○ Score: _____

1.3.2.2 Hardware Security Module (HSM) in use. Measurements include proper implementation of cryptographic keys and key management for secure communications (e.g. PKI, etc.).

○ Score: _____

1.3.2.3 Memory (storage) encryption with unique keys (not a global key) is utilized. Measurements include full encryption of mass storage devices and firmware flash memory.

○ Score: _____

1.3.2.4 Network transmitted data is properly encrypted. Measurements include the use of SSL/TLS and proper public and private key management.

○ Score: _____

1.3.2.5 Process isolation, such as containers. Measurements include services running without root/system level access, sandboxed in a virtual environment, or running in a dedicated chroot environment.

○ Score: _____

1.3.2.6 Trusted execution or containerized processes. Measurements include the use of containers or other virtual environments to isolate critical processes.

○ Score: _____

1.3.3 Metric: Firmware and Boot Integrity

1.3.3.1 Secure boot is implemented. Measurements include secure boot loaders and firmware verification.

○ Score: _____

1.3.3.2 Secure firmware updates are implemented. Measurements include over-the-air updates, roll-back protection, and signed/encrypted firmware.

○ Score: _____

1.3.3.3 Signed firmware images are used. Measurements include restriction of firmware to only images that are cryptographically signed and verified.

○ Score: _____

- 1.3.3.4 All debug ports (JTAG, SWD, UART, etc.) are protected or disabled. Measurements include the presence of headers on a circuit board (populated or unpopulated) and the ability to query the JTAG chain.

○ Score: _____

- 1.3.3.5 Debug ports (JTAG, SWD, UART, etc.) are disabled. Measurements include JTAG disabled by the microchip or an e-fuse in the chain.

○ Score: _____

1.3.4 Metric: Side-Channel Attack Resistance

- 1.3.4.1 Robustness against Fault Injection by protection of critical data and decisions. Measurement by code review and/or testing.

○ Score: _____

- 1.3.4.2 Timing attack resistance. Time constant password checking (including JTAG), encryption, and cryptographic signature generation. Measurement by code review and/or testing.

○ Score: _____

- 1.3.4.3 Fault injection or timing attack resistance features are present. Measurements include error detection and correction methods, checksums, signature verifications, and watchdog timers.

○ Score: _____

1.3.5 Metric: Authentication and Access Control

- 1.3.5.1 Authentication mechanism of allowed users (RFID, Bluetooth, NFC, etc.). Measurements include the use of authentication service, such as OCPP, to validate authorized access of system services.

○ Score: _____

- 1.3.5.2 Logging of all access attempts. Measurements include the use of remote logging of activities on the system, such as OCPP or remote syslog.

○ Score: _____

- 1.3.5.3 Authentication protocols, such as OCPP, are securely implemented (TLS, etc.). Measurements include the use of secure connectivity methods for remote management of the device.

○ Score: _____

1.4 Analysis, Scoring, and Interpretation

From the scoring results determined in the previous section, enter the values into the “Raw Score” column in the following table. Then using the weighting factor values, calculate the “Final Score” for each analysis criteria. Then calculate the sum “Total Score” for this section of assessment.

Table 3. Physical Assessment Scoring Table

<u>Physical Access Assessment</u> <u>Scoring Table</u>	<i>Raw Score (0,1,2)</i>	<i>Weight Factor</i>	<i>Final Score</i>
1.3.1			
<u>Physical and Anti-Tampering Protection</u>			
Tamper resistant or tamper evident enclosure		2	
Chip decapsulation countermeasures		0.3	
Self-destruct mechanisms		0.2	
Hardware component labeling		0.5	
Rogue device detection		2	
1.3.2			
<u>Hardware Based Security Features</u>			
Trusted Platform Module (TPM) utilized		1	
Hardware Security Module (HSM) utilized		1	
Memory (storage) encryption		5	
Network transmitted data encrypted		3	
Process isolation		5	
Trusted execution or containerized processes		1	
1.3.3			
<u>Firmware and Boot Integrity</u>			
Secure boot implemented		5	
Secure firmware updates implemented		3	
Signed firmware images utilized		1	
All debug ports protected or unpopulated		3	
All debug ports disabled		2	
1.3.4			
<u>Side-Channel Attack Resistance</u>			

Robustness against Fault Injection		1	
Timing attack resistance		1	
Fault injection or timing attack resistance		1	
1.3.5			
<i>Authentication and Access Control</i>			
Authentication mechanism of allowed users		5	
Logging of all access attempts		2	
Authentication protocols		5	
Total Score		-	

1.4.1 Interpretation of Scores

- **84- or greater points:** Excellent implementation of security measures.
- **66-83 points:** Good implementation, with minor areas for improvement.
- **50-65 points:** Moderate implementation, requiring significant improvements.
- **25-49 points:** Poor implementation, with critical areas needing attention.
- **0-24 points:** Very poor implementation, indicating severe security risks.

This scoring method allows for a systematic evaluation of the hardware security level based on the defined metrics and measurements.

2 Communications and Networking

This chapter focuses on the communications and networking aspects of DC fast chargers, providing a general overview before later chapters address protocol-specific procedures.

Reliable communication is essential for the secure and efficient operation of electric vehicle (EV) charging infrastructure. This section presents test cases that evaluate a charger's ability to establish, maintain, and recover communications with external

systems such as the Charging Station Management System (CSMS), utility interfaces, and other networked services.

2.1 Background

A DC fast charger employs multiple interfaces to fulfill its operational purposes. This section generalizes and describes these interfaces and their associated responsibilities.

A DC fast charger has two primary interfaces:

- EV interface – interface to the EV is governed by ISO 15118-2 and ISO 15118-20, which specify EV–EVSE communication for charging control, Plug & Charge authentication, and energy transfer negotiation.
- Infrastructure interface – interface to the infrastructure and charging network is typically implemented using the Open Charge Point Protocol (OCPP), enabling communication with the CSMS for configuration, monitoring, and control functions.

Beyond these two core interfaces, a charger will typically have additional communication interfaces serving distinct operational roles:

- Operator Management – In addition to OCPP, most chargers expose a management interface (SSH or a web UI) for status reporting, firmware updates, diagnostics, and remote control, and they also provide local access for provisioning, configuration, and troubleshooting.
- Manufacturer – Provides the OEM with remote access for maintenance, patching, and post-deployment servicing, often implemented over a dedicated cellular modem connection.
- Integration – Supports interoperability with site systems such as payment kiosks, energy meter, load management controllers, distributed energy resources (DER), building energy management systems, and utility demand response platforms. Protocols here may include MQTT, Modbus, IEEE 2030.5, or OPC UA.
- Payment – Enables financial transactions through payment terminals, with connections to payment gateways and financial networks.

These communications, while distinct in purpose, may coexist within the same hardware interface. They require careful design and segregation—both logically and physically—to maintain operational security, ensure protocol compliance, and protect the confidentiality and integrity of data flowing between systems.

2.2 Structure

The following content is grouped into the major communication interfaces or domains. Specific protocols, functional components, and interaction mechanisms in these domains were then identified and defined as *Units*. From these units, *Cases* were constructed and include the step-by-step procedure, purpose, expected outcome(s), and scoring criteria.

When applicable, assumptions and pre-conditions have been defined at the unit- and procedure-level. While many assumptions and pre-conditions are applicable to units as a whole and are defined at the unit-level, some are specific to a particular procedure and are defined at the procedure-level.

Pre-conditions: conditions or states that must exist before a procedure can be executed and yield accurate results. The tester is expected to have the capabilities to complete the steps necessary to perform and meet the conditions.

Assumptions: conditions or beliefs that must be true for a procedure to be valid. These conditions are thought to be beyond the tester's control and are generally related to system/device configuration and structure.

If one or more of the pre-conditions have not been completed, perform the necessary steps to meet the condition(s). A unit or procedure should be recorded as *NULL* and testers should proceed to the next unit or procedure if:

- Assumption(s) do not hold true; or
- Pre-conditions have not and cannot be completed.

2.3 Metrics and Measurements

The procedures in this section are intended to be recorded as pass or fail, measuring 0 or 2 points. Table 4 illustrates the metrics evaluated along with the related functionality and configurations measured.

Table 2 Communications and Networking Metrics and Measurements

Metric	Measurements
1. Infrastructure Interface	1. Network Surface protected from exposure 2. Offline Communication
2. OCPP	1. Session Establishment 2. Secure File Transport 3. Firmware Update
3. Operator Management	1. Operator Access and Identity Management 2. Secure SSH configuration
4. EV Interface	1. Network Surface protected from exposure 2. Multihoming 3. ISO 15118-2 4. PnC Authorization
5. Integration	1. MQTT

	2. Modbus
6. TLS	1. TLS Client 2. TLS Key Management and Certificate Provisioning

2.4 Assessment Procedures

Each case includes procedures to verify conformance with communication standards, assess behavior under normal and degraded conditions, and evaluate the charger's response to events such as loss of connectivity, protocol mismatches, and network reconfiguration. Validation of secure communication channels, timing requirements, and fault-handling mechanisms is also included.

The provided scoring system is designed to evaluate the implementation of critical hardware security measures through a structured and quantifiable approach. Each security metric is assessed using specific measurements, which are rated on a scale from 0 to 2 points: Fully Implemented (2 points), Partially Implemented (1 point), and Not Implemented (0 points). By summing the points for each measurement, a total score is calculated. This total score reflects the overall security posture of the communications and network systems. The scoring system provides a clear interpretation of the results, enabling organizations to identify strengths and weaknesses in their communications and networks security implementations and prioritize areas for improvement.

Scoring Criteria for each assessment criteria:

- **Fully Implemented:** 2 points
- **Partially Implemented:** 1 point
- **Not Implemented:** 0 points

These procedures ensure the charger operates predictably within a connected ecosystem, supports interoperability, and maintains cybersecurity and service availability across both typical and adverse network conditions.

2.4.1 Infrastructure Interface (INF)

A charger exposes a set of interfaces and protocols that support management, provisioning, and diagnostic functions. These interfaces enable operators to configure the charger, provision it for networked services, and troubleshoot faults or performance issues. Because they often provide deep access to the charger's control logic, software, and operational data, they are critical not only for maintaining reliable service but also

for ensuring safe operation within the broader charging network. At the same time, these access points represent potential attack surfaces. If left untested, weaknesses in management interfaces could allow unauthorized control, data manipulation, or service disruption. Systematic security measurements and testing are therefore essential to validate that these interfaces enforce proper protections, resist exploitation, and maintain their intended functionality under both normal and adversarial conditions. The following is a set of procedures to measure these properties and establish a baseline for assessing the security of charger management interfaces.

2.4.1.1 Network Surface

Background: EV chargers may expose network services that are not essential to their functionality. This unit evaluates open ports and assesses whether the exposed services are operationally necessary or intended for authorized operator access.

Assumption(s): Perform the procedures in this unit, *02-INF-01*, only if the following hold true:

- The EVCS has an Ethernet or Wi-Fi interface.

02-INF-01.01: Survey Network Surface

Purpose: Ensure only necessary and documented EVCS management and provisioning service ports are exposed.

Assumption(s): Perform this procedure only if the following hold true:

- A scanning host with `nmap` (or a similar tool) is available.

Pre-Condition(s): The following must be completed prior to performing this procedure:

- Configure the EVCS network addresses and record the assigned <EVCS_IPv4> and <EVCS_IPv6> values.
Note: Both need to be present; if either is unavailable, bypass the scan.
- Configure the scanning host with `nmap` (or a similar tool) to communicate with the EVCS.

Note: The commands used in the following procedure assume the use of `nmap`.

Procedure:

1. Enumerate common TCP/UDP services:

```
nmap -sS -p- -Pn -oA results_tcp_full <EVCS_IPv4>
nmap -sS -p- -Pn -oA results_tcp6_full -6 <EVCS_IPv6>
nmap -sU --top-ports 1000 --defeat-rst-ratelimit -Pn -oA
results_udp_fastscan <EVCS_IPv4>
```

```
nmap -sU --top-ports 1000 --defeat-rst-ratelimit -Pn -oA
results_udp6_fastscan -6 <EVCS_IPv6>

nmap -sU -p15118 -Pn -oA results_udp_15118 <EVCS_IPv4>

nmap -sU -p15118 -Pn -oA results_udp6_15118 -6 <EVCS_IPv6>
```

Expected Outcome(s):

- Only ports required for management and provisioning should be exposed. Review device documentation to determine which services the operator is authorized to access.
- In many cases, this is limited to TCP 443 (HTTPS) for secure web-based management.
- In some cases, TCP 22 (SSH) may be permitted, but only when a secure remote command-line interface is intentionally provided for operator use.
- No other ports should be observed.

Scoring Criteria:

2 points: Only the required management and provisioning ports are exposed.

0 points: Any other port is exposed.

- **Score:** _____

2.4.1.2 Offline Communication

Background: A charger can be selectively enabled to continue operating even when it cannot communicate with the CSMS. This capability is often referred to as "local fallback" or "standalone mode" and is critical to maintaining service availability during network outages or CSMS failures. However, enabling this mode introduces important operational and security considerations. It is essential to clearly define and qualify the charger's behavior under these conditions, such as permitted charging sessions, authorization methods, transaction logging, and any limitations on duration or functionality. Doing so ensures that fallback operation supports resilience while minimizing risks to security, privacy, and grid coordination.

Assumption(s): Perform the procedures defined within this unit, *02-INF-02*, only if the following hold true:

- The CSMS is deployed in a manner that allows the EVSE-to-CSMS connection to be selectively disabled. This can most easily be achieved by stopping the CSMS server.

Pre-Condition(s): The following must be completed prior to performing the procedures defined within this unit, *02-INF-02*:

- Configure two RFID tokens (ISO 14443 / ISO 15693) whose UUIDs share a common prefix:
 - Properly enroll one token to be recognized by the CSMS (the *enrolled* token).
 - Do not enroll the second token in the CSMS (the *unenrolled* token).

Note: This will be used to test unauthorized access scenarios.

- Configure the charger with a Security Profile 2 network profile that points to the CSMS.

02-INF-02.01: Deferred Authorization is Disabled By Default

Purpose: Deferred authorization, sometimes referred to as *Authorization During Offline*, should be disabled by default. With deferred authorization disabled, charging must not proceed if communication between the EVCS and the management system is lost. The following procedure verifies that deferred authorization is disabled by default and that the charging station properly denies authorization when communication to the management system is lost, preventing charging from starting.

Procedure:

1. Clear the authorization cache.
2. Disable EVSE-to-CSMS connectivity as configured in the test environment.
3. Wait two minutes and record the information shown on the charger's display, lights, or indicators.
4. Present the *enrolled* token and record the outcome.

Expected Outcome(s):

- Deferred authorization is disabled by default.
- The charger indicates that it is unavailable without disclosing the specific reason to the customer.
- Charging does not commence.

Scoring Criteria:

2 points: Charging is prevented while EVSE-to-CSMS communication is unavailable.

0 points: Charging is allowed while EVSE-to-CSMS communication is unavailable.

- **Score:** _____

02-INF-02.02: Offline Authorization—Deferred Authorization of RFID

Purpose: Deferred authorization is a process that occurs when communication between an EVCS and a CSMS is restored. This procedure verifies that when EVCS-to-CSMS communication is restored and the asynchronization authorization result for charging is denied, power flow is interrupted.

Pre-Condition(s): The following must be completed prior to performing this procedure:

- *Deferred Authorization* is enabled on the charger.

Procedure:

1. Clear the authorization cache.
2. Disable EVSE-to-CSMS connectivity as configured in the test environment.
3. Connect the EV–EVSE testing equipment and initiate a charging session.
4. Present the *un-enrolled* token. Charging should begin; if it does not, terminate the procedure.
5. Restore CSMS connectivity and observe the charging session state.

Expected Outcome(s): Power transfer ceases shortly after CSMS connectivity is restored.

Scoring Criteria:

2 points: Charging power flow stops when EVSE-to-CSMS communication is restored.

0 points: Charging power flow continues when EVSE-to-CSMS communication is restored.

- **Score:** _____

02-INF-02.03: Deferred Authorization – CSMS Priority over cached tokens

Purpose: When authorization is deferred due to loss of connectivity to the CSMS, the token may be cached. When communication with the CSMS is available, the charger should prefer the CSMS over tokens cached in the LAL.

Assumption(s): Perform this procedure only if the following hold true:

- The charger is configured with deferred authorization enabled.

Procedure:

1. Clear the authorization cache by deleting all tokens from the LAL.
2. Disable EVSE-to-CSMS connectivity as configured in the test environment.
3. Connect the EV–EVSE testing equipment and initiate a charging session.
4. Present an un-enrolled token/RFID to start the charging session.
5. Complete the charging session and disconnect the EV-EVSE testing equipment.
6. Restore CSMS connectivity.
7. Reconnect the EV-EVSE testing equipment and start a charging session.
8. Present the same *un-enrolled* token again to start a new charging session and observe the outcome.

Expected Outcome(s): The session is not authorized when CSMS connectivity is available, since the token is not enrolled.

Scoring Criteria:

2 points: Charging is not authorized when EVSE-to-CSMS communication is restored.

0 points: Charging is authorized when EVSE-to-CSMS communication is restored.

- **Score:** _____

2.4.2 OCPP

The Open Charge Point Protocol (OCPP) is the primary standard used for communication between chargers and central management systems, governing functions such as session control, transaction handling, metering, and firmware updates. Because it provides the foundation for remote management and integration of chargers into larger networks, OCPP is both operationally critical and highly exposed to cyber threats. Weaknesses in OCPP implementations could allow attackers to manipulate billing data, disrupt charging sessions, or gain unauthorized control of charging infrastructure. Testing and measuring OCPP implementations are therefore essential to ensure protocol compliance, validate secure handling of authentication and authorization, and assess resilience against malformed or malicious inputs. Establishing a repeatable set of measurements not only helps identify vulnerabilities but also provides operators and vendors with assurance that their OCPP deployments are robust, interoperable, and secure.

2.4.2.1 Session Establishment

Background: Three versions of the OCPP are widely deployed: OCPP 1.6j (likely the most widely deployed version at the time of writing), OCPP 2.0.1, and OCPP 2.1. Additionally, security extensions can be applied to OCPP 1.6j. This unit evaluates the charger's capability to support these protocol versions and security extensions, and to securely establish a connection between the charger and the CSMS.

Assumption(s): Perform the procedures defined within this unit, *02-OCPP-01*, only if the following hold true:

- A certificate authority is shared between the EVCS and CSMS.

Pre-Condition(s): The following must be completed prior to performing the procedures defined within this unit, *02-OCPP-01*:

- Ensure the EVCS and CSMS are online and reachable over OCPP.
- Configure the CSMS to support OCPP 2.0.1 over a secure TLS connection.
- Configure the CSMS to support OCPP 1.6J with security extensions over a secure TLS connection using a separate port from OCPP 2.0.1.

02-OCPP-01.01: Secure TLS

Purpose: This case confirms that the EVCS can successfully establish a secure TLS connection with the CSMS, demonstrating support for the protocol and its associated security parameters (detailed in Appendix F.1).

Procedure:

Perform procedures in Unit *TLS Client* (01-TLS-01).

Record total score.

Scoring Criteria:

2 points: Total raw score from Unit *TLS Client* (01-TLS-01) is 10 points.

0 points: Total raw score from Unit *TLS Client* (01-TLS-01) is less than 10 points.

Score: _____

02-OCPP-01.02: OCPP 2.0.1 Support

Purpose: This case confirms that the EVCS can successfully establish a secure connection with the CSMS using OCPP 2.0.1, demonstrating support for the protocol and its associated security profile.

Procedure:

1. Configure a OCPP 2.0.1 Security Profile 2 network profile, specifying the CSMS connection URL with the “wss://” prefix.
2. Wait for the charger to initiate and establish a secure connection with the CSMS.
3. Confirm successful connection establishment by examining the CSMS connection status and associated log entries.

Expected Outcome(s): The EVCS successfully establishes a secure OCPP 2.0.1 connection with the CSMS.

Scoring Criteria:

2 points: A secure OCPP 2.0.1 connection is established.

0 points: A secure OCPP 2.0.1 connection is not established.

• **Score:** _____

Note: If OCPP 2.0.1 does not establish a connection, proceed to the following unit, 02-OCPP-02. Omit and bypass the remaining cases within this unit.

02-OCPP-01.03: OCPP 1.6j and Security Extension Support

Purpose: Although chargers should support OCPP 2.0.1 and higher, interoperability with existing infrastructure often requires support for OCPP 1.6j. This procedure

verifies that the charger can be configured to operate using OCPP 1.6j/Security Extension, which uses a secure TLS connection with WebSockets.

Procedure:

1. Configure the network profile to use OCPP 1.6j over TLS. The CSMS connection URL must be specified with the “wss://” prefix. If OCPP 1.6j is unavailable stop the test.
2. Initiate a connection from the EVCS to the CSMS.

Expected Outcome(s): The EVCS successfully establishes a secure OCPP 1.6j connection with the CSMS.

Scoring Criteria:

2 points: OCPP 1.6j is unavailable or a secure OCPP 1.6j connection is established.

0 points: An insecure OCPP 1.6j connection is established.

- **Score:** _____

02-OCPP-01.04: Disallow OCPP Version Degradation

Purpose: EVCS enforces the use of OCPP 2.0.1 and does not permit version degradation by offering or accepting OCPP 1.6. This ensures that the EVCS maintains the intended security level and does not fall back to weaker protocol versions.

Assumption(s): Perform this procedure only if the following hold true:

- The CSMS certificate and keypair can be accessed by `test_csms_websockets.py` (Appendix F).

Procedure:

1. Run `test_csms_websockets`, specifying the CSMS certificate and key pair, and note the port used.
2. Configure the EVCS to connect using a standard OCPP 2.0.1 Security Profile 2 network profile to `test_csms_websockets`.
3. Observe the `test_csms_websockets` output.

Expected Outcome(s): The EVCS does not offer OCPP 1.6 and does not fall back and accept it.

EVCS offered Ocpp1.6: No

EVCS accepted Opp1.6: No

Scoring Criteria:

2 points: OCPP 1.6 is not offered, nor accepted.

0 points: OCPP 1.6 is offered or accepted.

- **Score:** _____

02-OCPP-01.05: OCPP 2.0.1 Security Profile Enforcement

Purpose: OCPP 2.0.1 supports multiple security profiles, ranging from Profile 0 (no security) to Profile 3 (mutual TLS with strong certificate handling). This test verifies that when configured for an insecure security profile (0 or 1), the charger provides clear warnings or lockouts to inform the operator that the selected configuration does not meet secure deployment requirements.

Procedure:

1. Establish an OCPP 2.0.1 network profile on the EVCS. Set the security profile to 0 (no security) or 1 (password-based authentication without TLS) and specify the CSMS URL with the ws:// prefix.
2. Observe the interface behavior, including lockouts, warnings, and confirmation prompts.

Expected Outcome(s):

- The EVCS informs the operator that the selected profile is insecure.
- The EVCS should generate a warning or confirmation message requiring explicit acknowledgement before continuing with the insecure configuration.

Scoring Criteria:

2 points: The operator is notified of an insecure configuration.

0 points: The operator is not notified of an insecure configuration.

- **Score:** _____

02-OCPP-01.06: CSMS Root Certificate Authority Is Prioritized

Purpose: The procedure confirms the EVSE prioritizes the configured CSMS certificate authority (CSMSRootCertificate) over the ISO 15118 V2G root certificate (V2GRootCertificate) and over general WebPKI certificate authorities.

Pre-Condition(s): The following must be completed prior to performing this procedure:

- Configure a CSMS certificate issued by a Certificate Authority (CA) recognized by the WebPKI.
- Configure the CSMS to listen on two distinct ports, both using Security Profile 2:
 - a. Port 1 uses a certificate issued by the WebPKI.
 - b. Port 2 uses the certificate and key pair generated at the unit level.

- Ensure the EVSE's `CSMSRootCertificate` is configured to include only the local CA certificate.

Procedure:

1. Configure the EVSE's network profile to connect to *CSMS Port 1* with Security Profile 2.
2. Wait for the EVSE to attempt an OCPP connection.
3. Configure the EVSE's network profile to connect to *CSMS Port 2* with Security Profile 2.
4. Wait for the EVSE to attempt an OCPP connection.

Expected Outcome(s):

After Step 2 of the procedure:

- The EVSE must fail to connect.
- The EVSE logs must show that the connection failure is caused by an untrusted or unknown certificate authority.

After Step 4 of the procedure:

- The EVSE must successfully establish a secure OCPP connection.

Scoring Criteria:

2 points: The EVSE appropriately logs a failed connection (after Step 2), and a secure OCPP connection is established (after Step 4).

0 points: Any one of the expected outcomes fail.

- **Score:** _____

02-OCPP-01.07: (OCPP) Charging Station Certificate Usage

Purpose: This procedure confirms the use of `ChargingStationCertificate`.

Pre-Condition(s): The following must be completed prior to performing this procedure:

- Configure the CSMS to support OCPP 2.0.1 Security Profile 3.

Note: This will likely need a second CA.

Procedure:

1. From the charger, trigger communication requiring Security Profile 3 authentication.
2. Observe connection state and logs

Expected Outcome(s): Both the EVSE and CSMS should report a successful connection.

Scoring Criteria:

2 points: Both the EVSE and CSMS report a successful connection.

0 points: Either the EVSE or the CSMS do not report a successful connection.

- **Score:** _____

2.4.2.2 Secure File Transport

Background: The following cases verify that the charger enforces the use of secure transport protocols when transmitting diagnostic and security logs to a CSMS. The objective is to ensure that sensitive operational data is protected in transit and that insecure options cannot be used by default.

Secure transport protocols ensure confidentiality and integrity for diagnostic data transfers. Insecure protocols, when permitted, must require explicit opt-in so that weaker options are only used under controlled circumstances, such as legacy system support.

Pre-Condition(s): The following must be completed prior to performing the procedures defined within this unit, *02-OCPP-02*:

- Ensure EVCS and CSMS are online and reachable over OCPP.
- Ensure there exists a shared CA, trusted by the EVSE and CSMS.
 - a. On the EVSE: `CSMSRootCertificate` contains only this CA certificate.
 - b. On the CSMS: presents a server certificate issued by this CA.
- Configure EVSE to connect to the CSMS using Security Profile 2.
 - a. The EVSE is configured with connectivity supporting HTTPS to a file server configured with a certificate anchored at `CSMSRootCertificate (server1)`

02-OCPP-02.01: Secure Transfer Protocols Are Enabled By Default

Purpose: This case confirms that the charger supports only secure file transfer protocols by default, ensuring that diagnostic and security data is transmitted with confidentiality and integrity, and that any insecure protocols require explicit opt-in for legacy or controlled use.

Procedure:

1. From the CSMS, issue a `GetVariable` request for the `FileTransferProtocols` variable.
2. Record the set of transport protocols reported by the charger.
3. Attempt to initiate a diagnostic log transfer over each reported protocol.
4. Review the CSMS to determine success

Expected Outcome(s):

- The charger reports only secure transport protocols such as HTTPS, FTPS, or SFTP.
- The charger successfully uploads the diagnostic logs over HTTPS. The CSMS records a `GetLogResult` with status set to `Accepted`.

Scoring Criteria:

2 points: A secure file transfer protocol is supported and used.

0 points: No secure file transfer protocols are supported or used.

- **Score:** _____

02-OCPP-02.02: Unadvertised File Transfer Protocols

Purpose: This case confirms that the charger enforces protocol restrictions by rejecting file transfer requests over unadvertised or insecure protocols, thereby preventing the use of unauthorized channels and ensuring only approved secure transport methods are accepted.

Procedure:

1. From the CSMS, invoke a diagnostic log download request using URLs with prefixes <http://> and <ftp://>.
2. Review CSMS logs for the charger's response.

Expected Outcome(s): The charger rejects the request by returning a `GetLogResult` with status set to `Rejected` and may optionally indicate that the requested protocol is unsupported.

Scoring Criteria:

2 points: The log download request is rejected.

0 points: The log download request is accepted.

- **Score:** _____

02-OCPP-02.03: Trusted CA – Successful File Transfer

Purpose: This procedure confirms that file transfer succeeds when the file server certificate is anchored to the `CSMSRootCertificate`.

Procedure:

1. Trigger a `GetLogRequest` from the CSMS, specifying the `remoteLocation` as the file server that is signed by the shared CA.

Expected Outcome(s):

- The log file should be successfully uploaded to the file server.
- The CSMS should confirm completion without errors.

Scoring Criteria:

2 points: The log file is successfully uploaded and the CSMS confirms completion without errors.

0 points: The log file is not uploaded or the CSMS confirms completion with errors.

- **Score:** _____

02-OCPP-02.04: Untrusted CA – Rejection

Purpose: This case confirms that the charger verifies the certificate it receives from the CSMS during a log request and properly handles a log request when the CA is not preconfigured or trusted.

Pre-Condition(s): The following must be completed prior to performing this procedure:

- Configure the EVSE to have connectivity to a second file server configured with the WebPKI certificate (*server2*) and supporting HTTPS:
- Configure a certificate issued by the WebPKI CA or a second CA that is not trusted by the EVSE (not included in *CSMSRootCertificate*).

Procedure:

1. Trigger a *GetLogRequest* from the CSMS to the file server *server2*.

Expected Outcome(s):

- The log file transfer fails.
 - The charger should reject the connection attempt due to certificate validation failure.
- The CSMS records a *logStatusNotification* with status *UploadFailure*.
 - Review of the EVSE security logs should record the reason for failure, unable to verify the certificate

Scoring Criteria:

2 points: The log file transfer fails, and the event is recorded to the EVSE security log.

0 points: The log file transfer succeeds, or an event is not recorded to the EVSE security log.

- **Score:** _____

2.4.2.3 Firmware Update

Background: OCPP can be used to trigger a firmware update on the charger. In this process, the CSMS issues an *UpdateFirmware* request that provides the charger with the location of the firmware package, a start time for the update, and optional parameters such as retry intervals. The charger then downloads the firmware image directly from the specified server using a supported file transfer protocol, validates it, and applies the update.

Note: Some EVSE manufacturers have disabled firmware updates via OCPP, opting instead to perform remote updates through their proprietary management interfaces.

Assumption(s): Perform the procedures defined within this unit, *02-OCPP-03*, only if the following hold true:

- EVCS and CSMS are online and reachable over OCPP.
- The charger is capable of performing firmware updates using the OCPP 2.0.1 update mechanism.

02-OCPP-03.01: Firmware Updates Disabled by Default

Purpose: By default, firmware updates via OCPP are disabled and can only be enabled through explicit operator action.

Pre-Condition(s): The following must be completed prior to performing this procedure:

- The *UpdatesDisabled* setting is configured in its default state.

Procedure:

1. Review the EVCS management interface to confirm that firmware updates are disabled by default and can only be enabled when explicitly selected

Expected Outcome(s): The EVCS management interface indicates that firmware updates are disabled by default, and explicit operator action is required to enable them.

Scoring Criteria:

2 points: Firmware updates are disabled and therefor explicit operator action is required to enable them.

0 points: Firmware updates are not disabled.

- **Score:** _____

02-OCPP-03.02: Enforcement of Signature Presence in FirmwareUpdate Requests

Purpose: The signature field in the `FirmwareUpdate` request is optional. Verify that the charger supports enforcing the presence and validation of a signature in `FirmwareUpdate` requests.

Procedure:

1. Review the EVCS management interface to confirm that firmware updates require a valid digital signature provided in the `UpdateFirmwareRequest`.

Expected Outcome(s): The EVCS enforces the presence and validation of a digital signature included in the `UpdateFirmware Request` before applying any firmware update.

Scoring Criteria:

2 points: Digital signing and signature validation are set as required.

0 points: Otherwise

- **Score:** _____

2.4.3 Operator Management (OM)

A charger typically includes an additional interface for operator use in provisioning, management, and diagnostics. This interface is usually provided through SSH or a web application.

2.4.3.1 Operator Access and Identity Management

Background: In many deployments, chargers are not operated as isolated devices but as part of larger fleets spanning multiple sites, facilities, or regions. In these environments, a range of personnel, including field technicians, operators, service contractors, and auditors, may require access to charger interfaces for diagnostics, configuration, and maintenance. Relying on a single shared username and password is neither scalable nor secure because it obscures accountability, complicates credential rotation, and increases the risk of unauthorized access. To support fleet-wide operations, chargers must be capable of integrating with a central identity and policy management framework that eliminates shared passwords and enables consistent, auditable operator access.

02-OM-01.01: Centralized Identity and Sign-On Integration

Purpose: Determine whether the operator management interfaces support integration with a centralized sign-on framework (for example, OIDC) rather than using locally configured accounts and shared credentials.

Assumption(s): Perform this procedure only if the following hold true:

- The charger documentation (specifically related to operator authentication) and operator management interfaces are available.

Procedure:

1. Review the charger documentation and operator management interfaces to determine how operator authentication is implemented.
2. Identify whether the charger supports integration with a centralized identity service that provides OpenID Connect (OIDC), certificate-based authentication, or an equivalent open standard for operator authentication.
3. Confirm that the charger can enforce role-based access controls (RBAC) derived from the user identity or token claims.
4. Verify that shared credentials can be disabled or replaced.
5. Record the configuration steps required to enable centralized identity integration.

Expected Outcome(s): The charger can integrate with a logically centralized identity framework.

Scoring Criteria:

2 points: The charger can integrate with a centralized identity service.

0 points: Otherwise

- **Score:** _____

2.4.3.2 SSH

Background: SSH is a secure network protocol that allows users to remotely create encrypted connections to a host. Once a successful connection is established, SSH can then be used to send commands or transfer data between the client and the host. SSH can be used within EVCI in two primary contexts:

- **OEM Charger Access:** SSH may be used by the charger manufacturer to remotely access the charger or purposes such as diagnostics, firmware updates, maintenance, or troubleshooting.
- **Charging Station Operator Access:** The CSO may use SSH to securely manage charger configurations, perform system maintenance, monitor operational status, or respond to incidents at the site or device level.

These procedures ensure that SSH connections to the charging station are secure and enable their intended operational function for the CSO. These procedures also verify that systems utilizing SSH are hardened against threats in an effort to prevent unauthorized access and other forms of system compromise.

Assumption(s): Perform the procedures defined within this unit, 02-OM-02, only if the following hold true:

- Case 02-INF-01.01 has been completed, and the results are available for review.
- The charger's IPv4 and IPv6 addresses are known.

Note: These should have been configured and recorded in Case 02-INF-01.01.

Network Survey

Background: The context of which SSH is used for making remote connections to a charging station should be understood by a manufacturer. Having a readily available network survey can assist in understanding security posture by ensuring that all used services and ports are known to a vendor or test operator and that SSH is not being used by default.

02-OM-02.01: Network Survey Review

Purpose: This procedure ensures that the default port (TCP/22; alternatively, TCP/2222) is closed if SSH is not intended for use by an operator or the end customer. Additionally, this case verifies that SSH is not in use on unintended ports.

Pre-Condition(s): The following must be completed prior to performing this procedure:

- Review the charger's documentation to determine whether SSH is intended for use by the Operator.

Procedure:

1. Review the results of procedure 02-INF-01.01 to verify whether the SSH service is active.
2. Record the port(s) running SSH.

Expected Outcome(s):

If SSH is not intended for use by the operator or end customer:

- SSH service should be unavailable.

If SSH is intended for use by the operator or end customer:

- SSH service is available only on intended ports.

Note: SSH typically operates on the default port 22, but it may be configured to use a different port.

Scoring Criteria:

If SSH is not intended for use by the operator or end customer:

2 points: SSH is not available on any ports.

0 points: SSH is available

If SSH is intended for use by the operator or end customer:

2 points: SSH is available and intended for the CSO

0 points. SSH is not available.

- **Score:** _____

Decision Point: If SSH is not available, skip the remaining cases in this unit and proceed directly to *Section 2.4.4, EV Interface*.

Establishment and Authentication

Background: SSH session establishment and authentication are essential in maintaining confidentiality and integrity in an SSH session. It is important to note that OpenSSH disables protocol 1 (SSH version 1) by default and testing that SSH1 connections are rejected is a formality as version 2 should be forced by OpenSSH.

Assumption(s): Perform the procedures in this subsection only if the following assumptions hold true:

- The test host functions as the SSH client.
- The charging station provides an SSH server and is available to the operator for maintenance or administrative access.
- A valid user account with SSH privileges has been configured on the EVCS
- The valid user account credentials (username, password and/or SSH private key) are known
- Only public-key and password-based authentication methods are enabled on the charger's SSH server—all other SSH authentication methods are disabled.
- The tester has access to common networking tools such as NMAP, Wireshark, or Netcat.

02-OM-02.02: Establish SSH Connection

Purpose: This procedure ensures that a successful SSH2 connection can be established between the Tester and EVSE.

Procedure:

1. Attempt SSH2 connection to charger from test host machine with verbose output
Note: Refer to subsection SSH under Tools for command guidance.
2. Review the output to verify SSH2 is being used between the charger and host.

Expected Outcome(s): SSH connection accepted and established using SSH2.

Scoring Criteria:

2 points: SSH connection is established using SSH2.

0 points: SSH connection fails or is established using a version less than SSH2.

- **Score:** _____

02-OM-02.03: Legacy SSH Versions Not Supported

Purpose: This case verifies that the SSH service on the charger only supports SSH2 and that legacy versions, such as SSH1, are not advertised.

Assumption(s): Perform this procedure only if the following hold true:

- SSH port(s) were identified in Case 02-OM-02.01.

Procedure:

1. Execute the following `nmap` commands to identify the SSH service version in use.
Replace the port values with those identified during procedure 02-OM-02.01.

For IPv4:

```
# nmap -sV -p<ports> <EVCS_IPv4>
```

For IPv6:

```
# nmap -6 -sV -p<ports> <EVCS_IPv6>
```

Example:

```
# nmap -sV -p22,80,443 <EVCS_IPv4>
# nmap -6 -sV -p22,80,443 <EVCS_IPv6>
```

Example Output:

```
# nmap -sV -p22,80,443 <EVCS_IPv4>
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-11-07 14:18 PST
Nmap scan report for <EVCS_IPv4>
Host is up (0.00028s latency).

PORT      STATE SERVICE  VERSION
22/tcp    open  ssh      OpenSSH 10.0p2 Debian 7 (protocol 2.0)
80/tcp    open  http     nginx
443/tcp   open  ssl/http nginx
MAC Address: 52:54:00:B2:C3:65 (QEMU virtual NIC)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect
results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 25.41 seconds
```

2. Review the `nmap` output to identify the SSH service and its protocol version.

Expected Outcome(s): SSH protocol version is 2.0 or higher.

Scoring Criteria:

2 points: The SSH protocol version is 2.0 or higher

0 points: The SSH protocol version is below 2.0.

- **Score:** _____

02-OM-02.04: Secure Cryptographic Methods

Purpose: This case ensures that the key exchange between charger and test host is done using secure cryptographic methods. Additionally, this test ensures that only secure ciphers are used during the process of data transmission between charger and host. The purpose of these tests is to ensure confidentiality exists in communications between the client and the host when connecting to the Infrastructure Interface via SSH.

Assumption(s): Perform this procedure only if the following hold true:

- SSH port(s) were identified in Case 02-OM-02.01.

Procedure:

1. Run `nmap` cipher enumeration scan. Replace `<ssh_port>` with the SSH port(s) identified in procedure 02-OM-02.01.

For IPv4:

```
# nmap -p <ssh port> --script ssh2-enum-algos <EVCS_IPv4>
```

And for IPv6:

```
# nmap -6 -p <ssh port> --script ssh2-enum-algos <EVCS_IPv4>
```

2. Review the `nmap` output to identify the available `KexAlgorithms`, `HostKeyAlgorithms`, `Ciphers`, and `MAC` algorithms.

Expected Outcome(s): Only secure algorithms are offered and negotiated.

Note: Refer to subsection SSH under Standard Configuration Settings for recommended secure cryptographic methods.

Scoring Criteria:

2 points: Only secure algorithms defined in subsection *SSH* under *Standard Configuration Settings* are offered and negotiated.

0 points: Any algorithm not defined in subsection *SSH* under *Standard Configuration Settings* is offered or negotiated.

- **Score:** _____

02-OM-02.05: Public-Key Authentication Functionality

Purpose: This case ensures that the mechanisms used to authenticate to a charger when connecting via SSH operate as intended.

Procedure:

1. Attempt to connect via SSH to charger with verbose output
Note: Refer to subsection SSH in the Tools section for command guidance.
2. Observe `KexAlgorithm` behavior in verbose output, ensuring authentication is working properly

Expected Outcome(s): Public-key authentication is successful.

Scoring Criteria:

2 points: SSH authentication is successful using the public/private key-pair.

0 points: SSH authentication is unsuccessful or prompts the user for alternate form of credentials.

- **Score:** _____

02-OM-02.06: Invalid Public-Key Authentication Behavior

Purpose: This case ensures that the authentication mechanisms used by the charging stations SSH server operate as intended.

Pre-Condition(s): The following must be completed prior to performing this procedure:

- An invalid private key has been configured on the host.

Procedure:

1. Using invalid private key, attempt connection to charger via SSH
2. Observe `KexAlgorithm` behavior in verbose output

Expected Outcome(s): Public-key authentication is not successful.

Note: Agent should refuse operation

Scoring Criteria:

2 points: SSH authentication is not successful.

0 points: SSH authentication is successful or prompts the user for alternate form of credentials.

- **Score:** _____

02-OM-02.07: Login Grace Time Functionality

Purpose: This procedure ensures that Login grace time functions properly when a ssh connection is made between a user and a charger. Login grace time allows users a small window of time to authenticate when the initial connection is made to a charger. While it is assumed that public key authentication and access controls will be in place, Login grace time should be configured to minimize the time an attacker can spend connected to the charger via SSH, reducing the window for which brute-force attempts can be made.

Procedure:

1. Connect to the charger via SSH

Note: Refer to subsection SSH under the Tools section for command guidance.

2. When prompted for password, do not type or send input for 30s.

3. Observe if connection is closed

Note: Input may be required to observe results

Expected Outcome(s): After 30s, the connection is closed.



```
[x]~[user@parrot]~[~]
$ssh EVSE-Serv@192.168.56.101
EVSE-Serv@192.168.56.101's password:
Connection closed by 192.168.56.101 port 22
[x]~[user@parrot]~[~]
$
```

Figure 2 Connection closed message when *LoginGraceTime* limit reached

Scoring Criteria:

2 points: SSH connection is closed within 15-30 seconds.

0 points: SSH connection remains open for more than 30 seconds.

- **Score:** _____

02-OM-02.08: Reject Empty Passwords

Purpose: Empty passwords should be disabled in SSH by default, preventing clients from logging in without authentication. This case verifies that empty passwords are indeed disallowed. Although this is the default SSH configuration, it is strongly recommended to confirm this setting.

Pre-Condition(s): The following must be completed prior to performing this procedure:

- A list of typical usernames¹⁹ and valid usernames for the charger are known.

Procedure:

1. For each typical and valid username, attempt SSH using password authentication only.
Note: Refer to subsection SSH under the Tools section for command guidance
2. When prompted for credentials, leave password space empty and hit enter
3. Note the server response (for example, "Permission denied") and the number of prompts before disconnecting.

¹⁹ Lists of common usernames available at <https://github.com/danielmiessler/SecLists/blob/master/Usernames/top-usernames-shortlist.txt> (accessed October 29, 2025) and <https://github.com/danielmiessler/SecLists/blob/master/Usernames/xato-net-10-million-usernames-dup.txt> (accessed October 29, 2025).

Expected Outcome(s): Empty-password attempts are rejected for all tested usernames.

Scoring Criteria:

2 points: All SSH connection attempts are rejected.

0 points: Any of the SSH connection attempts are successful.

- **Score:** _____

02-OM-02.09: Reject Root Login

Purpose: This case ensures users cannot log in directly as root. Disabling direct root access deters automated attacks, enforces least privilege, and limits impact if an account is compromised.

Procedure:

1. Review charger documentation for root password or key.
2. Connect to the charger via SSH as user `root`. Supply the key if found in documentation. If the credential is unknown, press Enter to continue.

Note: Refer to subsection SSH in the Tools section for command guidance.

Expected Outcome(s): Direct root login is denied.



```
[user@parrot]~  
$ssh root@192.168.56.101  
root@192.168.56.101's password:  
Permission denied, please try again.  
root@192.168.56.101's password: █
```

Figure 3 SSH Root Login Output

Scoring Criteria:

2 points: SSH connection is denied.

0 points: SSH connection is successful.

- **Score:** _____

02-OM-02.10: Limit Failed Logins

Purpose: This case verifies that the charger limits authentication attempts over SSH. Limiting attempts reduces brute force risk by capping failed logins during a connection or within a time window. This control still applies when public key

authentication is enabled so that attackers have minimal opportunities to guess credentials.

Procedure:

1. Request SSH connection to charger using password-only authentication.

Note: Refer to subsection SSH in the Tools section

2. Enter invalid password.
3. Repeat Step 2, two more times (total of three attempts).

Expected Outcome(s): Received disconnect from charger due to too many authentication failures.

Scoring Criteria:

2 points: SSH session is unsuccessful and disconnected after 3 or less failed authentication attempts.

0 points: SSH session is successful and/or SSH connection remains live after 3 failed authentication attempts.

- **Score:** _____

02-OM-02.11: Port Forwarding Disabled

Purpose: This case affirms that SSH remote port forwarding is disabled. Remote port forwarding allows traffic from a port on the SSH server to be tunneled back to the client, which could expose internal services and bypass network controls.

Procedure:

1. Create connection to charger via ssh

```
ssh <user>@<IP>
```

2. Once session is established, attempt to create a port forward from charger.

```
ssh -R <remote port>:localhost:<ssh port> <user>@<EVCS_IPv4>
```

3. Observe if message appears stating port forward failed.

Expected Outcome(s): Message stating port forward failed is displayed.

Scoring Criteria:

2 points: Port forward fails for specified port and message is displayed stating port forwarding failed for specified port.

0 points: Specified port is successfully forwarded, or no message is displayed stating specified port failed to forward.

- **Score:** _____

Session

Background: During a SSH session, if an attacker is able to control of the session, they will have the same level of privilege as the compromised account. The cases below evaluate behavior and configurations after a SSH session with the charger has been fully authenticated and established. These cases ensure that hardened configurations are in place to prevent a sessions compromise.

Assumption(s): Perform the procedures in this subsection only if the following assumptions hold true:

- The charging station provides an SSH server and is available to the operator for maintenance or administrative access.
- A valid user account with SSH privileges has been configured on the EVCS and the credentials are known.

02-OM-02.12: Inactivity Timeout

Purpose: This case confirms that management SSH sessions terminate automatically after a period of inactivity.

Procedure:

1. Connect to the charger via SSH.
Note: Refer to subsection SSH under the Tools section for command guidance.
2. Confirm shell access.
3. Wait and observe the shell for 15 minutes without typing or sending input. Record the time at which the session closes.

Expected Outcome(s): The SSH session is terminated by the charger after 5 minutes or less of inactivity.²⁰

Scoring Criteria:

2 points: SSH sessions shall terminate after no more than 5 minutes of inactivity.
0 points: Otherwise.

- **Score:** _____

²⁰ The configured inactivity timeout value is derived from the Security Technical Implementation Guide (STIG) for Cisco NX-OS Switch NDM, finding https://stigviewer.com/stigs/cisco_nx_os_switch_ndm/2025-05-19/finding/V-220493 (accessed October 30, 2025).

Management Logging

Background: Under OCPP 2.0.1, security events are reported via `SecurityEventNotification`.²¹

In the case of SSH, security events should be defined as follows:

- Failed Authentication attempts
- Successful Authentication attempts

Assumption(s): Perform the procedures in this subsection only if the following hold true:

- The charging station provides an SSH server and is available to the operator for maintenance or administrative access.
- The charger is enrolled with the CSMS and operates under OCPP Security Profile 2.

02-OM-02.13: Login `SecurityEventNotification`

Purpose: This procedure confirms that an OCPP 2.0.1 `SecurityEventNotification` is generated upon an operator management login.

Procedure:

1. Attempt SSH connection to charger with invalid credentials.
2. Wait until authentication completes.
Note: If attempt is successful, end session and continue to Step 3
3. Attempt SSH connection to charger using valid credentials.

Expected Outcome(s):

Step 1:

- Access is denied.
- A `SecurityEventNotification` record with the event type `MaintenanceLoginFailed` is recorded by the CSMS.

Step 3:

- Access is granted.
- A `SecurityEventNotification` with the event type `MaintenanceLoginAccepted` is recorded by the CSMS

Scoring Criteria:

2 points: Two `SecurityEventNotification` records, of event types `MaintenanceLoginFailed` and `MaintenanceLoginAccepted`, are recorded by the CSMS.

²¹ "OCPP 2.0.1 Edition 3 Part 2 - Specification." Open Charge Alliance

0 points: Otherwise.

- **Score:** _____

2.4.4 EV interface (EV)

The EV interface encompasses the protocols that define the layered interactions between an electric vehicle and a charger, from the moment a plug is inserted until the session is terminated. These interfaces span physical safety signaling, session control, high-level communications, metering, and optional value-added services, working together to govern battery charging and ensure that the process is performed safely, reliably, and in accordance with both vehicle and grid requirements.

2.4.4.1 Network Surface

Background: By default, a charger should initialize with only a minimal and tightly controlled set of services enabled. This default configuration plays a critical role in minimizing the system's attack surface and supporting a secure deployment posture, especially in environments where chargers may be exposed to public networks or untrusted devices. Services not essential to core charging functionality—such as web interfaces, remote shells, or auxiliary protocols—should be disabled unless explicitly required and secured.

This set of cases is designed to evaluate the charger's default state immediately after boot or factory reset, with no custom configuration applied. It enumerates the active services, network listeners, and open ports, and compares them against a predefined minimal baseline—representing what should be enabled by default for safe and functional operation. Any deviation from this minimal service set may indicate unnecessary exposure, poor hardening, or insecure defaults that warrant remediation.

Pre-Condition(s): The following must be completed prior to performing the procedures defined within this unit, *02-EV-01*:

- Configure the AcCCs and note the interface name of the powerline communication (PLC) network device.

Note: For all procedures within the following section, replace "plc0" with the PLC interface name noted during this pre-condition.

02-EV-01.01: Survey Network Surface

Purpose: To identify and document all network-accessible services exposed by the charger, this case validates that only expected ISO 15118 and OCPP-related ports

are available and flag any additional open ports as potential unintended attack surface.

Procedure:

1. Determine the charger's network address along with any ports associated with ISO 15118 services. This can be accomplished using the `sdpquery` tool:

```
# ./sdpquery plc0
```

2. The output reports the charger's network addresses and ports. Be sure to record any unique IPv6 addresses:

```
fe80::9c29:f2ff:fe94:95fa : [ IP address:
fe80::9c29:f2ff:fe94:95fa, Port: 64109 , Security: TLS ,
Transport: TCP ]

fe80::9c29:f2ff:fe94:95fa : [ IP address:
fe80::9c29:f2ff:fe94:95fa, Port: 61341 , Security: NO_TLS ,
Transport: TCP ]
```

3. Conduct a `nmap` scan for each of the discovered IPv6 addresses. To reduce overall execution time, it is advisable to perform the scans in three separate rounds. Use the following three commands:

```
# nmap -6 -e plc0 -Pn -p - -sS fe80::9c29:f2ff:fe94:95fa
# nmap -6 -e plc0 -Pn -max-rtt-timeout 25ms -max-retries 3 -
sU fe80::9c29:f2ff:fe94:95fa
# nmap -6 -e plc0 -Pn -p U:15118 -sU
fe80::9c29:f2ff:fe94:95fa
```

4. Check the `nmap` results for open services. For instance, two services may appear that correspond to the ones reported by `sdpquery`.

```

Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-07-18
11:14 PDT

Nmap scan report for fe80::9c29:f2ff:fe94:95fa

Host is up (0.000010s latency).

Not shown: 65533 closed tcp ports (reset)

PORT      STATE SERVICE
61341/tcp  open  unknown
64109/tcp  open  unknown

MAC Address: 9E:29:F2:94:95:FA (Unknown)


Nmap done: 1 IP address (1 host up) scanned in 1.56 seconds

```

Expected Outcome(s):

- Up to three open ports may be observed: UDP 15118 and one or two TCP ports (including the port reported by `sdpquery`).
- No additional ports should be open. Any other discovered ports must be flagged, as the EV should not have access to them.

Scoring Criteria:

2 points: Only ISO 15118 communication ports are observed (UDP 15118 and up to two TCP ports).

0 points: Any port other than a 15118-communication port is observed.

- **Score:** _____

2.4.4.2 Multihoming

Background: A charger is a multihomed host. A multihomed host is a networked device that has multiple network interfaces and IP addresses, connected to different networks or subnets. This configuration enables the device to maintain connectivity across multiple network paths—such as to the EV and to infrastructure / cloud.

In the context of EV charging, multihoming introduces complexity in routing behavior, address selection, and interface prioritization, particularly when IPv6 is used.

The following set of procedures is designed to evaluate the charger's behavior under multihoming and IPv6 routing conditions. The basic principle guiding these procedures is the identification of unsafe handling of routing and routing information.

This includes, but is not limited to, the improper acceptance of unsolicited route advertisements.

Assumption(s): Perform the procedures defined within this unit, *02-EV-02*, only if the following hold true:

- EVCS is online and reachable via EV to EVSE interface

02-EV-02.01: Interface Isolation Enforcement

Purpose: This case verifies that the charger does not leak infrastructure interface information onto the EV interface.

Procedure:

1. Note infrastructure interface IP address(es).
2. From EV to EVSE interface, perform ARP/NDP request for IP address noted in Step 1.

Expected Outcome(s): No response

Scoring Criteria:

2 points: There is no response to the ARP/NDP request.

0 points: The ARP/NDP request returns a response.

- **Score:** _____

02-EV-02.02: IPv6 Router Advertisement Single Prefix

Purpose: This case verifies that the charger announces at most a single IPv6 prefix for SLAAC or DHCPv6, and that this prefix is limited strictly to the EV–EVSE interface. The charger must convey the prefix in ICMPv6 Router Advertisement (Type 134) messages using the Prefix Information Option. No additional prefixes or duplicate announcements shall be observed, ensuring the EVSE does not introduce extraneous prefixes beyond the EV–EVSE link.

Procedure:

1. Have the testing equipment initiate a charging session.

Expected Outcome(s):

- If the charger performs SLAAC or DHCPv6, it announces a single IPv6 prefix.
- Prefixes are conveyed in an ICMPv6 Router Advertisement (RA) message (Type 134, Code 0) using a Prefix Information Option.
- No additional prefixes or duplicate announcements shall be observed.

Scoring Criteria:

2 points: A single IPv6 prefix for SLAAC or DHCPv6, limited strictly to the EV–EVSE interface, is conveyed in ICMPv6 Router Advertisement (Type 134) messages using the Prefix Information Option.

0 points: Otherwise.

- **Score:** _____

02-EV-02.03: Auto Address Configuration Host Behavior Prohibition

Purpose: This case verifies that the charger does not configure or accept IPv6 addresses on the EV–EVSE interface. The charger must not originate Router Solicitations (ICMPv6 Type 133) or derive an IPv6 address from prefixes advertised by the EV. This ensures the EV cannot influence the charger’s addressing or routing table.

Procedure:

1. Connect the testing equipment to the charger’s EV–EVSE interface.
2. Observe ICMPv6 traffic on the EV–EVSE segment.

Expected Outcome(s):

- No IPv6 addresses are configured on the charger based on prefixes advertised by the EV.
- No ICMPv6 Router Solicitation (Type 133) messages are observed from the charger on the EV–EVSE interface.
- No Duplicate Address Detection (ICMPv6 Neighbor Solicitation, Type 135 and Neighbor Advertisement, Type 136) messages are observed from the charger on the EV–EVSE interface, confirming that it does not configure or validate an IPv6 address from an advertised prefix.

Scoring Criteria:

0 points: Any of the following occur:

- ICMPv6 Router Solicitation (Type 133) message from the charger on the EV–EVSE interface.
- Duplicate Address Detection (ICMPv6 Neighbor Solicitation, Type 135 and Neighbor Advertisement, Type 136) message from the charger on the EV–EVSE interface

2 points: Neither of the previous occur.

- **Score:** _____

02-EV-02.04: Inter-Interface Isolation

Purpose: The charger should not route network traffic between the EV–EVSE interface and its infrastructure interface. The charger must not forward packets or act

as an intermediary network device; traffic should be terminated or dropped at the interface boundary.

Pre-Condition(s): The following must be completed prior to performing this procedure:

- Configure the testing equipment with an additional IPv6 routable address (e.g., /64 or /128).
- Set the charger's EV–EVSE interface as the IPv6 default route on the testing equipment.

Procedure:

1. From the testing equipment, attempt to ping an arbitrary IPv6 address that is not part of the announced prefix.

Expected Outcome(s):

- No ICMPv6 echo requests or other packets from the EV–EVSE segment are forwarded by the charger to its infrastructure interface.
- Packet capture confirms that the charger does not route traffic between interfaces.

Scoring Criteria:

2 points: The charger does not forward ICMPv6 echo requests or other packets from the EV–EVSE segment to its infrastructure interface, and the packet capture confirms the charger does not route traffic between interfaces.

0 points: The charger forwards ICMPv6 echo requests or other packets from the EV–EVSE segment to its infrastructure interface, or packet capture confirms the charger routed traffic between interfaces.

- **Score:** _____

02-EV-02.05: Inter-Interface Switching Isolation

Purpose: This case verifies that the charger does not switch Layer 2 traffic between the EV interface and its infrastructure interface. The charger must not bridge frames or act as a switching device; all traffic should be terminated or dropped at the EV–EVSE boundary.

Pre-Condition(s): The following must be completed prior to performing this procedure:

- Identify two IPv4 addresses, *ip1* and *ip2*, within the same subnet.
- Configure the testing equipment (plc0) with *ip1*.
- Add a static MAC address entry for the second address and its associated hardware address.
- Enable monitoring of the charger's infrastructure interface.

Procedure:

1. From the testing equipment, transmit ICMP echo requests or Layer 2 frames sourced from *ip1* and directed to *ip2*.

Expected Outcome(s):

- Packet capture confirms that the charger does not switch or bridge traffic between interfaces.

Scoring Criteria:

2 points: The charger does not switch or bridge packets (compliant).

0 points: The charger switches or bridges packets (non-compliant).

- **Score:** _____

02-EV-02.06: Two Charging Port Isolation

Purpose: The charger should enforce strict isolation between its charging ports and does not forward or switch traffic between EV interfaces. This ensures that traffic from one connected EV cannot reach another.

Pre-Condition(s): The following must be completed prior to performing this procedure:

- Connect a set of testing equipment to each charging port.
- Record the IPv4/IPv6 addresses assigned to each testing system.
- Enable monitoring of both EV–EVSE interfaces.

Procedure:

1. Establish connectivity from both sets of testing equipment to their respective EV–EVSE interfaces.
2. From the first testing system, send ICMPv6 echo requests (and optionally other IPv4/IPv6 traffic) to the second testing system's address.
3. Repeat in the reverse direction (from the second testing system to the first).
4. Review packet captures on both EV–EVSE interfaces for evidence of forwarded traffic.

Expected Outcome(s):

- No ICMPv6 echo replies or other traffic from one testing system are observed on the other system.
- Packet captures confirm that the charger does not switch or bridge traffic between its charging ports.
- Each EV–EVSE interface remains logically isolated during concurrent connections.

Scoring Criteria:

2 points: No cross-system traffic is observed (e.g. ICMPv6 echo replies), EV–EVSE interfaces are logically isolated.

0 points: Cross-system traffic is not observed (e.g. ICMPv6 echo replies), EV-EVSE interfaces are not logically isolated.

- **Score:** _____

02-EV-02.07: ARP/NDP Response Isolation Test

Purpose: The charger should not respond to ARP (IPv4) or NDP (IPv6) requests for infrastructure-facing IP addresses when such requests originate from the EV-EVSE interface. This ensures Layer 2/Neighbor Discovery isolation between EV and infrastructure networks.

Pre-Condition(s): The following must be completed prior to performing this procedure:

- The charger shall be powered on and have its infrastructure interface configured with valid IPv4 and IPv6 addresses
 - If the charger does not support IPv6 configuration, omit the NDP tests.
- Test equipment has the ability to send and capture ARP (for IPv4) and NDP (for IPv6) traffic from the EV-EVSE side.

Procedure:

1. Identify infrastructure interface IP address(es)
2. From configuration interface, obtain the IPv4 and/or IPv6 addresses assigned to the charger's infrastructure network interface(s).
3. Record each address for testing.
4. Prepare EV-EVSE test interface
5. Connect a test system to the EV-EVSE communication interface in place of, or in parallel with, an EV.
6. Confirm that the test system can send Layer 2 packets on this segment.
7. Begin packet capture
8. Send ARP/NDP requests
9. For each IPv4 address recorded, send a standard ARP request from the EV-EVSE side asking "Who has <infrastructure_IP>?"
10. For each IPv6 address recorded, send a Neighbor Solicitation (NDP) request for that address from the EV-EVSE side.

Expected Outcome(s):

- Pass: No ARP Reply or NDP Neighbor Advertisement is received from the charger in response to the requests.
- Fail: Any ARP Reply or NDP Neighbor Advertisement is received from the charger for the tested infrastructure-facing IP addresses.

Scoring Criteria:

2 points: ARP Reply and NDP Neighbor Advertisement are not received from charger.

0 points: ARP Reply or NDP Neighbor Advertisement is received from charger.

- **Score:** _____

02-EV-02.08: Infrastructure Address Isolation

Purpose: This case is intended to verify that the charger does not respond to ARP (IPv4) or NDP (IPv6) requests for infrastructure-facing IP addresses when such requests originate from the EV–EVSE interface. This ensures Layer 2/Neighbor Discovery isolation between EV and infrastructure networks.

Pre-Condition(s): The following must be completed prior to performing this procedure:

- Prepare EV-EVSE testing equipment:
 - Ensure capability of running `nmap` for network scanning
 - Assign an IPv4 address to the testing equipment's EV–EVSE interface
 - Ensure the assigned IPv4 address is within the same subnet and compatible with the charger's infrastructure interface IPv4 address.
- Initiate a packet capture to monitor traffic on the EV–EVSE interface.
 - Note: This may be performed either on the testing equipment or via an inductive collar with a CCS listener.

Procedure:

1. From the testing equipment, perform a TCP/UDP service enumeration of the charger's infrastructure interface IPv4 address using a network scanning tool such as `nmap`.

Expected Outcome(s):

- No infrastructure-facing services should be discovered on the charger when scanned from the EV–EVSE interface.
- Packet capture should confirm that the charger blocks or drops traffic from the EV–EVSE segment attempting to reach infrastructure services.

Scoring Criteria:

2 points: No infrastructure-facing services are discovered on the charger when scanned from the EV-EVSE interface.

0 points: Infrastructure-facing services are discovered on the charger when scanned from the EV-EVSE interface.

- **Score:** _____

2.4.4.3 ISO 15118-2

Background: ISO 15118-2 is a key part of the ISO 15118 standard series that defines the communication between an EV and EVCS during the charging process. It specifies the application-layer protocol and security requirements for Plug & Charge (PnC) functionality and secure data exchange using TCP/IP over a Power Line Communication (PLC) link (GreenPHY). This layer is responsible for enabling mutual authentication, encrypted communication, and exchange of contract certificates between the EV and EVCS.

The security model of ISO 15118-2 relies heavily on TLS and X.509 certificates, making it essential to validate not only the functional behavior of the protocol but also the cryptographic configuration, certificate handling, and network exposure of the charging system. These cases are designed to evaluate the correct implementation and secure handling of ISO 15118-2 sessions, including certificate presentation, TLS parameter enforcement, and resilience against misconfiguration or legacy protocol fallback.

15118-2 TLS Server

Background: This assessment focuses on the behavior of ISO 15118-2 when operating over TLS, evaluating whether the implementation adheres to secure communication practices. It involves measuring the TLS configuration of the server to verify that only secure protocol versions are enabled (e.g., TLS 1.2 or higher), weak cipher suites are excluded, and additional security features, such as strong key exchange groups and the absence of renegotiation or compression, are enforced. The goal is to ensure that the communication channel established under ISO 15118-2 maintains the confidentiality, integrity, and authenticity of messages in accordance with modern cryptographic standards and best practices.

The first set of procedures is designed to confirm that the ISO 15118-2 server can be configured with its own X.509 certificate and private key, separate from those used by other applications such as OCPP and the management interface.

Assumption(s): Perform the procedures defined within this subsection only if the following hold true:

- EVCS and CSMS are online and reachable over OCPP 2.0.1.
- EVCS and (simulated) EV are reachable over ISO 15118-2
- A certificate authority is shared between the EVCS and CSMS.
- The EVCS is capable of assigning and managing separate TLS certificates for different services (e.g., ISO 15118-2 server, OCPP interface, and management interface).

02-EV-03.01: TLS Certificate Presentation Verification

Purpose: This case verifies that the ISO 15118-2 server can be configured with a certificate specific to ISO 15118-2.

Pre-Condition(s): The following must be completed prior to performing this procedure:

- Generate P-256 private key and provision an associated X.509 certificate.
- Generate a second key and certificate pair for the management interface to ensure separation of credentials.
- AcCCS is configured to establish the PLC/GreenPHY network but does not proceed with connecting to the charger's ISO 15118-2 server. The AcCCS environment includes access to both `ssllscan` and `sdpquery` tools.

Procedure:

1. Configure the charger's ISO 15118-2 server to use the generated private key and corresponding certificate, and configure the management interface to use the second key and certificate pair
2. Initiate AcCCS to establish the network
3. On AcCCS, use `sdpquery` to identify the network port associated with the ISO 15118 TLS server.
4. Verify that the server presents the correct certificate during the TLS handshake using the `ssllscan` methodology described later in this document.

Expected Outcome(s): The certificate presented by the server matches the one that was configured for ISO 15118-2.

Scoring Criteria:

2 points: Certificate presented by server is a match for ISO 15118-2 certificate.

0 points: Certificate presented by server is not a match for ISO 15118-2 certificate.

- **Score:** _____

02-EV-03.02: Secure TLS validation

Purpose: This case is designed to ensure that the server does not support or present weak SSL/TLS protocols or outdated protocol versions. This case verifies that only secure and current versions, such as TLS 1.2 or TLS 1.3, are enabled, while insecure versions like SSL 2.0, SSL 3.0, TLS 1.0, and TLS 1.1 are explicitly disabled. Disabling legacy protocols reduces the attack surface and helps prevent downgrade attacks that could compromise the confidentiality and integrity of communications.

Procedure:

1. Review the `ssllscan` results reported in procedure *02-EV-03.01*.

Expected Outcome(s): The TLS session complies with the parameters defined in Appendix B.1.

Scoring Criteria:

2 points: The server supports TLS v1.2,²² with compression disabled, and provides all the following cipher suites:

- TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384
- TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256
- TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA256

Optional (may also support):

- TLS_ECDHE_ECDSA_WITH_CHACHA20_POLY1305_SHA256
- TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA384

And other cipher suites returned by the command:

```
openssl ciphers -s -v -tls1_2 "ECDHE+ECDSA:@SECLEVEL=3"
```

All groups and curves used shall provide no less than 256-bit security and include Curve25519, P-256, P-384, Curve448, and P-521.

0 points: Does not meet the above criteria.

- **Score:** _____

Note: TLS_ECDH_ECDSA_WITH_AES_128_CBC_SHA256 is required by ISO 15118-2. *The use of TLS_ECDH_ECDSA_WITH_AES_128_CBC_SHA256 is discouraged in draft-ietf-tls-deprecate-obsolete-kex-06 Deprecating Obsolete Key Exchange Methods in (D)TLS 1.2, which proposes deprecating obsolete key exchange methods in TLS 1.2. Similar guidance is proffered by RFC 9325 Recommendations for Secure Use of Transport Layer Security (TLS) and Datagram Transport Layer Security (DTLS). Support for this cipher suite was removed from the OpenSSL TLS library in 2016, and it is likely that other mainstream libraries have followed suit. As a result, if this cipher suite is present, it likely indicates the use of an outdated TLS library, which may contain known vulnerabilities.*

02-EV-03.03: TLS Session Key Exposure

Purpose: Many EV charger vendors offer a diagnostic feature that exports ISO 1518 TLS session keys in a UDP packet,²³ enabling tools such as Wireshark to decrypt

²² ISO 15118-2 normatively requires TLS 1.2 for EV–EVSE communication, without a path to adopt TLS 1.3.

²³ Commonly used formats for reporting TLS session keys include the NSS Key Log Format (https://nss-crypto.org/reference/security/nss/legacy/key_log_format/index.html) and the SSLKEYLOGFILE

encrypted traffic for troubleshooting purposes. This feature is typically used to diagnose communication or protocol issues between vehicle and charger. In some cases, these UDP packets have been observed on the network, an unfortunate occurrence that exposes sensitive session keys.

While this feature should be customer-selectable (meaning it can be enabled or disabled by the end user or site operator), this case confirms key protection and verifies this feature's configurability.

Assumption(s): Perform this procedure only if the following hold true:

- Conductive collar, CCS listener,²⁴ and Wireshark are available for capturing and analyzing traffic.
- A Certificate Authority (CA) is trusted by both the charger and test system.
- A conductive collar is installed around the charging cable to enable passive monitoring.

Pre-Condition(s): The following must be completed prior to performing this procedure:

- Configure the charger to use TLS, with a valid certificate provisioned.
- If the feature is selectable, configure the charger to report TLS session keys.
- Configure AcCCS to initiate a TLS connection to the charger.

Procedure:

1. Start AcCCS and initiate a charging session with the charger.
2. Once the charging session is established, review the packet capture for any unusual messages that may contain TLS session keys.
3. If such a networked message is found, attempt to use its contents in Wireshark to decrypt the TLS traffic.²⁵

Expected Outcome(s): Only related messages are found.

Scoring Criteria:

2 points: No session keys are found.

0 points: UDP packet containing session keys are found.

- **Score:** _____

(<https://datatracker.ietf.org/doc/html/draft-ietf-tls-keylogfile>). The two are very similar and often interchangeable.

²⁴ Example CCS listening devices include the Vector Informatique's VH5110A and Keysight's SL1556.

²⁵ Guidance for TLS Wireshark decryption can be found at <https://wiki.wireshark.org/TLS#tls-decryption>.

2.4.4.4 PnC – Authorization

Background: This unit focuses on verifying the security posture of the Plug and Charge (PnC) authorization process implemented within the EV charging station. PnC enables automated mutual authentication between the electric vehicle and the charging station using contract certificates, making it a critical component of secure EV charging infrastructure. To ensure that this process is not vulnerable to unauthorized access or impersonation, the tester must validate the handling of various certificate-related scenarios.

The procedures defined in this unit evaluate whether the charging station correctly enforces cryptographic and certificate validation requirements as specified by ISO 15118 and the V2G PKI framework. The following controls are essential to ensure secure, standards-compliant PnC operation and to prevent malicious or misconfigured clients from initiating a charging session:

- Compliance with the ISO 15118-2 `ContractCertificateChain` profile
- Trust chain validation against the `MORootCertificate` / SAE Certificate Trust List (CTL)
- Use of secure cryptographic algorithms (e.g., P-256)
- Certificate status verification for expiration and revocation (via OCSP/CRL)
- Electric Mobility Account Identifier (EMAID) validation

Note: A “valid” contract certificate refers to a certificate meeting all requirements defined in ISO 15118-2, including proper structure, cryptographic parameters, trust anchor, validity period, and status. In cases where an invalid certificate is tested, the certificate is deliberately misconfigured to fail a single targeted requirement (e.g., algorithm, trust, or revocation status), while satisfying all other validation criteria.

Assumption(s): Perform the procedures defined within this unit, 02-EV-04, only if the following hold true:

- EV and EVCS allow/support PnC
- EV, EVCS, and CSMS are online
- EVSE connected to CSMS, Security Profile 2

02-EV-04.01: Secure PnC (Authorization)

Purpose: This case verifies that the EV charging station correctly handles a complete and secure Plug and Charge session using a valid contract certificate. It ensures compliance with ISO 15118-2 security requirements, including proper TLS handshake, certificate validation, and communication with the CSMS. This procedure demonstrates expected behavior under ideal, secure conditions.

Assumption(s): Perform this procedure only if the following hold true:

- EV has a valid p256 contract certificate installed.

Procedure:

1. Plug EV into charger
2. Capture logs/traffic
3. Un-plug EV once charging is complete
4. Analyze logs to verify the following:
 - a. The EV sends a contract certificate chain with secp256r1
 - b. ContractCertificateChain signature verification
 - c. Live eMAID validation

Expected Outcome(s): Successful authorization and PnC session without any WARNING_ responses.

Expected traffic summation:

- TLS handshake (from ISO 15118-2 TLS Case)
- Authorization:
 - o EV sends p256 ContractCertificateChain
 - o SECC validates ContractCertificateChain against MORootCertificate / SAE CTL
 - o Verify signature
 - o Forward eMAID and certificate data to CSMS for validation
 - o CSMS checks revocation status with OCSP

Scoring Criteria:

2 points: Authorization is successful, PnC session has no WARNING_ response

0 points: Authorization fails or PnC session contains WARNING_ response

- **Score:** _____

02-EV-04.02: Insecure (algorithm) contract certificate

Purpose: This case is to verify that the SECC will not use a contract certificate that supports insecure algorithms and that if this is not met, the session or authorization should fail, preventing charging. In this test, secp192r1 is implemented as the insecure algorithm, however, other insecure algorithms exist and can be tested.

Pre-Condition(s): The following must be completed prior to performing this procedure:

- A secp192r1 ContractCertificateChain, signed by the unit-level certificate authority has been issued.

Procedure:

1. Plug EV into charger

2. Un-plug EV once charging is complete
3. Analyze EVSE and CSMS logs

Expected Outcome(s): Authorization fails, the session is terminated, and no energy is transferred.

- CSMS reports an error message similar to “FAIL: Signature verification failed (invalid key curve)”

Scoring Criteria:

2 points: Authorization fails, session is terminated, and energy is not transferred

0 points: Authorization is successful, session is not terminated, or energy is transferred.

- **Score:** _____

02-EV-04.03: Invalid Trust

Purpose: This case verifies that the EV charging station correctly rejects a contract certificate that is not signed by a trusted Certificate Authority (CA). Specifically, it ensures that the charger enforces trust chain validation against the recognized list of CAs defined in the SAE Certificate Trust List (CTL).

Pre-Condition(s): The following must be completed prior to performing this procedure:

- A `secp256r1` contract certificate signed with test CA not in SAE CTL is deployed on the EV.

Procedure:

1. Plug EV into charger
2. Capture logs/traffic
3. Un-plug EV once charging is complete
4. Analyze logs and traffic

Expected Outcome(s): Authorization fails and power transfer doesn't commence.

- OCPP `AuthorizeResponse` with “ContractCancelled” and `idToken` status “Invalid or Unknown”

Scoring Criteria:

2 points: Authorization fails, and power transfer doesn't commence.

0 points: Authorization is successful, or power transfer is observed.

- **Score:** _____

2.4.5 Integration (INT)

2.4.5.1 MQTT

Background: If MQTT is used by the operator to integrate systems such as meter readings, payment telemetry, or other backend services, a configuration switch to enable MQTT should be present and set to off by default. This ensures the service is only activated when explicitly needed and properly secured.

Assumption(s): Perform the procedures defined within this unit, *02-INT-01*, only if the following hold true:

- The charger supports MQTT communication, operating as either a broker or a client.

02-INT-01.01: Network Surface

Purpose: This procedure ensures that MQTT is disabled by default, therefore, not active or externally accessible.

Procedure:

1. Review network surface surveys conducted during the early stages.

Expected Outcome(s): In the default configuration, TCP port 1883 (MQTT) or TCP port 8883 (MQTT over TLS) should not be active or externally accessible.

Scoring Criteria:

2 points: Both TCP port 1883 and TCP port 8883 are not active or externally accessible.

0 points: TCP port 1883 or TCP port 8883 is active or externally accessible.

- **Score:** _____

02-INT-01.02: Network Surface Re-enumeration

Purpose: This case ensures that the network attack surface remains consistent after enabling MQTT. It specifically validates that no additional or unintended ports are exposed on either the EV-facing or infrastructure-facing interfaces following re-enumeration, and that MQTT services remain properly constrained to their expected network footprint.

Assumption(s): Perform this procedure only if the following hold true:

- The charger includes an MQTT broker.

Procedure:

1. Configure MQTT broker:
 - a. Enable the MQTT broker.
 - b. Enable TLS to secure MQTT communications.
 - c. Disable all non-TLS (unencrypted) connections.

Note: If any of these configurations are not supported or cannot be applied, the implementation should be considered deficient.

2. Re-enumerate the network surface.

Expected Outcome(s):

- MQTT broker configuration is successful
- No new ports appear on the EV–EVSE interface.
- Only a single MQTT-related port is visible on the infrastructure interface.

Scoring Criteria:

2 points: No new ports appear on EV-EVSE interface. Only one MQTT-related port (TCP port 8883) is visible.

0 points: New ports appear on EV-EVSE interface. More than one MQTT-related port is visible.

- **Score:** _____

02-INT-01.03: TLS Configuration Validation

Purpose: This case validates that the MQTT broker enforces secure TLS configurations.

Assumption(s): Perform this procedure only if the following hold true:

- A scanning tool (e.g., `ssllscan`, `testssl.sh` or `nmap --script ssl-*`) is available and permitted to run in the test environment.

Pre-Condition(s): The following conditions must be completed prior to performing this procedure:

- Enable the MQTT broker to actively listen on a TLS-secured port (e.g., TCP 8883).

Procedure:

1. Run `ssllscan` against the MQTT broker.

Expected Outcome(s): Results should conform to the TLS configuration guidance provided in Appendix B.1.

Scoring Criteria:

2 points: Results do conform to TLS configuration guidance provided in Appendix B.1.

0 points: Results do not conform to TLS configuration guidance provided in Appendix B.1.

- **Score:** _____

02-INT-01.04: Credentialed Access Verification

Purpose: This case verifies that only credentialed users can establish a secure connection to the MQTT broker, and that all provisioned usernames and passwords function as intended. This additionally ensures that the MQTT broker enforces strict authentication by rejecting all anonymous connection attempts. It validates that a username and password are required to establish a connection and that unauthenticated access is explicitly denied. This helps ensure that unauthorized entities cannot access or interact with the messaging system.

Assumption(s): Perform this procedure only if the following hold true:

- A non-provisioned username and password are known username and password.

Procedure:

1. Attempt to connect to the MQTT broker without providing a username or password.
2. Set a username and password without MQTT access.
3. Set a username and password with MQTT access.
4. Attempt to connect using a non-provisioned username and password.
5. Attempt to connect using the non-credentialed user from Step 2.
6. Attempt to connect using the provisioned username and password from Step 3.
 - a. If successful, end connection and continue to next step
7. Attempt to connect using the provisioned usernames and an invalid password.

Expected Outcome(s): Only Step 5 successfully establishes a secure session.

- Steps 1, 4, and 5: A secure session is denied.

Scoring Criteria:

2 points: Each credentialed user successfully establishes secure session and a secure session is not established using the non-provisioned user, non-credentialed user, and invalid password.

0 points: The credentialed user fails to establish secure session or a secure session is established without a username/password or using a non-provisioned user, non-credentialed user, or invalid password.

- **Score:** _____

2.4.5.2 Modbus

Background: If Modbus is available, it must be disabled by default. This ensures the protocol is not unintentionally exposed to the network, reducing the risk of unauthorized access or control. Modbus lacks built-in authentication and encryption, making it a common target for exploitation when left enabled without adequate safeguards. Enabling Modbus should require an explicit configuration change and be accompanied by appropriate access controls and network segmentation.

Assumption(s): Perform the procedures defined within this unit, *02-INT-02*, only if the following hold true:

- All procedures in Unit *02-INF-01* have been performed, and the network surveys have been recorded and are accessible.

02-INT-02.01: Network Service Survey

Purpose: This case ensures that Modbus is disabled by default.

Procedure:

1. Review network surface surveys conducted during the early stages.

Expected outcome(s): In the default configuration, Modbus (default: TCP Port 502) is not active nor externally accessible.

Scoring Criteria:

2 points: TCP Port 502 is inactive, and Modbus is unavailable and inaccessible.

0 points: TCP Port 502 is active, or Modbus is available or accessible.

- **Score:** _____

2.4.6 TLS

Background: Transport Layer Security (TLS) is a foundational protocol for securing communications across charger interfaces, including EV–EVSE interactions (via ISO 15118-2/20), CSMS integration (via OCPP), remote management interfaces (such as MQTT or HTTPS), and vendor/operator diagnostics. TLS ensures confidentiality, integrity, and authenticity of transmitted data by providing encryption, mutual authentication (where configured), and certificate validation.

TLS is critical in contexts where the charger acts as:

- A TLS client (e.g., connecting to a CSMS over OCPP, uploading logs or firmware updates to a remote file server),
- A TLS server (e.g., accepting connections from the EV under ISO 15118-2/20 or from a browser or MQTT client).

2.4.6.1 TLS Client

Background: This unit evaluates the charger's TLS handshake behavior across interfaces that require secure transport. It examines negotiation parameters, protocol version selection, cipher-suite restrictions, and fallback handling. The objective is to confirm that the charger consistently initiates or accepts TLS sessions with modern cryptographic methods and rejects insecure or deprecated configurations.

This unit ensures that session parameters are supported, and secure defaults are enforced (e.g., TLS 1.3 preferred, legacy protocols are disabled, and weak ciphers are not negotiated).

Pre-Condition(s): The following must be completed prior to performing the procedures defined within this unit, *02-TLS-01*:

- A private key (`server.key`) has been generated.
- A CA has issued a certificate (`server.crt`) based on a CSR created from `server.key`, ensuring that the certificate and key form a valid pair.
- The Subject Alternative Name (SAN) in `server.crt` includes the OpenSSL testing tool's hostname or IP address.
- The charger is configured to trust the issuing CA.
- The charger can establish a network connection to the tester's OpenSSL server using one of the identities specified in the SAN field.

02-TLS-01.01: TLS 1.3 Support

Purpose: This case verifies that the charger establishes a TLS 1.3 session when initiating a connection to a server.

Procedure:

1. Start the `openssl` server:

```
openssl s_server -accept 443 -cert server.crt -key server.key -
tls1_3 -www -state -msg
```

1. From the charger connect to the `openssl` server.
2. Review the output of the command to confirm a TLS 1.3 connection.

Expected Outcome(s): A TLS 1.3 connection is established between charger and server.

Scoring Criteria:

2 points: The connection is established between charger and server over TLS 1.3.

0 points: The TLS 1.3 connection is unsuccessful or established with a lower TLS version. Log score as 0, then move on to next unit.

- **Score:** _____

02-TLS-01.02: TLS 1.2 Fallback

Purpose: This case confirms the charger falls back to TLS 1.2 if TLS 1.3 cannot be negotiated.

Procedure:

1. Start the `openssl` server:

```
openssl s_server -accept 443 -cert <server.crt> -key  
<server.key> -www -state -msg -min_protocol TLSv1.2 -  
max_protocol TLSv1.3
```

2. From the charger connect to the `openssl` server.
3. Review the output of the command to confirm a TLS 1.2 connection.

Expected Outcome(s): A TLS 1.2 is established between charger and server.

Scoring Criteria:

2 points: A TLS 1.2 connection is successfully established between charger and server.

0 points: Otherwise.

- **Score:** _____

02-TLS-01.03: Enforce Minimum TLS 1.2 Support

Purpose: This case confirms the charger enforces a minimum TLS version of 1.2.

Procedure:

1. Start the `openssl` server:

```
openssl s_server -accept 443 -cert server.crt -key server.key -  
tls1.1 -www -state -msg
```

2. From the charger connect to the `openssl` server

Expected Outcome(s):

- No TLS connection is established.
- The charger logs or reports a failure indicating that the TLS version is unsatisfactory.

Scoring Criteria:

2 points: No TLS connection established; Charger logs or reports failure due to unsatisfactory TLS version.

0 points: TLS connection established. Charger does not log or report failure due to unsatisfactory TLS version.

- **Score:** _____

02-TLS-01.04: TLS Ciphers

Purpose: This case verifies that cipher suites recommended in RFC 9325 are accepted and that any non-compliant suites are rejected.

Procedure:

1. Start an `openssl` server:

```
openssl s_server -accept 443 -cert <server.crt> -key  
<server.key> -www -state -msg -min_protocol TLSv1.2 -  
max_protocol TLSv1.3
```

2. Direct the charger to connect to the server and inspect the trace log

Expected Outcome(s): Trace log entry “`cipher_suites`” includes ciphers detailed in Appendix B.1.

Scoring Criteria:

2 points: Trace log entry for “`cipher_suites`” contains only and all secure and recommended algorithms listed in Appendix B.1.

0 points: Trace log entry for “`cipher_suites`” contains any ciphers not recommended in Appendix B.1.

- **Score:** _____

02-TLS-01.05: Certificate SAN Mismatch

Purpose: A TLS connection should fail to establish due to certificate validation and that the condition is logged when the presented certificate’s Subject Alternative Name (SAN) does not match the connection URL.

Pre-Condition(s): The following conditions must be met prior to performing this procedure.

- Two certificates are issued from the testing CA:
 - A valid certificate whose SAN matches the connection URL.
 - An *invalid* certificate whose SAN does not match the connection URL.

Procedure:

1. With the *invalid* certificate installed on the server, connect to the server

Expected Outcome(s): The connection is not established, and the condition and associated reason (certificate SAN mismatch) are logged.

Scoring Criteria:

2 points: Connection failed and reason logged.

0 points: Otherwise.

- **Score:** _____

2.4.6.2 TLS Key Management and Certificate Provisioning

Background: The charger may need to be provisioned with multiple digital certificates to support functions such as secure EV–EVSE communication, network authentication, and integration with third-party services. To perform this securely, the operator should not be required to upload a private key—doing so would expose a sensitive asset and increase the risk of compromise. Instead, the charger’s management interface should provide the capability to generate an X.509 Certificate Signing Request (CSR) locally within the device.

A CSR is a standardized message containing the public key to be certified along with identifying information such as the organization name, domain name, and other attributes. The charger securely generates its own private key and uses it to sign the CSR, proving possession without exposing the key. The operator then transmits the CSR to a trusted Certificate Authority (CA), which validates the request and issues the corresponding certificate. This process ensures the private key remains securely stored within the charger’s cryptographic module, maintaining operational security and aligning with public key infrastructure (PKI) best practices.

If the charger lacks the capability to generate a CSR, it does not meet the requirements for secure certificate provisioning.

Pre-Condition(s): The following must be completed prior to performing the procedures defined within this unit, *02-TLS-02*:

- The management interface is set as the target.²⁶

02-TLS-02.01: Certificate Provisioning

Purpose: This case verifies that the charger supports secure and standards-compliant X.509 certificate provisioning using a CSR process. The procedure ensures the charger can generate a CSR internally—without exposing the private key—and correctly install and associate a signed certificate issued by a trusted CA. Proper certificate provisioning is essential for secure TLS communication across EVSE interfaces.

Assumption(s): Perform this procedure only if the following hold true:

- The test environment includes tooling to inspect and parse CSR files (e.g., `openssl`, `cfssl`, or similar).

²⁶ If the charger lacks a web management interface or the management does not support TLS (note: this is considered a deficient), an alternative interface may be used.

- The charger's management interface is accessible and operational (via HTTPS or local console).
- The charger is configured to generate and manage private keys locally, without exporting them.
- The charger supports certificate installation through its UI, API, or CLI.

Pre-condition(s): The following conditions must be met prior to performing this procedure.

- A valid test CA (internal or external) is available to issue certificates.

Procedure:

1. Use the charger's management interface to initiate a CSR generation process
2. Ensure the charger generates its own private key internally and does not export it.
3. Verify that the CSR contains the correct public key and identifying attributes (e.g., organization name, common name, domain name) for the intended certificate use.
4. Submit the CSR to the appropriate CA and request issuance of a certificate.
5. Using the management interface, upload the signed certificate to the appropriate certificate store within the charger. Note any warnings.
6. Use diagnostic tools or logs to confirm that the charger is presenting the newly installed certificate when initiating secure connections.
7. Check that the certificate validates correctly against the CA trust chain.

Expected Outcome(s):

- CSR Generation
 - The charger successfully generates a CSR without exporting the private key.
 - CSR content matches the intended configuration (correct public key, common name, organization, and other attributes).
 - CSR file format complies with X.509/PKCS#10 standards and can be parsed by standard tools (e.g., `openssl req -in file.csr -noout -text`).
- CA Signing
 - The CA successfully validates and signs the CSR.
- Certificate Installation
 - The charger accepts the signed certificate without errors.
 - The certificate is correctly linked to the private key generated during the CSR process.
 - The charger logs confirm successful installation and readiness for use.
- Operational Verification
 - During secure communications (e.g., TLS handshake), the charger presents the newly installed certificate.
 - The certificate validates successfully against the configured CA trust chain.
 - No errors or warnings appear in client or server logs related to certificate trust or mismatched keys.

Scoring Criteria:

2 points: All of the following are true:

- The charger successfully generates a CSR without exporting the private key.
- The CSR content matches the intended configuration, including the correct public key, common name, organization, and other attributes.
- The CSR complies with X.509/PKCS#10 standards
- The CA validates and signs the CSR without issues.
- The charger accepts the signed certificate and logs successful installation.
- The charger presents the newly installed certificate and is successfully validated against the configured CA trust chain.
- No errors or warnings appear in client or server logs.

0 points: Otherwise.

- **Score:** _____

2.5 Cybersecurity Analysis and Scoring

From the scoring results determined in the previous section, enter the values into the “Raw Score” column in the following table. Then using the weighting factor values, calculate the “Final Score” for each analysis criteria. Then calculate the sum “Total Score” for this section of assessment.

Table 4. Communications and Networking Scoring Table

<i><u>Communications and Networking Scoring Table</u></i>	<i>Raw Score (0,1,2)</i>	<i>Weight Factor</i>	<i>Final Score</i>
2.4.1			
<i><u>Network Surface</u></i>			
02-INF-01.01: Survey Network Surface		5	
<i><u>Offline Communication</u></i>			
02-INF-02.01: Deferred Authorization is Disabled By Default		2	
02-INF-02.02: Offline Authorization—Deferred Authorization of RFID		1	
02-INF-02.03: Deferred Authorization – CSMS Priority over cached tokens		1	
2.4.2			

<u>Session Establishment</u>			
02-OCPP-01.01: Secure TLS		10	
02-OCPP-01.02: OCPP 2.0.1 Support		10	
02-OCPP-01.03: OCPP 1.6j and Security Extension Support		2	
02-OCPP-01.04: Disallow OCPP Version Degradation		2	
02-OCPP-01.05: OCPP 2.0.1 Security Profile Enforcement		10	
02-OCPP-01.06: CSMS Root Certificate Authority Is Prioritized		1	
02-OCPP-01.07: (OCPP) Charging Station Certificate Usage		1	
<u>Secure File Transport</u>			
02-OCPP-02.01: Secure Transfer Protocols Are Enabled By Default		5	
02-OCPP-02.02: Unadvertised File Transfer Protocols		1	
02-OCPP-02.03: Trusted CA – Successful File Transfer		1	
02-OCPP-02.04: Untrusted CA – Rejection		1	
<u>Firmware Update</u>			
02-OCPP-03.01: Firmware Updates Disabled by Default		1	
02-OCPP-03.02: Enforcement of Signature Presence in FirmwareUpdate Requests		1	
2.4.3			
<u>Operator Access and Identity Management</u>			
02-OM-01.01: Centralized Identity and Sign-On Integration		1	
<u>SSH</u>			

02-OM-02.01: Network Survey Review		1	
02-OM-02.02: Establish SSH Connection		1	
02-OM-02.03: Legacy SSH Versions Not Supported		1	
02-OM-02.04: Secure Cryptographic Methods		1	
02-OM-02.05: Public-Key Authentication Functionality		1	
02-OM-02.06: Invalid Public-Key Authentication Behavior		1	
02-OM-02.07: Login Grace Time Functionality		0.6	
02-OM-02.08: Reject Empty Passwords		1	
02-OM-02.09: Reject Root Login		1	
02-OM-02.10: Limit Failed Logins		0.8	
02-OM-02.11: Port Forwarding Disabled		1	
02-OM-02.12: Inactivity Timeout		0.6	
02-OM-02.13: Login SecurityEventNotification		1	
2.4.4			
<u>Network Surface</u>			
02-EV-01.01: Survey Network Surface		1	
<u>Multihoming</u>			
02-EV-02.01: Interface Isolation Enforcement		1	
02-EV-02.02: IPv6 Router Advertisement Single Prefix		1	
02-EV-02.03: Auto Address Configuration Host Behavior Prohibition		1	
02-EV-02.04: Inter-Interface Isolation		1	

02-EV-02.05: Inter-Interface Switching Isolation		1	
02-EV-02.06: Two Charging Port Isolation		1	
02-EV-02.07: ARP/NDP Response Isolation Test		1	
02-EV-02.08: Infrastructure Address Isolation		1	
<u>ISO 15118-2</u>			
02-EV-03.01: TLS Certificate Presentation Verification		2	
02-EV-03.02: Secure TLS validation		2	
02-EV-03.03: TLS Session Key Exposure		2	
<u>PnC – Authorization</u>			
02-EV-04.01: Secure PnC (Authorization)		2	
02-EV-04.02: Insecure (algorithm) contract certificate		1	
02-EV-04.03: Invalid Trust		1	
2.4.5			
<u>MQTT</u>			
02-INT-01.01: Network Surface		5	
02-INT-01.02: Network Surface Re-enumeration		2	
02-INT-01.03: TLS Configuration Validation		1	
02-INT-01.04: Credentialed Access Verification		1	
<u>Modbus</u>			
02-INT-02.01: Network Service Survey		1	
2.4.6			

<u>TLS Client</u>			
02-TLS-01.01: TLS 1.3 Support		5	
02-TLS-01.02: TLS 1.2 Fallback		1	
02-TLS-01.03: Enforce Minimum TLS 1.2 Support		5	
02-TLS-01.04: TLS Ciphers		1	
02-TLS-01.05: Invalid TLS certificate SAN		1	
<u>TLS Key Management and Certificate Provisioning</u>			
02-TLS-02.01: Certificate Provisioning		1	
Total Score		-	

2.5.1 Interpretation of Scores

- **187- or more points:** Excellent implementation of security measures.
- **150-186 points:** Good implementation, with minor areas for improvement.
- **112-149 points:** Moderate implementation, requiring significant improvements.
- **56-111 points:** Poor implementation, with critical areas needing attention.
- **0-55 points:** Very poor implementation, indicating severe security risks.

This scoring method allows for a systematic evaluation of the hardware security level based on the defined metrics and measurements.

3 EVSE Energy Management Cybersecurity Evaluation

3.1 Overview

Energy management systems for EVSE are broadly responsible for monitoring, controlling, and optimizing charging. The associated information systems exchange data covering billing, metering, energy savings, and more, and rely on communication protocols for effective management. To test the security of the information exchanges required for controlled and safe energy management, the following procedures assess encryption vulnerabilities for energy management systems using OpenADR, OCPP, and MQTT protocols.

The initial section outlines metrics, measurements of efficacy, and a scoring system to enable measurement of the cybersecurity posture of the system under assessment. The subsequent section describes procedures and methods for testing encryption under each protocol to ensure secure transmission of data and energy flow.

3.2 Metrics, Measurements, and Scoring

The metrics and measurements that are used for EVSE energy management cybersecurity evaluation is in Table 5. The provided scoring system is designed to evaluate the implementation of critical security measures through a structured and quantifiable approach. Each security metric is assessed using specific measurements, which are rated on a scale from 0 to 2 points: Fully Implemented (2 points), Partially Implemented (1 point), and Not Implemented (0 points). By summing up the points for each measurement, a total score is calculated. This total score reflects the overall security posture of the EVCI energy management systems. The scoring system provides a clear interpretation of the results, enabling organizations to identify strengths and weaknesses in their hardware security implementations and prioritize areas for improvement.

Table 5. EVSE Energy Management Metrics and Measurements

Metric	Description	Measurements
Presence of plaintext traffic	Check if there is any plaintext traffic.	Yes/No
Percentage of unencrypted traffic	Percentage of total captured traffic transmitted without encryption	Capture the traffic for a certain amount of time Calculate percentage of total captured traffic transmitted without encryption.

Types of exposed data	Types of data without encryption	Check if the types of unencrypted data are sensitive data such as credentials, tokens, identifiers, control commands, PII, etc.
Successful interception attempts	Measures how easy it is to intercept data	Calculate the ratio of the number of successful interceptions and the total number of attempts.
Time required to intercept	Measures speed of unencrypted data interception	Measure time in seconds to get readable, plaintext data after data interception.

3.3 Procedures

Each case includes procedures to verify conformance with communication standards, assess behavior under normal and degraded conditions, and evaluate the charger's response to events such as loss of connectivity, protocol mismatches, and network reconfiguration. Validation of secure communication channels, timing requirements, and fault-handling mechanisms is also included.

These procedures ensure the charger operates predictably within a connected ecosystem, supports interoperability, and maintains cybersecurity and service availability across both typical and adverse network conditions.

3.3.1 Open ADR

OpenADR enables information exchange and plays a role in regulating charging of electric vehicles through communication between charger and vehicle. Network traffic over electric vehicle supply equipment (EVSE) requires encryption via transport layer security (TLS). This test seeks to verify the encryption of traffic between a VTN and a VEN in a system in case that a CSMS uses OpenADR for EVSE energy management, like using demand response from VTN for EVSE energy management.

3.3.1.1 Process: Attempt to intercept unencrypted data

This test checks if there are unencrypted data in the communication between the virtual top node (VTN) and the virtual end node (VEN). To perform the test, set up a VTN by installing SW such as OpenLEADR from LF Energy Foundation (<https://openleadr.org>).

Procedure:

1. Run an OpenADR server (VTN). OpenLEADR 3.0 and 3.1 implement OpenADR protocol in Rust To launch a VTN, run the following command:

```
>> docker compose up -d db
>> cargo sqlx migrate run
>> RUST_LOG=trace cargo run -bin openleadr-vtn
>> docker compose up -d
```

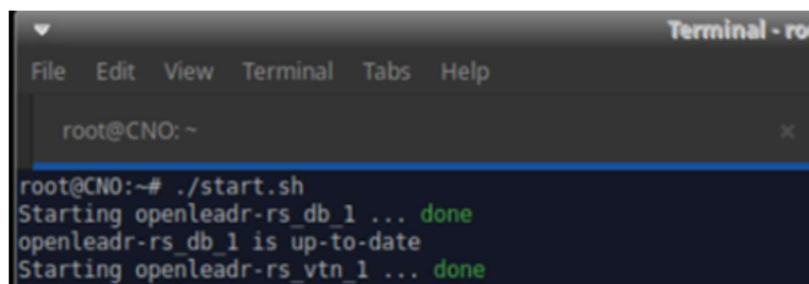


Figure 3. A screenshot of a VTN virtual machine.

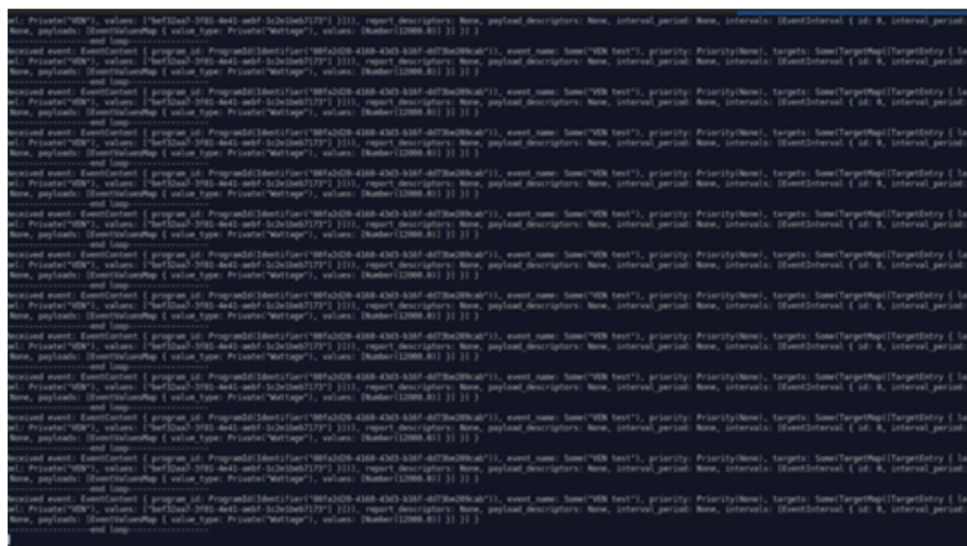


Figure 4. A screenshot of a VTN virtual machine.

2. Install WireShark or other packet capturing utilities and start capturing traffic using the packet capturing utility.

```
>> sudo apt update
>> sudo apt install wireshark
>> wireshark
```

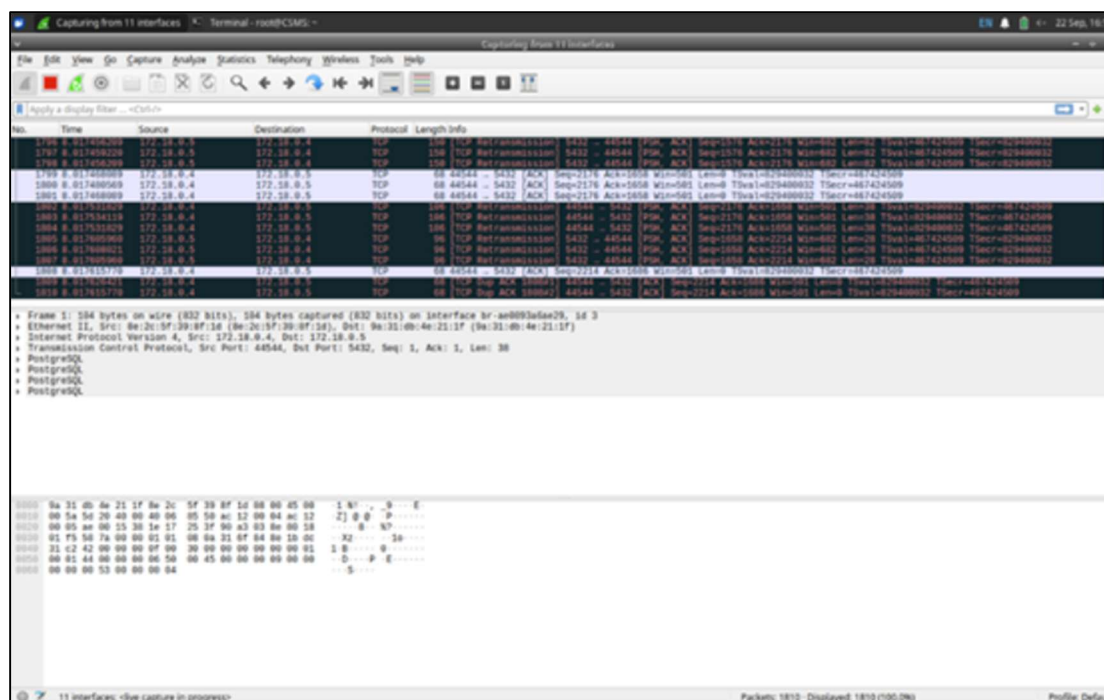
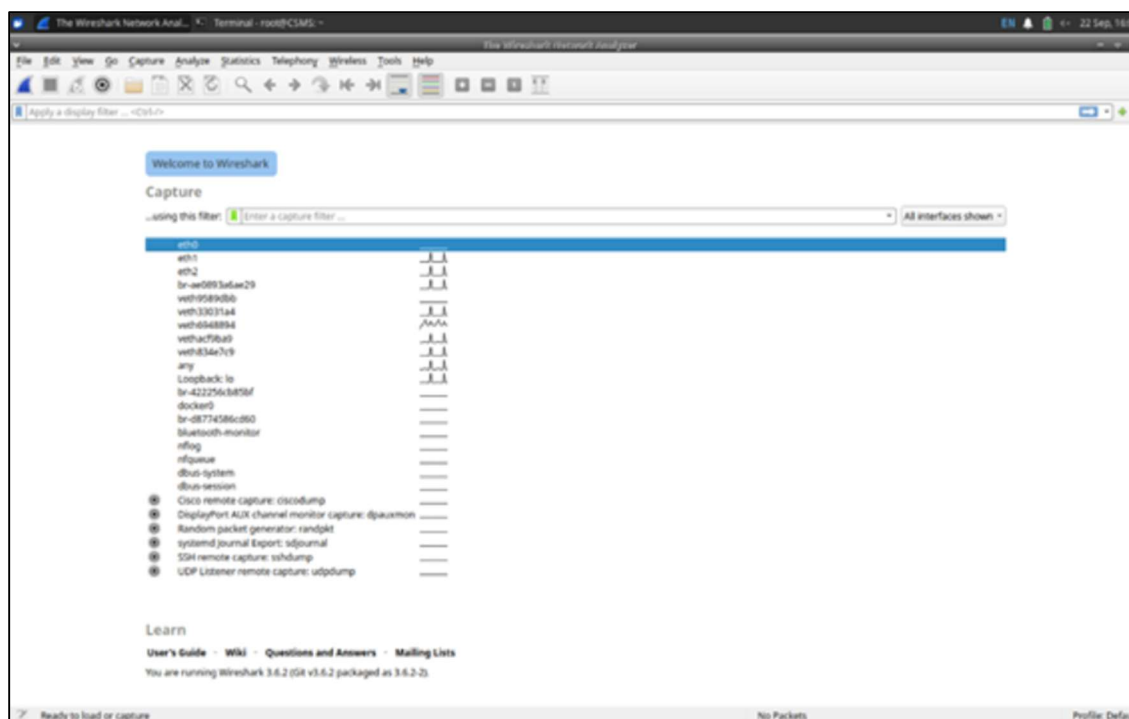


Figure 5. Packet capturing using WireShark.

3. Attempt to search for OpenADR packets and view inside the packets. Figure 6 shows packet capturing of unencrypted OpenADR messages.

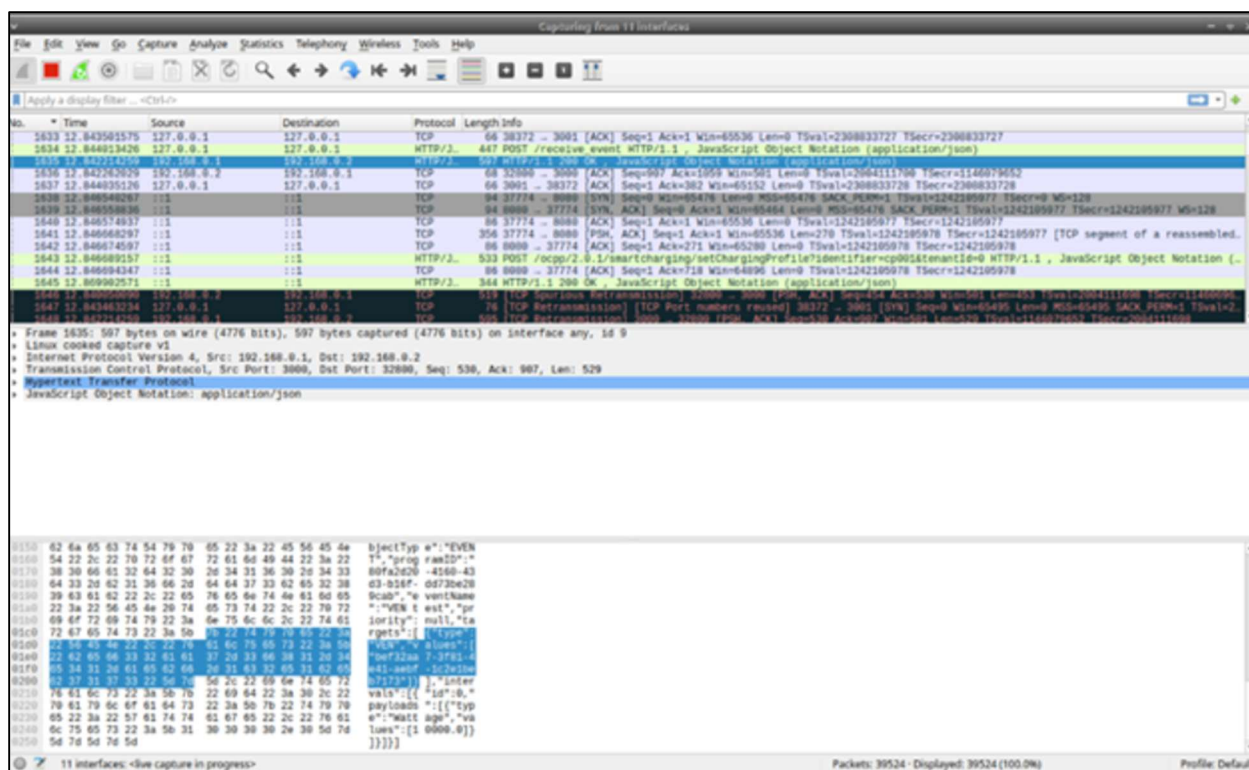


Figure 6. OpenADR packet capturing with Wireshark.

The information within the packets should not be visible without the corresponding key(s) for decryption because OpenADR communications is encrypted. If unencrypted data is captured, it indicates OpenADR is not implemented properly.

Scoring Criteria:

- 2 points: No unencrypted OpenADR traffic captured
- 0 points: Unencrypted OpenADR traffic captured
 - Score: _____

3.3.2 Open Charge Point Protocol (OCPP)

3.3.2.1 Process: Traffic Capture and Analysis

This test tries to capture OCPP traffic and analyze the data. To conduct the test, set up an OCPP server and a client by installing CitrineOS (<https://citrineos.github.io/latest/>) or MaEVe (<https://github.com/thoughtworks/maeve-csms>) for an OCPP server, and Everest (<https://github.com/EVERest>) for an OCPP client.

Procedure:

1. Run an OCPP server and a client.

```

Terminal - root@CSMS: ~ - ssh@root@csms-server
root@CSMS: ~
x root@CSMS: ~ - ssh@root@csms-server
x root@CSMS: ~ - ssh@root@csms-server
x root@CSMS: ~ - ssh@root@csms-server

Citrine-1 }
Citrine-1 2025-09-22 16:28:28.132 DEBUG /usr/local/apps/citrineos/03_Modules/Configuration/dist/module/module.js:267 CitrineOS Logger:ConfigurationModule Heartbeat response sent: {
Citrine-1 success: true
Citrine-1 }
Citrine-1 2025-09-22 16:28:28.133 DEBUG /usr/local/apps/citrineos/02_Util/dist/queue/rabbit-mq/receiver.js:193 CitrineOS Logger:RabbitMQReceiver oMessage:Received message: {
Citrine-1 contentType: 'application/json',
Citrine-1 contentEncoding: 'utf-8',
Citrine-1 headers: {
Citrine-1   action: 'Heartbeat',
Citrine-1   correlationId: 'c28d35f7-3087-4842-bc2d-0672fe662a3d',
Citrine-1   eventGroup: 'general',
Citrine-1   origin: 'csms',
Citrine-1   state: '2',
Citrine-1   stationId: 'cp001',
Citrine-1   tenantId: '',
Citrine-1   timestamp: '2025-09-22T16:28:28.128Z'
Citrine-1 },
Citrine-1 deliveryMode: undefined,
Citrine-1 priority: undefined,
Citrine-1 correlationId: undefined,
Citrine-1 replyTo: undefined,
Citrine-1 expiration: undefined,
Citrine-1 messageId: undefined,
Citrine-1 timestamp: undefined,
Citrine-1 type: undefined,
Citrine-1 userId: undefined,
Citrine-1 appId: undefined,
Citrine-1 clusterId: undefined
Citrine-1 } [{"origin":"csms","eventGroup":"general","action":"Heartbeat","state":"2","context":{"stationId":"cp001","correlationId":"c28d35f7-3087-4842-bc2d-0672fe662a3d","tenantId":"","time
stamp":"2025-09-22T16:28.128Z"},"payload":{"currentTime":"2025-09-22T16:28.132Z"},"protocol":"ocpp.0.1"}]
Citrine-1 2025-09-22 16:28:28.134 DEBUG /usr/local/apps/citrineos/08_Bsm/dist/interfaces/router/AbstractRouter.js:80 CitrineOS Logger:MessageRouterImpl Received message: Message {
Citrine-1   origin: 'csms',
Citrine-1   eventGroup: 'general',
Citrine-1   action: 'Heartbeat',
Citrine-1   state: '2',
Citrine-1   context: {
Citrine-1     stationId: 'cp001',
Citrine-1     correlationId: 'c28d35f7-3087-4842-bc2d-0672fe662a3d',
Citrine-1     tenantId: '',
Citrine-1     timestamp: '2025-09-22T16:28:28.128Z'
Citrine-1   },
Citrine-1   payload: {
Citrine-1     currentTime: '2025-09-22T16:28:28.132Z'
Citrine-1   },
Citrine-1   protocol: 'ocpp.0.1'
Citrine-1 }
Citrine-1 }
Citrine-1 ["detail":{"http_info":{"content_encoding":null,"http_version":"HTTP/1.1","ip":"172.18.0.4","method":"GET","status":200,"url":"/healthz"},"operation":{"request_id":"69ba2687-7c1a-486b-9a16-5ead2e18b15b","request_mode":"non-graphql","response_size":2,"uncompressed_response_size":2},"request_id":"69ba2687-7c1a-486b-9a16-5ead2e18b15b","level":"info","timestamp":"2025-09-22T16:28:27.743+0000","type":"http-log"}]

```

Figure 7. An OCPP server using CitrineOS.

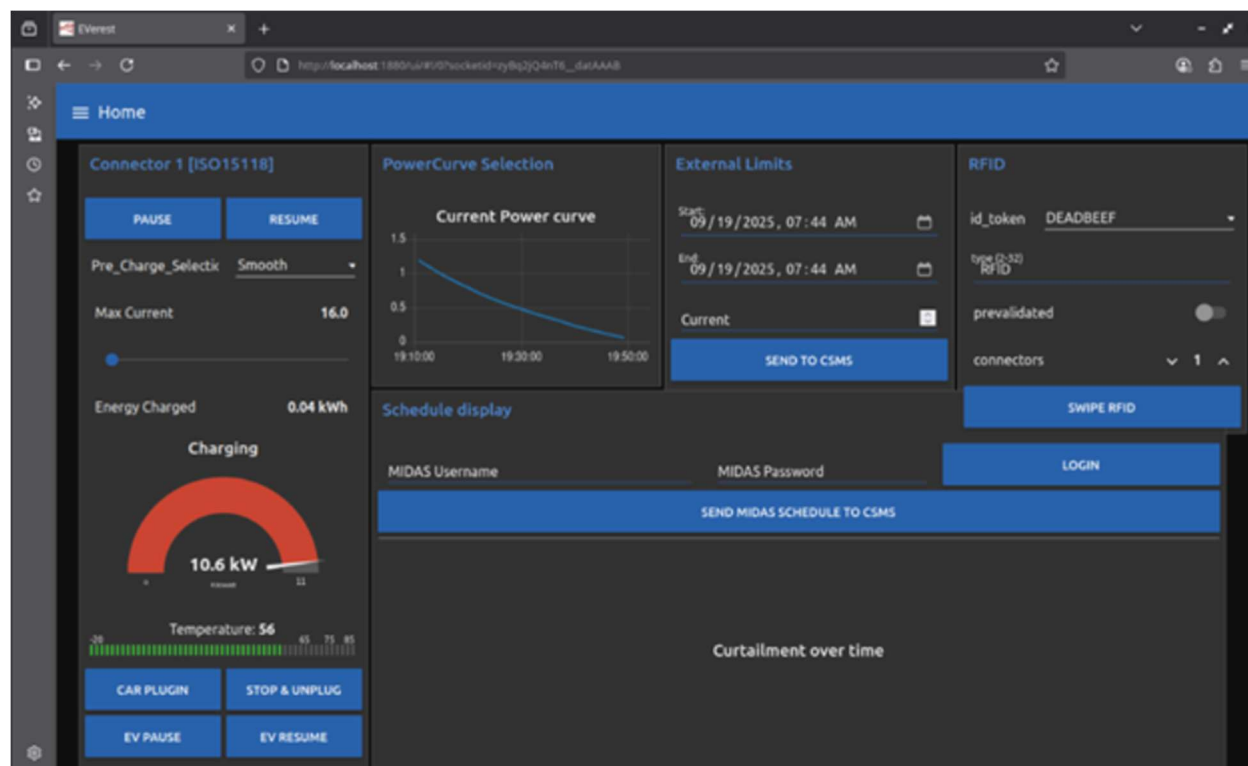


Figure 8. User interface of Everest OCPP client.

2. Start capturing traffic using WireShark.
3. Attempt to search for OCPP messages and view inside packets. Figure 9 illustrates a screenshot of unencrypted OCPP packet capturing by WireShark.

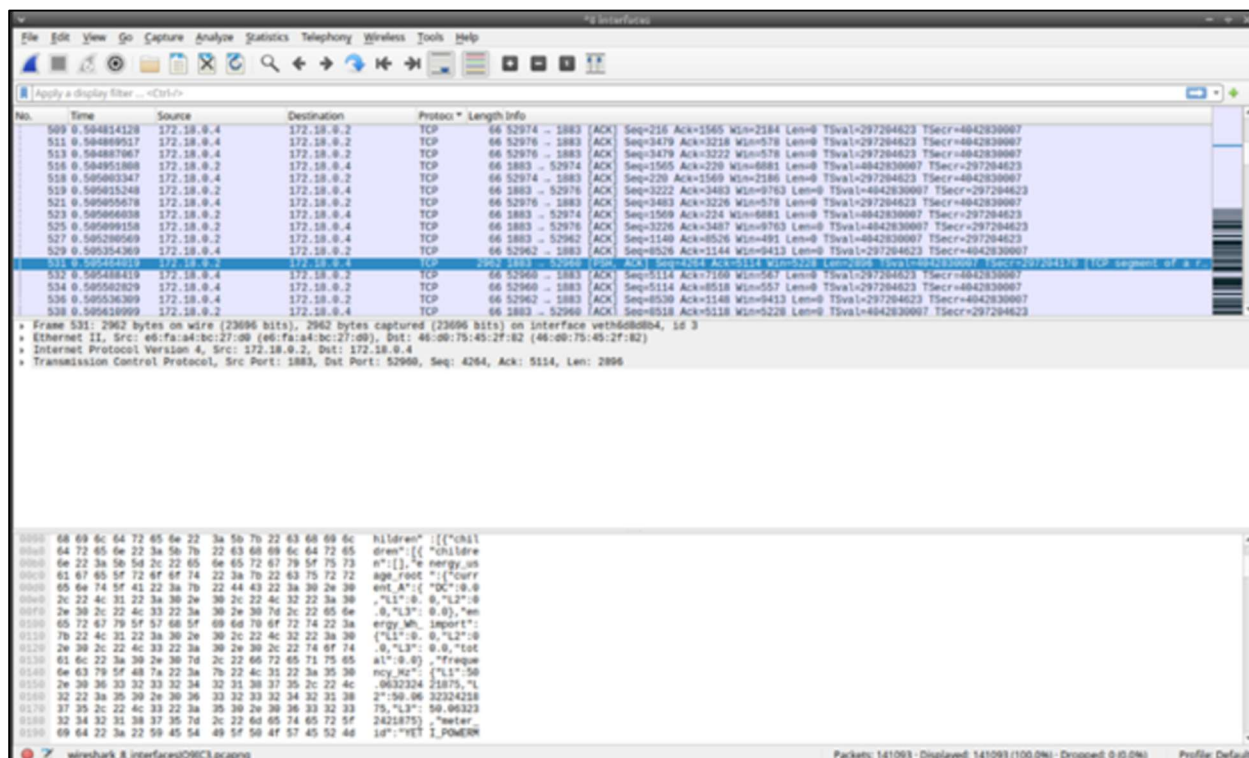


Figure 9. A screenshot of OCPP packet captures with WireShark.

If OCPP communications is encrypted, this prevents an unknown user from issuing start/stop orders for a charge session, power curtailment, or other commands disruptive to the system.

Scoring Criteria:

- 2 points: No unencrypted OCPP traffic captured
- 0 points: Unencrypted OCPP traffic captured
 - Score: _____

3.3.2.2 Process: Protocol Version Assessment for OCPP

Query the charge point's configuration to identify the protocol version and security settings. Determine the OCPP version in use and the implemented security profile. OCPP Versions 2.0.1 and above support enhanced security profiles, which includes TLS encryption and mutual authentication, which are critical for secure communications for energy management purposes.

First, set up an OCPP server and a client by installing CitrineOS (<https://citrineos.github.io/latest/>) or MaEVe (<https://github.com/thoughtworks/maeve-csms>) for an OCPP server, and Everest (<https://github.com/EVERest>) for an OCPP client.

Procedure:

1. Run an OCPP server and a client.
2. Start capturing traffic using WireShark.
3. Attempt to search for OCPP messages and view inside packets. In Figure 10, unencrypted OCPP packets captured by WireShark shows the version of OCPP implemented.

This test example uses OCPP 2.0.1. OCPP 2.0.1 or above mandates TLS communication. If messages are unencrypted, this indicates that OCPP 2.0.1 is not implemented properly.

Scoring Criteria:

- 2 points: No unencrypted OCPP traffic captured and OCPP 2.0.1 or above is implemented
- 1 points: No unencrypted OCPP traffic captured and OCPP 1.6 is implemented
- 0 points: Unencrypted OCPP traffic captured
 - Score: _____

3.3.2.3 Process: Security Profile Verification for OCPP

The primary objective is to ensure that appropriate encryption and authentication mechanisms, as defined in protocol security profiles, are correctly deployed.

This involves checking for the use of Transport Layer Security (TLS) to secure communication channels and verifying the presence and validity of digital certificates on both the server and the client. These certificates should be issued by a trusted certificate authority and configured correctly to support mutual authentication, which prevents unauthorized entities from participating in communication.

Properly implementing these security features not only encrypts sensitive data in transit, such as user credentials and charging session details, but also helps ensure the integrity and authenticity of the entities involved, significantly reducing the risk of data interception, spoofing, or unauthorized access.

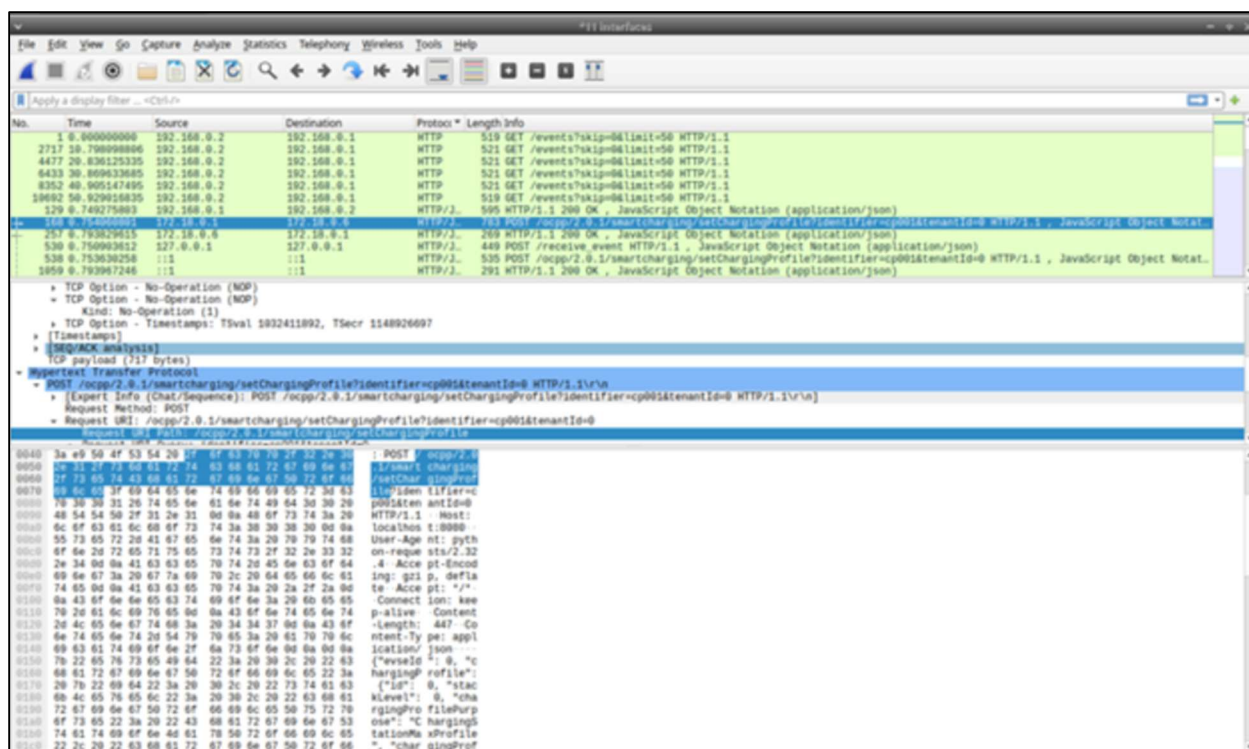


Figure 10. A screenshot of OCPP packet captures showing OCPP version.

If OCPP security profile using Transport Layer Security (TLS) is implemented, this requires TLS encryption and mutual authentication.

Scoring Criteria:

- 2 points: No unencrypted OCPP traffic captured and OCPP 2.0.1 or above is implemented
 - 1 points: No unencrypted OCPP traffic captured and OCPP 1.6 is implemented
 - 0 points: Unencrypted OCPP traffic captured
- Score: _____

3.3.3 Messaging Queuing Telemetry Transport (MQTT)

3.3.3.1 Process: Traffic Capture and Analysis for MQTT

This test tries to capture OCPP traffic and analyze the data. To conduct the test, set up a MQTT broker and a MQTT like Eclipse Mosquitto (<https://mosquitto.org/>).

Procedure:

1. Run a MQTT broker or use a publically available MQTT server.

```
>> mosquitto
```

2. Publish an unencrypted MQTT message from a client.

```
>> mosquitto_pub -h <mqtt_broker_ip> -t <mqtt_message_topic> -m  
<mqtt_message_data>
```

3. Start capturing traffic using WireShark or try to subscribe all MQTT messages if the IP address of MQTT broker is known.

```
>> mosquitto_sub -h <mqtt_broker_ip> -t # -m <mqtt_message_data>
```

4. Attempt to search for MQTT messages and view inside packets. Figure 10 shows packet capturing of unencrypted MQTT messages.

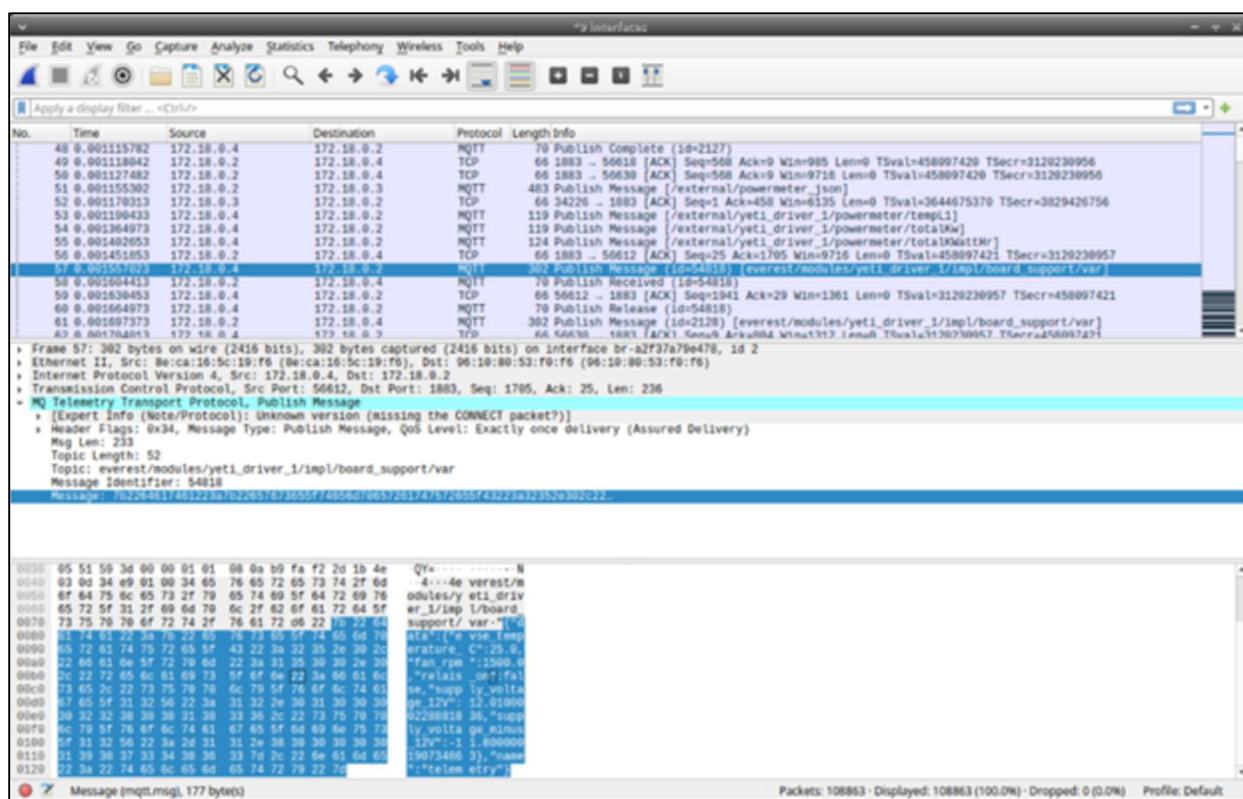


Figure 11. A screenshot of MQTT packet captures with WireShark.

If MQTT messages are encrypted with username/password or a certificate, WireShark should not see the contents of the captured MQTT messages or through MQTT message subscription without username/password or a certificate.

Scoring Criteria:

- 2 points: No unencrypted MQTT traffic captured

- 0 points: Unencrypted MQTT traffic captured
 - Score: _____

3.4 Cybersecurity Analysis and Scoring

From the scoring results determined in the previous section, enter the values into the “Raw Score” column in the following table. Then, using the weighting factor values, calculate the “Final Score” for each analysis criteria. Then calculate the sum “Total Score” for this section of assessment.

Table 6. EVSE Energy Management Assessment Scoring Table

EVSE Energy Management Assessment Scoring Table	Raw Score (0,1,2)	Weight Factor	Final Score
3.2.1			
<u>OpenADR</u>			
Unencrypted OpenADR data		1	
3.2.2			
<u>OCPP</u>			
OCPP Traffic Capture and Analysis		1	
Protocol Version Assessment for OCPP		2	
Security Profile Verification for OCPP		2	
3.2.3			
<u>MQTT</u>			
Traffic Capture and Analysis for MQTT		1	
Total Score	-		

3.4.1 Interpretation of Scores

- **12- or more points:** Excellent implementation of security measures.
- **10-11 points:** Good implementation, with minor areas for improvement.
- **8-9 points:** Moderate implementation, requiring significant improvements.

- **4-7 points:** Poor implementation, with critical areas needing attention.
- **0-3 points:** Very poor implementation, indicating severe security risks.

This scoring method allows for a systematic evaluation of the hardware security level based on the defined metrics and measurements.

4 Summary and Next Steps

This document is intended to assist technical staff for the purpose of conducting repeatable cybersecurity assessments and evaluation of electric vehicle critical infrastructure (EVCI) in order to identify gaps and vulnerabilities in hardware, software, and communications system. The results highlight and prioritize areas of concern for the EVCI requiring further focus and correction in order to improve the cybersecurity posture of the EVCI system.

These procedures are not intended to be static but rather updated and refined as the threat landscape evolves as threat actors advanced the strategies and tactics utilized against this electrified infrastructure sector.

The authors of CyberMESA envision this content, procedures, metrics, information, and appendixes to be migrated to an interactive tool (online or offline) that can be used interactively by an end user to effectively evaluate a component, sub-component, or the entire EVCI unit under test. If CyberMESA is successfully migrated into an interactive tool, it can be updated by the tool owner, as appropriate, as the threat landscape changes and evolves over time.

References

OWASP Vulnerability Management Guide (OVMG)

<https://owasp.org/www-project-vulnerability-management-guide/OWASP-Vuln-Mgm-Guide-Jul23-2020.pdf>

National Motor Freight Traffic Association, Inc. (NMFTA), Request for Proposal Contract Template

https://github.com/nmfta-repo/nmfta-rfp_templates

Van Woudenberg, J. and O'Flynn, C., 2021. The hardware hacking handbook: breaking embedded security with hardware attacks. No Starch Press.

https://www.google.com/books/edition/_/cBdIEAAQBAJ?hl=en&gbpv=1&pg=PR19&dq=hardware+hacking

Grand, J., Mitnick, K.D. and Russell, R., 2004. Hardware hacking: have fun while voiding your warranty. Elsevier.

https://www.google.com/books/edition/Hardware_Hacking/bGEbLt1C168C?hl=en&gbpv=1&dq=hardware+hacking&pg=PR2&printsec=frontcover

Huang, A.B., 2019. The hardware hacker: Adventures in making and breaking hardware. No Starch Press.

https://www.google.com/books/edition/The_Hardware_Hacker/qAYvDwAAQBAJ?hl=en&gbpv=1&dq=hardware+hacking&pg=PR17&printsec=frontcover

O'Flynn, C. and Van Woudenberg, J., 2021. The Hardware Hacking Handbook. No Starch Press.

Hamdioui, S., Danger, J.L., Di Natale, G., Smailbegovic, F., van Battum, G. and Tehranipoor, M., 2014, March. Hacking and protecting IC hardware. In 2014 Design, Automation & Test in Europe Conference & Exhibition (DATE) (pp. 1-7). IEEE.

<https://ieeexplore.ieee.org/abstract/document/6800313>

Grand, J. and Friday, J., 2004. Advanced hardware hacking techniques. DEFCON, 12, p.59.

https://grandideastudio.com/media/advanced_hardware_hacking_slides.pdf

National Institute of Standards and Technology, US Department of Commerce, NIST 800-53

Revision 5, <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-53r5.pdf>

Underwriters Laboratories, Cybersecurity Assurance Program (CAP), UL 2900 Series of Standards,

<https://www.ul.com/services/cybersecurity-assurance-program-cap>,

<https://www.shopulstandards.com/ProductDetail.aspx?productId=UL2900-1>

<https://github.com/IdahoLabResearch/AcCCS>

Appendix A - List of Hardware and Software Tools

Performing a cybersecurity assessment of most hardware devices requires several common tools. The following table is a list of tools to acquire as part of the assessment activity.

Table A1: Cybersecurity Assessment Tools

Tool	Description
Multimeter	Used for measuring voltage and may also be used for identifying communications circuits (waveforms).
Oscilloscope	Used for monitoring communications wires and other electrical signals. Also used to verify communication line peak voltages.
Logic Analyzer	Used for decoding and interpreting communications protocols.
Serial Interfaces	A collection of USB interfaces for TTL (basic UART), RS-232, and RS-486.
Serial Tap	An inline connection device to capture point-to-point serial communications, like RS-232.
JTAG and Serial Wire Debug (SWD) Interface	Segger J-Link or similar that supports the target processor architecture.
JTAGulator or Similar	Enumerates JTAG pins to identify correct signals (TDI, TDO, TCK, TMS, etc.).
USB Tools	Bash Bunny or Rubber Ducky types of devices capable of disguising themselves as benign USB devices while executing code on the target system.
Soldering Iron	Some circuit boards have unpopulated headers for serial or JTAG and require pins be soldered in place.
Wireless Interfaces	A WiFi interface that supports monitor mode for sniffing. Bluetooth that supports BLE and classic capture. Zigbee (802.15.4) analysis interface (e.g. TI CC2531 Evaluation Kit).
Bluetooth Interface	A Bluetooth interface that supports sniffing BLE and classic communications.

Wireless Spectrum Analyzer (Optional and expensive)	Monitors all RF signals and graphically displays what signals are strongest in a given location.
CAN Interface	USB CAN interface for monitoring CAN bus communications.
Ethernet Network Tap	A dedicated tap device, hub, or a managed switch that supports port mirroring.
Firmware Access or Flashing	Segger J-Link, Open OCD, and others that support the target processor architecture.
Network Capture	Wireshark, tshark , or tcpdump capable of monitoring wired or wireless networking.
Reverse Engineering Tools	IDA Pro, Binary Ninja, Ghidra, and hex editors are used to inspect binary files and firmware.

Appendix B - EVSE Hardware Overview

Table B1: EVSE Common Components

Component Name	Description
Human Machine Interface (HMI)	LCD panels or LEDs that are used to interact with the user and communicate status.
Authorization Interface	Payment or other authorization devices used to authenticate and start a charge session (e.g. RFID, NFC, Credit Card scanner, etc.).
Remote Communications Board	Vendor specific system board used for external communications and charge management.
Cellular Modem	Used to provide a dedicated network connection back to the vendor and or a remote management company (Charge Point Operator).
Local Management Interface	Connectivity for managing the EVSE locally at a charge site. Methods include wired Ethernet, Wi-Fi, or dedicated serial. Protocols might be proprietary or standardized (e.g. OCPP).
Charge Controller	Vendor specific or supplier provided UL Listed board for safe control of the charging process.

	This includes ground fault detection and vehicle communications.
Vehicle Controllers	Specific functionality for managing CCS or NACS communications with the vehicle. This might be a separate board in the EVSE or combined with the Charge Controller
Power Electronics	Electronics for power conversion between AC and DC power as well as the appropriate voltage required by the vehicle. Used in DC charging and bi-directional power transfer.

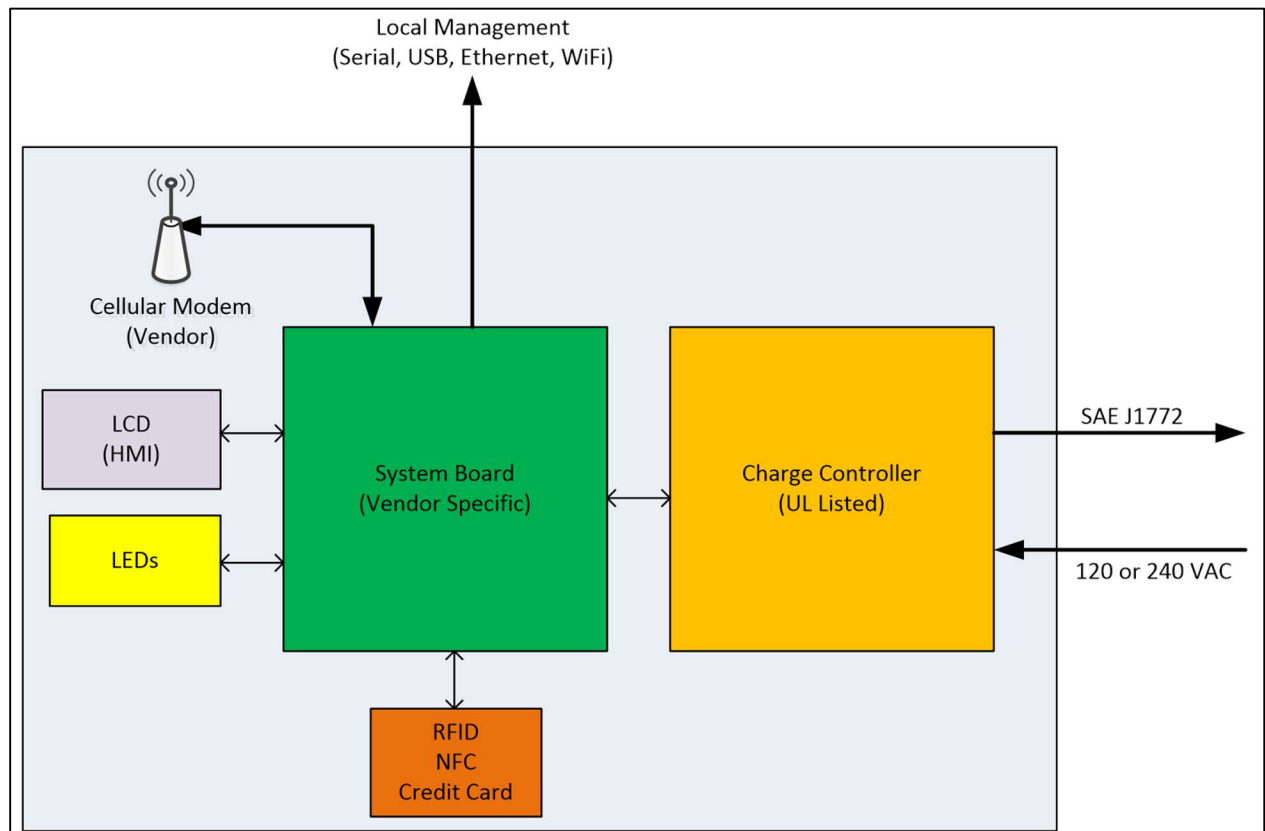


Figure B1: AC Level 1 or Level 2 EVSE Block Diagram

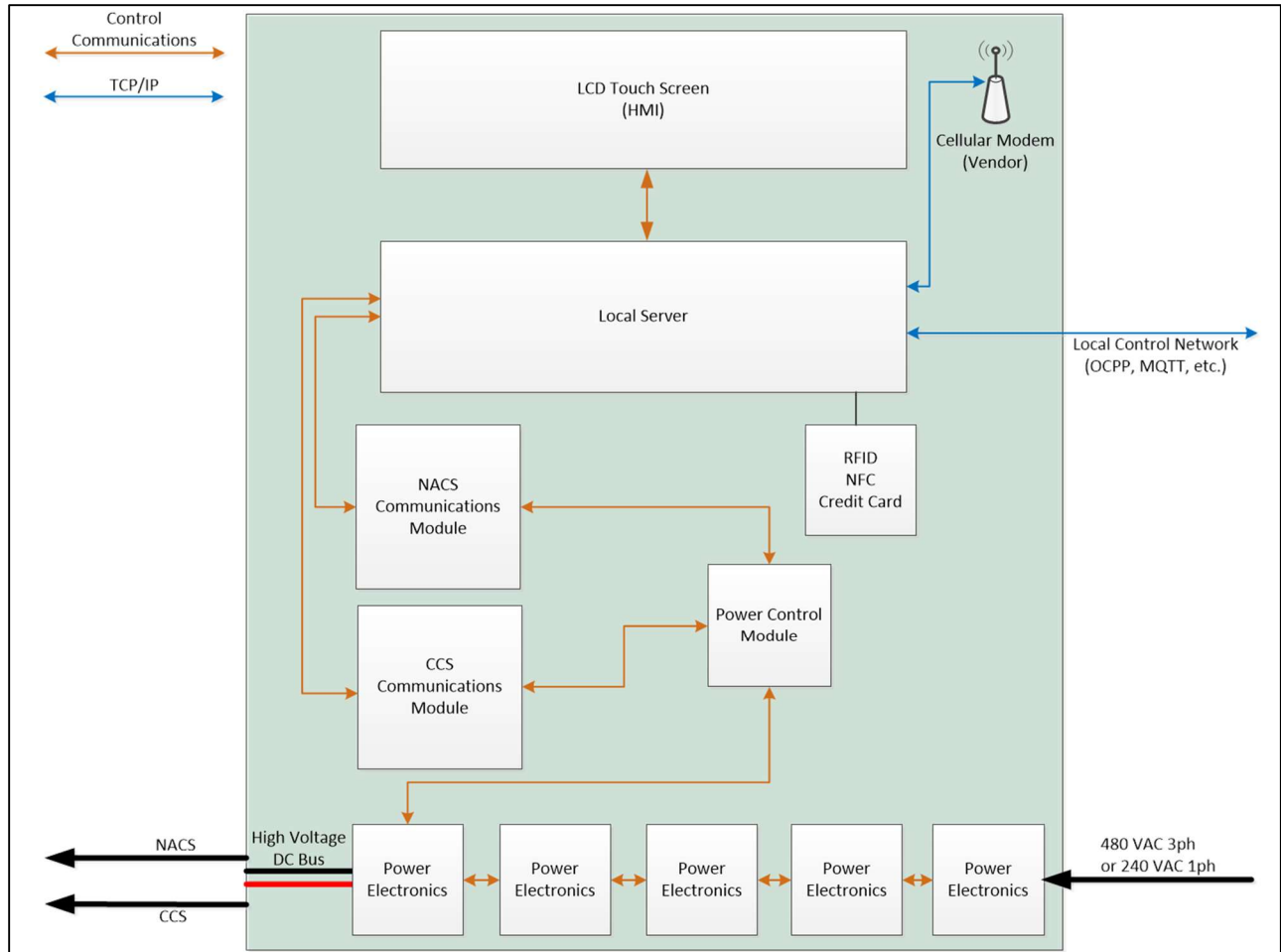


Figure B2: DC EVSE Block Diagram

Appendix C - Common Embedded Systems Hardware Components

Table C1: Common Circuit Board Communications Ports

Communication Interface	Description
Basic UART	A basic TTL serial interface that operates at the board voltage (3.3 or 5 vdc). This is typically used as console output or a debug interface. Use an appropriate FTDI basic interface to connect and interrogate this interface.
Serial RS-232 or RS-485	Serial communications interfaces typically used for remote communications. These connections operate at voltages higher than what is found on the circuit board. Use an appropriate USB serial

	device to connect and interrogate this interface. A common protocol found on a serial interface is Modbus. Note: RS-485 can be half-duplex (3 wires) or full-duplex (5 wires).
I2C or SPI Interfaces	Communications protocols for board-to-board or board-to-peripheral device serial messages. These communications lines can be monitored and decoded by using a Logic Analyzer such as Saleae Logic.
JTAG or Serial Wire Debug (SWD) Headers	A debug interface port or header for directly interacting with a processor or microcontroller on a circuit board. These interfaces can provide typical debug functionality (pause, resume, read/write memory) as well as boundary testing of the controller and PCB.
SD/MicroSD	Some systems will boot from external flash memory installed in SD ports.
USB Port	Determine if the port is used as a USB Host port (able to connect peripherals) or if it provides serial debug access.
Controller Area Network (CAN)	A serial broadcast bus for communications between control units. Use a USB-to-CAN interface to connect to and monitor the bus. Typical baud rates are 125K, 250K and 500K.
Ethernet Networking	Passively monitor the network communications after installing a network tap, hub, or small switch configured to mirror or SPAN traffic out to a single port. Ensure the laptop used for monitoring is configured with a silent interface in promiscuous mode. Use tools such as Wireshark or <code>tcpdump</code> to capture and decode the traffic.

Appendix D - Firmware Analysis Resources

D.1 Software Bill of Material (SBOM)

A Software Bill of Material (SBOM) is not usually provided by an equipment manufacturer, but online services exist for generating a SBOMs from extracted firmware files. Care must be taken that submission of extracted firmware to any of the online SBOM providers is done with the express permission from the equipment manufacturer.

Commercial products, like Dependency Track²⁷, and open-source tools, like Intel's CVE Binary Tool²⁸ can examine firmware and identify code within the firmware that is responsible for known vulnerabilities or CVEs. This automated analysis can help to generate a basic SBOM when one is not provided by the manufacture.

D.2 Filesystem Examination

If the assessment team is successful at extracting the contents of flash memory from the test device, there are several tools available to aid in the examination of the device image.

Binwalk is a Linux tool capable of looking through a binary file (extracted flash image) and attempting to identify different segments of the file. The following is an example of what **binwalk** can identify in a basic flash image from an ARM based device. This image file appears to contain a **uBoot** image as well as a Linux **boot** partition and **rootfs** partition.

```
$ binwalk extracted_flash.img
```

DECIMAL	HEXADECIMAL	DESCRIPTION
202340	0x31664	CRC32 polynomial table, little endian
393216	0x60000	uImage header, header size: 64 bytes, header CRC: 0x7365741A, created: 2020-08-24 21:42:23, image size: 474392 bytes, Data Address: 0x80800000, Entry Point: 0x0, data CRC: 0x8A7942DA, OS: Firmware, CPU: ARM, image type: Firmware Image, compression type: none, image name: "U-Boot 2019.04-00002-gc9b3922522"
398544	0x614D0	CRC32 polynomial table, little endian
641920	0x9CB80	SHA256 hash constants, little endian
694195	0xA97B3	Android bootimg, kernel size: 1684947200 bytes, kernel addr: 0x64696F72, ramdisk size: 1763734311 bytes, ramdisk addr: 0x6567616D, product name: "ddr 0x%08x size %u KiB"
709098	0xAD1EA	Unix path: /home/userid/targetNFS
756912	0xB8CB0	LZO compressed data
4194304	0x400000	Linux EXT filesystem, blocks count: 7790720, image size: 7977697280, rev 1.0, ext4 filesystem data, UUID=4f3e2f9a-57bf-460a-b9a9-ee8bb1ddb1dd, volume name "rootfs"

²⁷ Dependency Track - <https://dependencytrack.org/>

²⁸ Intel CVE Binary Tool - <https://github.com/intel/cve-bin-tool>

If the flash memory image appears to contain Linux filesystems, use `losetup` to automatically mount the firmware partitions for further examination. The following is an example of what this might look like.

```
$ losetup -fPr extracted_flash.img
$ losetup
```

NAME	SIZELIMIT	OFFSET	AUTOCLEAR	RO	BACK-FILE	DIO	LOG-SEC
...							
/dev/loop29	0	0	0	1	/tmp/extracted_flash.img	0	512

```
$ mount
/dev/loop29p1 on /media/root/rootfs type ext4
(ro,nosuid,nodev,relatime,errors=remount-ro,uhelper=udisks2)
```

In the above example, `losetup` was able to successfully mount the root filesystem found in the extracted flash image file. This filesystem is now mounted on the assessment laptop in `/media/root/rootfs`.

D.3 Static Analysis Tools

If `binwalk` is unable to identify meaningful segments within an extracted flash memory image, the microcontroller might just load the binary code directly from flash and begin execution (bare metal code execution of firmware). This is often the case with smaller embedded devices that do not require a full Linux operating system.

An extracted binary blob from flash memory can be loaded into reverse engineering software like IDA Pro²⁹ or Ghidra³⁰ to decompile the binary as executable code. There are several good guides publicly available that demonstrate this process, but in general, IDA or Ghidra need to be configured with the proper memory address layout of the target processor (ARM, PowerPC, etc.) as the binary file is opened. This will properly align the firmware segments with what is found in the microcontroller documentation so that the reverse engineer can locate the start segments and other important tables.

²⁹ IDA Pro - <https://hex-rays.com/ida-pro>

³⁰ Ghidra - <https://github.com/NationalSecurityAgency/ghidra>

Appendix E - Software Scanning Tools and Information

E.1 nmap

Nmap³¹ is a network scanning tool that identifies active devices, open ports, and running services on a target system.

E.2 sslscan

sslscan³² is a tool used to enumerate the SSL/TLS protocols and cipher suites supported by a server, aiding in the identification of potential misconfigurations or weaknesses in cryptographic configurations. Within the scope of this report, *sslscan* is employed to assess the TLS configurations of ISO 15118-2 and ISO 15118-20 servers, Web UI management, MQTT brokers, and other TLS-based services hosted on the charger.

The following instructions were written using *sslscan* version 2.2.0. To configure the TLS parameters used by *sslscan*, create a file named `openssl.cnf` with the following contents:

```
openssl_conf = openssl_init
[openssl_init]
    ssl_conf = ssl_module
[ssl_module]
    system_default = tls_system_default

[tls_system_default]
    Groups = P-224:X25519:P-256:Ed448:P-521:P-384
```

Then set the path to the file so that tools such as *sslscan* can reference it:

```
export OPENSSL_CONF=/path/to/your/openssl.cnf
```

It is important to specify the full path to the configuration file so that *sslscan* and other tools can locate it reliably.

To run *sslscan*, use the following command:

```
sslscan -iana-names '[fe80::3ca7:32c1:5ff1:c191%2]:64534'
```

³¹ <https://nmap.org/>

³² <https://github.com/rbsec/sslscan>

Replace the address and port with one returned by `sdpquery`. An example output from this command is shown in Figure E1.

When reviewing the output from `ssllscan`, it is important to evaluate the following sections, each of which provides insight into the cryptographic configuration and security posture of the server:

- **SSL/TLS Protocols:** This section lists which SSL/TLS protocol versions are supported by the server. It indicates whether each protocol is enabled or disabled. For example, in the figure, TLS 1.2 and TLS 1.3 are enabled, which are the currently recommended versions. Older versions (e.g., SSL 2.0, SSL 3.0, TLS 1.0, TLS 1.1) should be disabled, as they are considered insecure.
- **TLS Negotiation:** This section shows whether the server supports TLS session renegotiation. For the purposes of this report, renegotiation should be disabled in all cases, as it introduces security risks—including vulnerability to certain types of man-in-the-middle (MitM) attacks.
- **TLS Compression:** Indicates whether the server supports TLS-level compression. TLS compression should be disabled, as it can be exploited in attacks such as CRIME, which allow recovery of plaintext data from encrypted traffic.
- **Supported Server Cipher(s):** This section enumerates all the cipher suites supported by the server. Cipher suites define the algorithms used for encryption, key exchange, and message authentication. Strong, modern ciphers should be prioritized (e.g., AES-GCM with forward secrecy), while weak or deprecated ciphers (e.g., RC4, 3DES) should be avoided.
- **Server Key Exchange Group(s):** Lists the elliptic curves or finite field groups used by the server for key exchange in TLS connections. These groups determine the cryptographic strength of the key exchange mechanism. Secure and modern groups (e.g., X25519, secp256r1) should be used. Legacy or weak groups should be excluded from the server's configuration.
- **SSL Certificate:** This section provides information about the X.509 certificate presented by the server during the TLS handshake. It includes key details such as the certificate's subject, issuer, validity period, public key algorithm, and key size. When reviewing this section, ensure that the certificate is issued by a trusted Certificate Authority (CA), is not expired, and uses a strong public key (e.g., RSA 2048-bit or higher, or ECDSA with a secure curve such as secp256r1). The certificate should also match the expected domain or IP address. Self-signed, expired, or improperly issued certificates may indicate misconfiguration or a weakened trust model in the EVSE implementation.

Other information presented in the scan output may be relevant to a security assessor but is not referenced or evaluated in this report.

```

tomc@we36596:~/Downloads/CyberMESA/sslscan$ ./sslscan --iana-names '[fe80::3ca7:32c1:5ff1:c191%2]:64534'
Version: 2.2.0-1-gd8c0b92-static
OpenSSL 3.5.1 1 Jul 2025

Connected to fe80::3ca7:32c1:5ff1:c191

Testing SSL server fe80::3ca7:32c1:5ff1:c191%2 on port 64534 using SNI name fe80::3ca7:32c1:5ff1:c191%2

SSL/TLS Protocols:
SSLv2      disabled
SSLv3      disabled
TLSv1.0    disabled
TLSv1.1    disabled
TLSv1.2    enabled
TLSv1.3    enabled

TLS Fallback SCV:
Server supports TLS Fallback SCV

TLS renegotiation:
Session renegotiation not supported

TLS Compression:
Compression disabled

Heartbleed:
TLSv1.3 not vulnerable to heartbleed
TLSv1.2 not vulnerable to heartbleed

Supported Server Cipher(s):
Preferred TLSv1.3 128 bits TLS_AES_128_GCM_SHA256
Accepted TLSv1.3 256 bits TLS_AES_256_GCM_SHA384
Accepted TLSv1.3 256 bits TLS_CHACHA20_POLY1305_SHA256
Preferred TLSv1.2 256 bits TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384 Curve 25519 DHE 253
Accepted TLSv1.2 256 bits TLS_ECDHE_ECDSA_WITH_CHACHA20_POLY1305_SHA256 Curve 25519 DHE 253
Accepted TLSv1.2 256 bits TLS_ECDHE_ECDSA_WITH_AES_256_GCM Curve 25519 DHE 253
Accepted TLSv1.2 256 bits TLS_ECDHE_ECDSA_WITH_ARIA_256_GCM_SHA384 Curve 25519 DHE 253
Accepted TLSv1.2 128 bits TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256 Curve 25519 DHE 253
Accepted TLSv1.2 128 bits TLS_ECDHE_ECDSA_WITH_AES_128_CCM Curve 25519 DHE 253
Accepted TLSv1.2 128 bits TLS_ECDHE_ECDSA_WITH_ARIA_128_GCM_SHA256 Curve 25519 DHE 253
Accepted TLSv1.2 256 bits TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA384 Curve 25519 DHE 253
Accepted TLSv1.2 256 bits TLS_ECDHE_ECDSA_WITH_CAMELLIA_256_CBC_SHA384 Curve 25519 DHE 253
Accepted TLSv1.2 128 bits TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA256 Curve 25519 DHE 253
Accepted TLSv1.2 128 bits TLS_ECDHE_ECDSA_WITH_CAMELLIA_128_CBC_SHA256 Curve 25519 DHE 253

Server Key Exchange Group(s):
TLSv1.3 128 bits secp256r1 (NIST P-256)
TLSv1.3 192 bits secp384r1 (NIST P-384)
TLSv1.3 260 bits secp521r1 (NIST P-521)
TLSv1.3 128 bits x25519
TLSv1.3 224 bits x448
TLSv1.3 112 bits ffdhe2048
TLSv1.3 128 bits ffdhe3072
TLSv1.3 192 bits x25519MLKEM768
TLSv1.2 128 bits secp256r1 (NIST P-256)

SSL Certificate:
Signature Algorithm: ecdsa-with-SHA256
ECC Curve Name: prime256v1
ECC Key Strength: 128

Subject: fba1b454-be8a-4c44-b6f5-7380fdad1ab7
Issuer: PNWL Intermediate CA G1

Not valid before: Jul 31 19:56:05 2025 GMT
Not valid after: Jan 27 19:56:05 2026 GMT
tomc@we36596:~/Downloads/CyberMESA/sslscan$

```

Figure E1. sslscan Example Output

E.3 SSH

Initiate ssh connection

```
ssh <username>@<IP>
```


Initiate ssh connection on specific port

```
ssh -p <port> <username>@<IP>
```

Initiate ssh connection to charger with verbose output (-v)

```
ssh -v <username>@<IP>
```

Note: Level of verbosity can be specified with “-v” option in command (i.e. -v, -vv, -vvv).

Initiate ssh connection to charger as root

```
ssh root@<IP> -p <port>
```

NMAP cipher enumeration scan

```
nmap -p <port> --script ssh2-enum-algos <IP>
```

SSH keypair generation

```
ssh-keygen -t ed25519 -C “$(date)” <user>@<IP>
```

Note: Algorithm can be specified in command. This example shows how an ed25519 key-pair can be generated.

SSH keypair copy to SSH server

```
ssh-copy-id -i ~/.ssh/id_ed25519.pub <username>@<IP>
```

SSH password authentication only

```
ssh -o PreferredAuthentications=password -p <PORT>  
<username>@<IP>
```

OR

```
ssh -o PubkeyAuthentication=no <PORT> <username>@<IP>
```

OR

```
ssh -o KbdInteractiveAuthentication=no <PORT> <username>@<IP>
```

Note: Each command is a different method for achieving password only authentication

SSH remote port forward

Syntax

```
ssh -R remoteport:localaddress:localport <username>@<IP>
```

Example

```
ssh -R 9999:localhost:22 <username>@<IP>
```

Appendix F - Standard Configuration Settings

F.1 TLS Parameter Configuration

This section details the recommended TLS parameters for Electric Vehicle Supply Equipment (EVSE), which may function as either a TLS server (e.g., in ISO 15118-based Plug and Charge communications) or a TLS client (e.g., in OCPP connections to a central management system). The guidance below is based on RFC 9325, which provides modern recommendations for secure use of TLS and DTLS and obsoletes RFC 7525.

TLS Versions:

- Must support TLS 1.2 (RFC 5246 and related RFCs) and TLS 1.3 (RFC 8446 and related RFCs).
- Must not support any version earlier than TLS 1.2.

Compression:

- TLS-level compression must be disabled.

Renegotiation:

- TLS renegotiation must be disabled.

Session Resumption:

- Disable session resumption (0-RTT, tickets, PSK) by default to maintain strong cryptographic guarantees and prevent session hijacking or replay risks.

TLS Server Configuration

- Examples of components that may act as TLS servers include:
 - The V2G server in ISO 15118-2, where the EVSE presents a certificate to the vehicle.
 - An MQTT broker hosted on the charger or site controller, which accepts incoming connections from clients (e.g., local devices or backend systems) and requires TLS to secure published/subscribed messages.
 - A web-based management interface, typically exposed via HTTPS, which allows administrators or technicians to configure or monitor the charger. This interface must enforce strong TLS protections to prevent credential theft, session hijacking, or configuration tampering.
- TLS 1.2:
 - Must offer AEAD cipher suites with forward secrecy, such as:
 - ♣ TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256
 - ♣ TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384
 - Offer and accept only strong key exchange groups:
 - ♣ Implementations must support at least one of the following curves:
 - x25519 (Curve25519, preferred for performance and security)
 - secp256r1 (also known as NIST P-256 or prime256v1)
 - ♣ But also contemporary implementations may offer:
 - secp384r1 (NIST P-384)
 - secp521r1 (NIST P-521)
 - x448
 - If support for legacy TLS versions, cipher suites, or cryptographic primitives is required for compatibility (e.g., with older vehicles or backend systems), it must not be enabled by default.
 - ♣ Legacy features must only be activated through an explicit configuration step by the end user or administrator.
 - ♣ The process should include a warning indicating the associated security risks (e.g., exposure to known vulnerabilities, lack of forward secrecy, or weaker encryption).
 - Should not enable weak or discouraged
 - ♣ cipher suites using obsoleted key exchange mechanisms, as described in [IETF draft: Deprecate Obsolete Key Exchange Methods in TLS](#), including. E.g.,
 - Cipher suites with static key exchange
 - TLS_RSA_WITH_AES_*
 - TLS_ECDH_RSA_WITH_AES_*
 - TLS_ECDH_ECDSA_WITH_AES_*
 - ♣ Cipher suites using CBC-mode encryption, such as:
 - TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA256
 - Cipher suites should be AES based, and exclude cipher suites using insecure algorithms, such as:

- DES ()
- 3DES (TLS_RSA_WITH_3DES_EDE_CBC_SHA)
- RC4 (TLS_RSA_WITH_RC4_128_SHA)
- Export-grade, NULL, or anonymous cipher suites, such as:
 - ⌚ TLS_RSA_EXPORT_WITH_RC4_40_MD5
 - ⌚ TLS_DH_anon_WITH_AES_128_GCM_SHA256
 - ⌚ TLS_NULL_WITH_NULL_NULL
- ❑ Message authentication code should be SHA256 or longer and not include ones
 - MD5
 - SHA-1
- TLS 1.3:
 - Follow defaults in RFC 8446; all cipher suites in TLS 1.3 are AEAD and offer forward secrecy:
 - ♣ TLS_AES_128_GCM_SHA256
 - ♣ TLS_AES_256_GCM_SHA384
 - ♣ TLS_CHACHA20_POLY1305_SHA256
 - Named Groups (Elliptic Curves):
 - ♣ Must support at least one of the following:
 - x25519
 - secp256r1 (P-256)
 - ♣ And may offer:
 - secp384r1 (NIST P-384)
 - secp521r1 (NIST P-521)
 - x448
 - ♣ Forward-looking implementations may offer hybrid key exchange algorithms that combine conventional and post-quantum cryptographic techniques:
 - X25519MLKEM768
 - SecP256r1MLKEM768
 - SecP384r1MLKEM1024
 - ♣ Must not support or negotiate weak groups, producing less than 253, such as:
 - secp192r1 (P-192)
 - secp224r1 (P-224)
 - any custom or non-standard curves

Implementations must prioritize cryptographic agility, forward secrecy, and resistance to known downgrade and side-channel attacks.

F.2 SSH Parameters

This section details the recommended SSH parameters³³ for EVSEs.

Authentication:

- Public-Key Authentication is enabled

Cryptographic Methods:

- **KexAlgorithms:**
 - o Allowed key exchange (KEX) algorithms:
 - ♣ `curve25519-sha256`,
 - ♣ `curve448-sha512`,
 - ♣ `ecdh-sha2-nistp256` (and stronger NIST curves such as `nistp384` or `nistp521`), and
 - ♣ `diffie-hellman-group14-sha256` (and stronger SHA-2-based groups, e.g., `group16`, `group18s`).
 - o Post-quantum cryptography (PQC) or hybrid PQC KEX algorithms (where supported):
 - ♣ `mlkem768x25519-sha256`, `mlkem768nistp256-sha256` (and stronger traditional KEX / ML-KEM parameter sets), and
 - ♣ [sntrup761x25519-sha512@openssh.com](https://cvs.openssh.com/ssh-keyexchanges.txt)

Note: Post-quantum and hybrid KEX require an OpenSSH client and server version supporting these algorithms (e.g., OpenSSH 9.6+, with PQ hybrid KEX stabilized in OpenSSH 10.0+).
 - o Prohibited

³³ Acceptable SSH cryptographic algorithms and associated configuration guidance are informed by the following authoritative standards:

- NIST SP 800-131A Rev. 2, Transitioning the Use of Cryptographic Algorithms and Key Lengths.
- NIST SP 800-57 Part 1 Rev. 5, Recommendation for Key Management: General.
- RFC 8731, Secure Shell (SSH) Key Exchange Method Using Curve25519 and Curve448.

Standardizes `curve25519-sha256` and `curve448` as preferred elliptic-curve key exchange methods.

- RFC 8709, Ed25519 and Ed448 Public Key Algorithms for the Secure Shell (SSH) Protocol.

Defines `ssh-ed25519` and `ssh-ed448` as preferred host key algorithms.

- RFC 9142, Key Exchange (KEX) Method Updates and Recommendations for Secure Shell (SSH).

Specifies recommend algorithms for SSH.

- OpenSSH Security

<https://www.openssh.com/security.html>

- Mozilla OpenSSH security hardening guidelines.

<https://infosec.mozilla.org/guidelines/openssh>

- RFC 9457, Hybrid Post-Quantum Key Exchange for SSH.

Defines hybrid PQ KEX mechanisms, including `sntrup761x25519-sha512@openssh.com` and `mlkem768x25519-sha256`.

- ♣ Any KEX using SHA-1
- ♣ Any finite-field DH group < 2048 bits
- ♣ Any unlabeled or legacy KEX identifiers
- **HostKeyAlgorithms:**
 - o Allowed algorithms:
 - ♣ ssh-ed25519,
 - ♣ ssh-ed448,
 - ♣ rsa-sha2-256 (RSA key size ≥ 3072 bits), and
 - ♣ rsa-sha2-512 (RSA key size ≥ 3072 bits)
 PQC algorithms are allowed:
 - ssh-mldsa-44 (and high-security parameter sets)
 - o Prohibited:
 - ♣ ssh-rsa (SHA-1 signature; deprecated)
- **Ciphers:**
 - o Allowed authenticated encryption ciphers :
 - ♣ [aes128-gcm@openssh.com](https://ciphersuite.openssh.org/AES128GCM.html) (and larger key sizes), and
 - ♣ [chacha20-poly1305@openssh.com](https://ciphersuite.openssh.org/ChaCha20Poly1305.html)
 - o Prohibited :
 - ♣ RC4, Blowfish, 3DES, CBC and any non-AEAD cipher suites
- **MACs:**
 - Allowed encrypt-then-MAC constructions:
 - [umac-64-etm@openssh.com](https://ciphersuite.openssh.org/UMAC64ETM.html) and [umac-128-etm@openssh.com](https://ciphersuite.openssh.org/UMAC128ETM.html), and
 - [hmac-sha2-256-etm@openssh.com](https://ciphersuite.openssh.org/HMACSHA256ETM.html) (and larger key sizes)
 - Prohibited:
 - Any *-sha1 MAC
 - Any MAC not using ETM (Encrypt-Then-MAC)

Scoring Criteria

Score	Condition	Description
2 – compliant	<i>Only</i> cryptographic elements that are explicitly listed as Allowed are present	The implementation uses approved algorithms, key strengths, modes, and parameters as defined in the policy. No deprecated, discouraged, or unreviewed primitives appear in configuration, negotiation, or operation.

0 – Non-Compliant

One or more **Prohibited** cryptographic elements is present, enabled, negotiable, or otherwise available for use; or only unreferenced items

The presence or negotiability of prohibited algorithms or parameters introduces unacceptable risk. This requires corrective action before the system can be considered secure or deployed in production.

Appendix G - x.509 Certificate Issuance

The following sections assume OCPP is used for certificate management. However, it is important to note that operators may employ other methods for certificate installation and management.

There are three certificates:

- WebPKI
- SAE TCL
- OCPP private PKI

Root CA

1. Obtain V2G root CA from authorized V2G PKI provider or generate self-signed CSMS root CA

Note: The V2G root CA can be used for both OCPP client and server (ISO 15118) certificates. However, a separate CSMS Root CA should be used for the EVCS OCPP TLS client certificate.

2. Send `InstallCertificateRequest` to the charger, containing the new root CA certificate with `CertificateType = CSMSRootCertificate` for OCPP (TLS) or `V2GRootCertificate` for ISO 15118 (TLS).

Note: Follows Use Case M05³⁴ in OCPP 2.0.1.

³⁴ Use Case M05 defines CA certificate installation in a charging station. It can also be used to install the eMSP Root certificate for Authorization (during PnC) and OEM Root certificate for firmware updates.

Generate Self-Signed Root CA

Note: This is specifically for generating the CSMS Root CA used for signing the CSMS and Charging Station TLS certificates used in OCPP. You can create your own “mock V2G Root” with to test Plug & Charge, but it will not be trusted by real EVs.

1. Create a directory for the files:

```
mkdir private certs
```

2. Create a CA key:

```
openssl ecparam -out private/<private_key_file_name>.pem -name  
prime256v1 -genkey
```

3. Create a CA config file with appropriate entries

4. Generate a CA certificate:

```
openssl req -new -x509 -days 365 -config <ca_config_file> -  
extensions v3_ca -key <private_key_file> -out certs/<  
cert_file_name>.pem
```

Charging Station TLS Server Certificate

Note: Follows the alternative scenario described in Use Case A02 in OCPP 2.0.1.

1. Obtain `SeccIds` from charger (`GetReportRequest`).
Note: Repeat the following steps for each `SeccId`
2. Request CSR from charger.
 - a. The CSMS requests the Charging Station to update its certificate using the `TriggerMessageRequest` with the `requestedMessage = SignV2GCertificate`
3. Forward CSR to CA to be signed
4. Return signed certificate to charging station in `CertificateSignedRequest`

Charging Station OCPP Client Certificate

Note: Follows Use Case A02 of OCPP 2.0.1.

1. Request CSR from charger
 - a. The CSMS requests the Charging Station to update its certificate using the `TriggerMessageRequest` with the `requestedMessage = SignChargingStationCertificate`
2. Forward CSR to CA to be signed

3. Return signed certificate to charging station in `CertificateSignedRequest`